Theory of Computing

Matthew Barnes, Mathias Ritter

Books 4

Automata theory: Regular languages 5
- Deterministic Finite Automata 5
  - Alphabets and strings 5
  - Languages 6
  - Deterministic finite automata (DFAs) 6
  - Product construction 9
  - Regular languages 10
- Nondeterminism, subset construction and ϵ-moves 15
  - Nondeterminism 15
  - Subset construction 18
  - ϵ-moves 21
- Regular expressions and Kleene's theorem 23
  - Defining regular expressions 23
    - Atomic patterns 23
    - Compound patterns 24
    - Examples 24
  - Kleene's theorem 24
  - Reg exp to εNFA 25
  - εNFA to reg exp 26
- Limitations of regular languages 29
  - Pumping (🎃) Lemma for regular languages - contrapositive form 30

Automata theory: Context free languages 34
- Pushdown Automata (PDA) 34
  - Transition relation 34
  - Configuration 35
  - Acceptance 35
    - By final state 36
    - By empty stack 36
  - Example PDA accepting palindromes 36
- Context free grammars 38
  - Chomsky Normal Form 41
  - Greibach Normal Form 42
  - Removing epsilon and unit productions 42
  - Removing left-recursive productions 42
  - Converting between CFG, CNF and GNF 44

Converting a CFG to CNF 44
Converting a CFG to GNF 46
PDA and CFG conversions 49
CFG to PDA 49
PDA to CFG 51
Proving closure 58
Union 58
Concatenation 58
Kleene star 59
Intersection with regular languages 59
Limits of context-free languages 60
Pumping (🎃) Lemma for context-free languages - contrapositive form 63

Computability theory **67**
Turing machines 67
Decidability 68
Recursive and R.E languages 68
Proof: Recursive Sets Closed under Complement 68
(Semi) Decidable properties 70
Universal Turing machines 71
Multiple tapes 71
Simulating Turing machines 71
Encoding Turing machines 71
Constructing a UTM 72
Halting problem 72
Proof by contradiction 72
Proof by diagonalisation 73
Decidable / Undecidable problems 74
Reductions 77
Properties of reductions 82
Rice's theorem 83

Complexity theory **85**
Big-O and Big-Θ 86
Big-Θ 86
Big-O 86
The class P 87
Feasible and infeasible problems 87
Decision problems 87
Definition 88
The PATH problem 89
The HAMPATH problem 90
Regular and context-free languages 90

The class NP 91
- Non-deterministic Turing machine 91
- Time complexity of Turing machines 92
- Definition 95
- The SAT problem 95
- The HAMPATH problem 96
- The TSP(D) problem 97
- The 3COL problem 97
- NP = P? 97
- Presburger arithmetic 98

NP-Completeness 99
- Polynomial time reduction 99
- 3COL to SAT 100
- NP-hard 101
- The Cook-Levin theorem 101
- What is NP-completeness 101

Space complexity 101
- Space complexity 101
- PSPACE 103
- NPSPACE 103
- Savitch's Theorem 104
- EXPTIME 104
- Relationships between complexity classes 104
- PSPACE-hard 104
- PSPACE-completeness 105

TL;DR 105

Automata theory: Regular languages 105

Automata theory: Context free languages 106

Computability theory 107

Complexity theory 108

Books

- There are two main books for this module. I've added PDF links to both of them.
    - Dexter C. Kozen, Automata and Computabilty, Springer, 1999 (Kozen)
    - Sipser M, Introduction to the Theory of Computation, PWS, 1997 (Sipser)

Automata theory: Regular languages

- Automata relates to machines with finite memory.
- Unlike maths, where we can have infinite of something, we cannot have infinite of something in an automata because it'll run out of memory.

- Automata also relates to pattern matching, modelling, verification of hardware etc.

Deterministic Finite Automata

Alphabets and strings

- An **alphabet** is a finite set of elements. It is denoted with capital sigma, Σ.
- Examples:
  - Σ = {0, 1}
  - Σ = {a, b, c}
  - Σ ≠ {1, 2, 3, 4, ...}

- A **string** 's' over Σ is a finite sequence of elements of Σ.
- Two strings 's' and 't' are equal when they have the same elements of the same order.
- Examples:
  - Σ = {0, 1}, s = "0100110"
  - Σ = {a, b}, s = "abbaba"

- The **set of all strings** is denoted with sigma star, Σ*.
- It will always be infinite, unless the alphabet is empty.
- In every alphabet, including an empty alphabet, the set contains the **empty string** 'ϵ'.
- More names for Σ* are "free monoid on Σ", or "Kleene star of Σ".
- Examples:
  - Σ = {}, Σ* = { ϵ }
  - Σ = {a}, Σ* = { ϵ, a, aa, aaa, aaaa ... }
  - Σ = {0, 1}, Σ* = { ϵ, 0, 1, 10, 11, 100, 001, 000 ...}
- We can clean up our notation by not including Σ, but just replacing it with its definition:
  - {}* = { ϵ }
  - {a}* = { ϵ, a, aa, aaa, aaaa ... }
  - {0, 1}* = { ϵ, 0, 1, 10, 11, 100, 001, 000 ...}

- The **length** # of a string is its number of elements.
- Examples:
  - #(abba) = 4
  - #(00101) = 5

    - #(joestar) = 7

- The **empty string** is usually denoted with a lunate epsilon character, ϵ.
- Laws:
    - ϵ = “”
    - #(ϵ) = 0
    - ϵϵ = ϵ

- You can **concatenate** two strings, ‘s’ and ‘t’, like this: ‘st’.
- Example:
    - s = “ab”, t = “ba”, st = “abba”
    - s = 011, t = 101, st = 011101
    - s = “za”, t = “warudo”, st = “zawarudo”
    - s = ϵ, t = ϵ, st = ϵ
    - ϵs = sϵ = s

Languages

- A **language** L over an alphabet Σ is some subset of strings in Σ*.
- Examples:
    - Σ = {a, b, c}
    - L_1 = {ϵ, a, bb, cba}
    - L_2 = {bbb, aaa}
    - L_3 = {cccccacccccc}
    - L_4 = Ø = {}
- Two languages are equal when they contain the same strings (in terms of sets, they are equal).
- If the language should also contain the empty string ϵ it must be explicitly added to the language as an element of the set.

Deterministic finite automata (DFAs)

- A **finite automata** is a mathematical model of a system with a finite number of states and a finite amount of memory space.
- A **state** is a description of a system at some point in time.
- A **transition** allows the DFA to go from one state to the other.
- Think of DFAs like flow-charts; you go from one state to the next until you reach the goal state.
- There are three kinds of finite automata:
    - Deterministic
    - Nondeterministic
    - Nondeterministic with ϵ-moves
- Right now, we’ll focus on *deterministic* finite automata.

- A DFA (deterministic finite automata) has 5 properties:

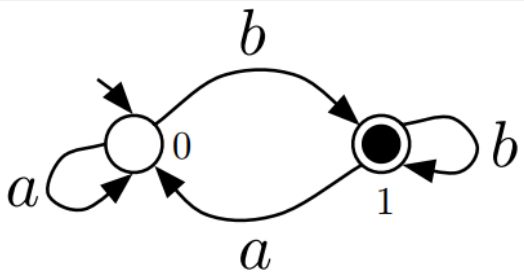
Property	Symbol	Description

States	Q	Possible states that the automata can be in.
Alphabet	Σ	The alphabet that the transition names depend on.
Transitions	δ	A function that allows the automata to go from one state to the other by consuming a symbol. This takes an element of Σ and a state from Q, and outputs the next state from Q to transition to.
Start state	s	The state that the automata starts at.
Final states (accept states)	F	Set of states at which, if the automata ends at them, it accepts the string that was entered into it.

- To define a DFA mathematically, use the properties above into a tuple:
- Example, given that M is a DFA:
    - M = (Q, Σ, δ, s, F)

- An automata takes in a string that contains letters from the alphabet and reads it from left to right, with each letter being an action (or transition) that changes the state of the automata.
- All states **must** have all possible transitions, i.e. each state must have exactly one transition for every element of the alphabet
- If the automata **ends on a final state**, it'll **accept** the string.
- If the automata **does not end on a final state**, it'll **reject** the string.

- You can also represent DFAs as graphs (pictures), similar to state-machine diagrams:



$$Q = \{0, 1\}, \Sigma = \{a, b\}$$

$$s = 0, F = \{1\}$$

$$\delta(0, a) = 0, \delta(0, b) = 1$$

$$\delta(1, a) = 0, \delta(1, b) = 1$$

	a	b
$\rightarrow 0$	0	1
1F	0	1

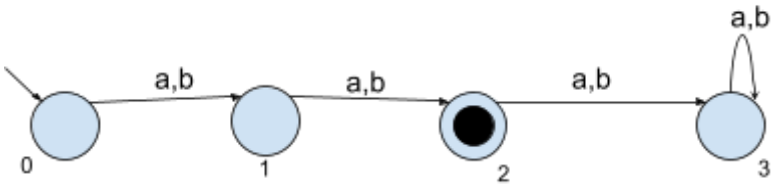
- This small arrow pointing to the '0' state coming from nowhere means that '0' is the initial state.
- The states that are not filled in are not final states.
- The states that are filled in are final states.
- The arrows represent the transitions between states, which are triggered by the input character shown

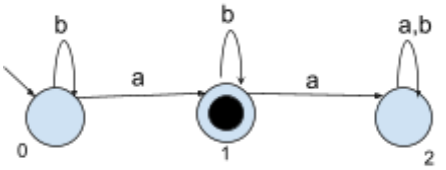
- This DFA accepts strings like abbab and babbbab, but not abba or ϵ.
- To say that a string 's' takes the DFA from state 0 to state 1, you would write that as:
  - $0 \xrightarrow{s} 1$
  - Where:
    - Left-hand side = beginning state
    - Right-hand side = ending state
    - Text on the top = string transition
- More examples:
  - $0 \xrightarrow{abb} 1$
  - $1 \xrightarrow{bba} 0$
  - $1 \xrightarrow{b} 1$

Product construction

- Product construction is where you merge two DFAs together, so that the combined DFA is actually the two component DFAs running in parallel.
- The states aren't just single string / character names. The states are referenced in a tuple (or pair), with each element being a state name from one of the component DFAs.

- Let's define the DFAs of L_1 and L_2 as this, respectively:





- Now, if we found the product of the DFA of L_1 and the DFA of L_2, then the states of that product would be:
- (0, 0), (0, 1), (0, 2), (1, 0), (1, 1), (1, 2), (2, 0), (2, 1), (2, 2), (3, 0), (3, 1), (3, 2)

- The product transition would need a pair state and a transition name. For example, you could go from (2,0) to (3,1) by using 'a', or you could go from (0,0) to (1,0) by using 'b'.

- The set of final states F is $F_1 \times F_2$, where F_1 is the set of final states of L_1 and F_2 is the set of final states of L_2. Therefore, with normal, unedited product construction, for the DFA to accept the string, we need to be in a final state for both component DFAs.
- This can be changed, however, to just "at least one DFA to be on a final state" if we define F to be $(Q_1 \times F_2) \cup (F_1 \times Q_2)$, where Q_x is the set of states for a DFA.

- Therefore, the language expressed by the product of L_1 and L_2 is $L_1 \cap L_2$.

- It's best to view this as a table, where you can see the DFAs of L_1 and L_2 simultaneously:

L_1's DFA state	**L_2's DFA state**	**Product state**	**Transition**	**Step**
0	0	(0,0)	None (yet)	0
1	1	(1,1)	a	1
2	1	(2,1)	b	2

3	2	(3,2)	a	3

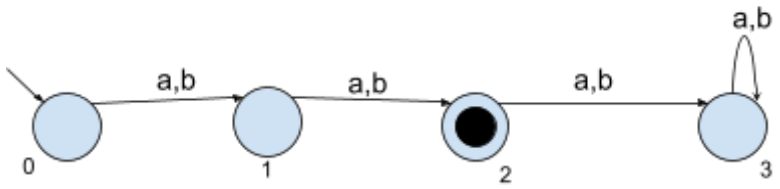
- To view this more simply, you can just follow two DFAs at the same time, and put one's state on the left of the tuple and the other's state on the right of the tuple. It's nothing more than running two DFAs at the same time, inputting the same word into both of them.
- If you're confused check this helpful video out.

Regular languages

- When you have a DFA called M, then L(M) is the language of all accepted strings.
- For example, in the previous DFA, L(M) = {b, ab, aab, bb, abb ...} and it will not include elements like 'a', 'ϵ', 'ba' or 'bba' because they all will not be accepted by that DFA.

- A **regular language** L is when there exists some DFA M such that L(M) exists.
- More informally, a language L is regular if you can come up with some DFA where it'll accept *all* the words in L and *only* those words in L.

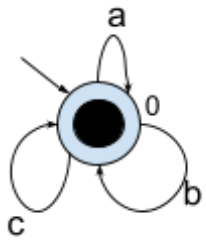
- For example, the language { ab, ba, aa, bb } is regular because I can construct a DFA that accepts only those words:



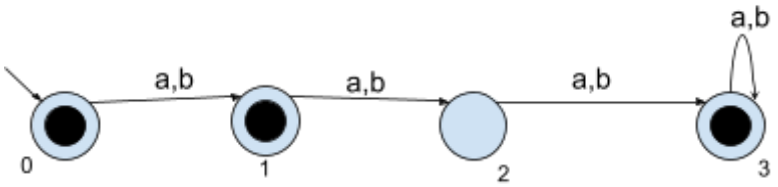
- All finite languages are regular, because we could (in theory) just build a control flow tree of constant size (you can visualize it as a bunch of nested if-statements that examine one digit after the other) (citation).

- The empty language Ø is regular, because you can create a DFA that can't accept anything (for example, a DFA with no final states), not even the empty string. If a DFA can't accept any strings, then the L(M) of that DFA would be the empty set Ø, making Ø regular.

- The set of all strings Σ* is regular, because you can set up a DFA consisting of a single final state that transitions to itself for any symbol. This accepts any element from the alphabet Σ, like this one when Σ = {a, b, c} and Σ* = {ϵ, a, b, c, ab, bc, abc, cba ...}:



- If L is regular, then the complement (Σ* - L, or ~L) is always regular.
- For example, let's take the language L = {ab, ba, aa, bb} from before because we know that's regular (therefore there must be a DFA whose L(M) is L). The alphabet that it's from is Σ = {a, b}. The complement of L, ~L, will be the set of all strings from Σ* except for the ones in L.
- If we take the DFA whose L(M) is L from before and flip all the final states to non-final states and vice versa, we can create a new DFA whose L(M) is ~L (citation).



- This works on any language. We're simply getting the "inverse" of this DFA.

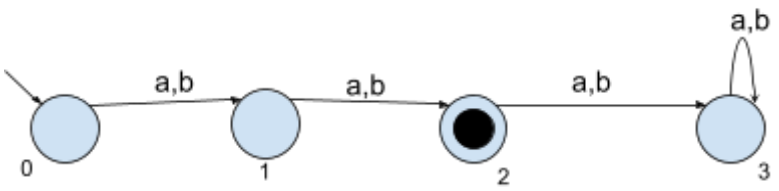
- If L_1 and L_2 are regular, then the union, $L_1 \cup L_2$, is regular.
- If we want to prove that $L_1 \cup L_2$ is regular, we need to show a DFA exists for $L_1 \cup L_2$.
- We can use product construction for L_1 and L_2 , then we can modify it a bit to suit what we need.
- First of all, let's define the result of the product of $L_1 \cup L_2$ to be M.
- Now, we change the set of final states F of M to be "at least one DFA must be on a final state to accept the string", so we define F to be $(Q_1 \times F_2) \cup (F_1 \times Q_2)$, where Q_x is the set of states for a DFA.
- By doing that, we have now defined a DFA where L(M) is $L_1 \cup L_2$.

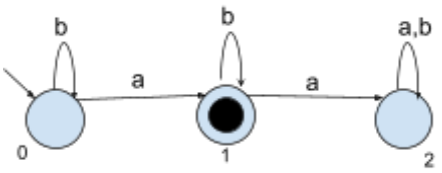
- If L_1 and L_2 are regular, then the intersection, $L_1 \cap L_2$, is regular.
- This one's even easier than the union one.
- Basically, we use product construction for the DFAs L_1 and L_2.
- That's it. We don't even need to change the set of final states. This is because, by default, the product of two DFAs require both DFAs to be on a final state for the aggregate DFA to accept the string, which is the intersection.

- If L_1 and L_2 are regular then $L_1 L_2 = \{ xy \mid x \in L_1, y \in L_2 \}$ is regular.
- This proof requires NFAs (nondeterministic finite automata), which are covered in more detail later, but basically it's the same as DFA except you can have multiple transitions in NFAs, and you can take multiple paths at the same time. Think of it like two threads running in parallel, or two trains travelling on different paths in parallel.

It's similar to a fork node in an activity diagram in UML and how it models concurrency in a system.

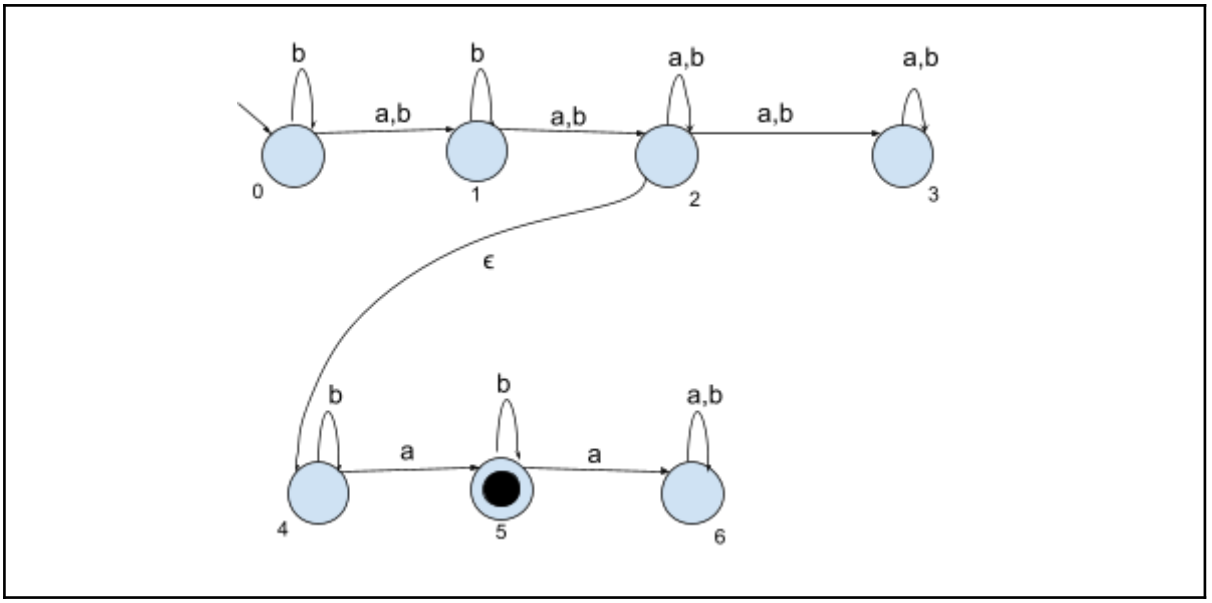
- First, we define two DFAs for L_1 and L_2:
  - $M_1 = (Q_1, \Sigma, \delta_1, s_1, F_1)$
  - $M_2 = (Q_2, \Sigma, \delta_2, s_2, F_2)$
- Now, we're going to create an NFA from these two DFAs:
  - $M_3 = (Q_1 \cup Q_2, \Sigma, \delta_1 \cup \delta_2, s_1, F_2)$
  - and for each state $q \in F_1$, $\delta(q, \epsilon) = s_2$ (what this means is, for each final state of the first DFA, make a transition that goes from that final state to the starting state of the second DFA). The transition has the name 'ϵ', which means that we can take this transition without using up any characters in the string, it's basically a free move, but more on that in a later topic. See 'ϵ-moves').
- What this NFA really means is that we start off in terms of M_1, but we need to finish off in terms of M_2.
- For example, take the two example DFAs for a second:



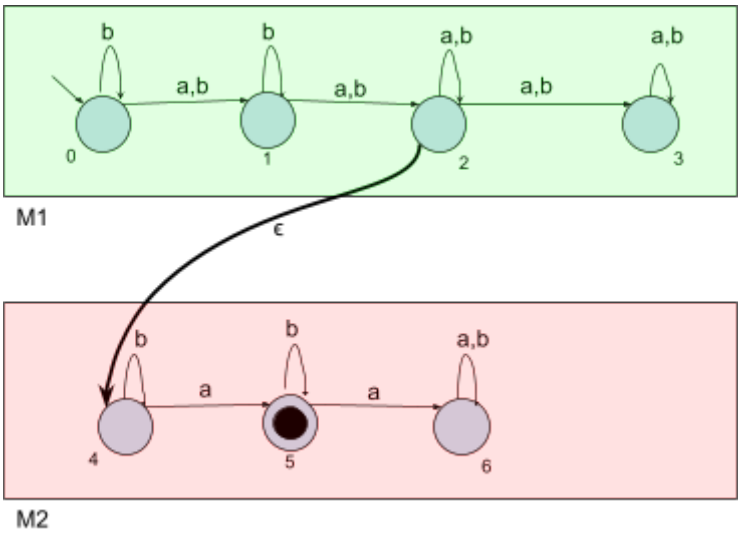


- If we define a new NFA, like we did above, for these two example DFAs, it'll look something like the illustration below. I've created two illustrations of the same NFA: one without annotations, and one with annotations.

The proper one



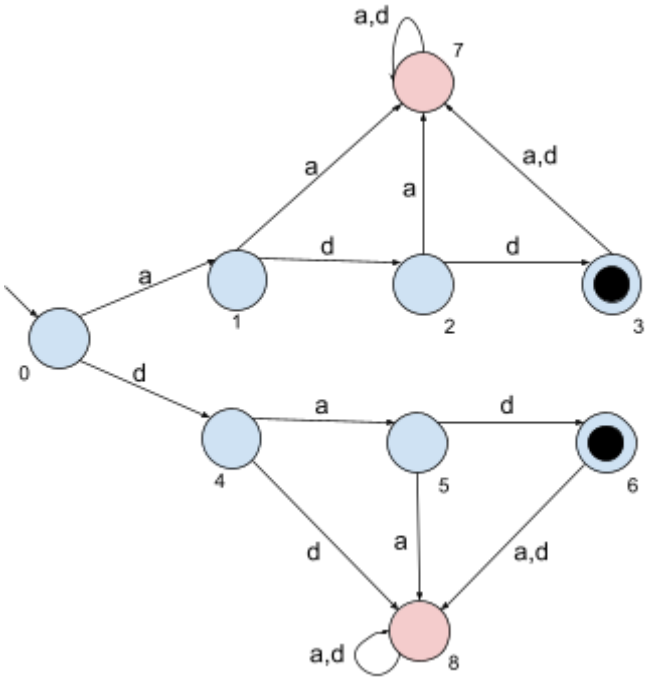
The illustrative one



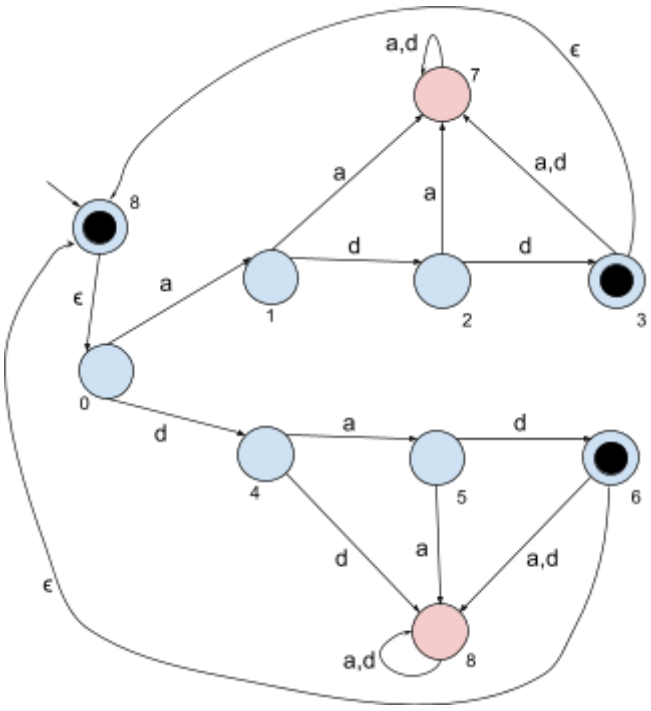
- First of all, the top half of this NFA is basically M_1. However, when we reach M_1's final state, there's actually an epsilon move taking the NFA to its bottom half. If we don't reach that epsilon move, we can't move onto the bottom half of the NFA. When we reach that epsilon move, the string (up to this point) should be one of M_1's accepted strings.
- Second, the bottom half of the NFA is M_2. Once we've reached the bottom half, we've found an M_1 string, now we just need an M_2 string. Once we've got an M_2 string, we'll be on a final node and the string will be accepted.
- By reaching the final node, we know that the input string is made up of an M_1 accepted string as the first half, and an M_2 accepted string as the second half. This is literally the definition of L_1 L_2, so we can safely say that the L(M) of this NFA is L_1 L_2.
- This NFA should accept any concatenated string from L_1 and L_2. Therefore, L_1 L_2 is regular.

- If L is regular then $L^* = \{ x_1 \ldots x_k \mid k \in N, x_i \in L \}$ is regular.
- L* is basically the set of every possible word you can make by concatenating the words of L onto themselves.
- So if L had “add” and “dad”, L* would have words like “adddad”, “dadadddad”, “daddaddad” etc. and it would go on infinitely.
- The operator * is called “Kleene star”, and we’ve been using it already (when we write Σ* to refer to the set of all words of that alphabet).

- Yet again, we’ll need NFAs to prove this.
- Let’s use my “add dad” example above, and construct a DFA, M, where L(M) is L:



- The red nodes are “error nodes”, where the DFA goes when we know the string will be rejected.
- This accepts “add” and “dad”, but we need it to accept words like “adddad” as well.
- Luckily, epsilon-moves allow us to very easily construct an NFA that does this:



- Now, when we complete the word “add” or “dad”, we can move straight back to the start node and search for “add” or “dad” again. Remember, epsilon moves don’t require any characters to use, so we can go straight from the end of “add” to the start of “dad”, for example.
- The reason why we create a new final start state that immediately goes to the old start state is because Kleene star supports the empty string (in the picture, it’s state 8)
- To more generalise this proof, let’s define a general DFA:
    - $M = (Q, \Sigma, \delta, s', F \cup s')$
- Now, we set epsilon moves branching from all the final states to the new final starting state:
    - $\forall q \in F, \delta(q, \epsilon) = s'$
- (For all states q in the set of final states F, there is a transition function where when the NFA is in state q, and the empty epsilon move ϵ is input, the state transitioned to is the new final start state s’.)
- That’s all there is to this proof.

Nondeterminism, subset construction and ϵ-moves

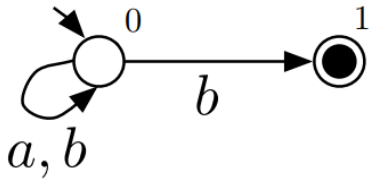
Nondeterminism

- With a DFA, there is always a next state to go to when you input a letter.
- However, what if there are cases where there isn’t always a next state to go to? What if there’s multiple states we can go to?
- That’s what an NFA is (nondeterministic finite automata), it’s basically a DFA that can either branch off to multiple states, or fail if there are no states to go to. In other words, it guesses where it’s supposed to go. If the guesses are wrong, they “die”.

- In other words, a state no longer has |Σ| transitions, where each transition is every letter of the automata's alphabet.
- The whole gimmick of an NFA is that we can go through different "paths" simultaneously. Some paths can die, some paths can survive and reach final states. We only need one path to be on a final state to accept the string.

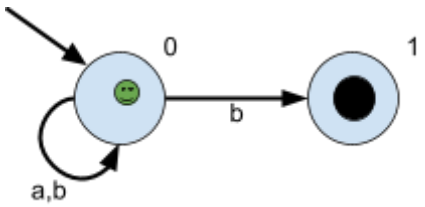
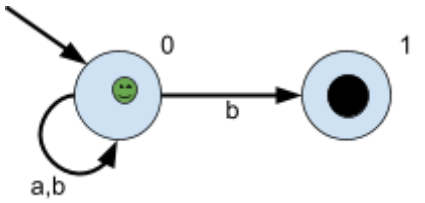
- "It is helpful to think about this process in terms of guessing and verifying. On a given input, imagine the automaton guessing a successful computation or proof that the input is a "yes" instance of the decision problem, then verifying that its guess was indeed correct." Source: Kozen textbook

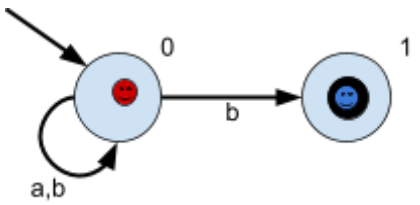
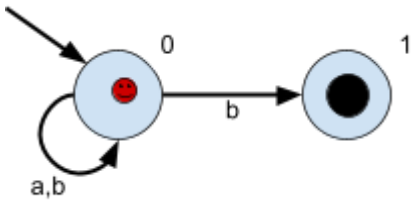
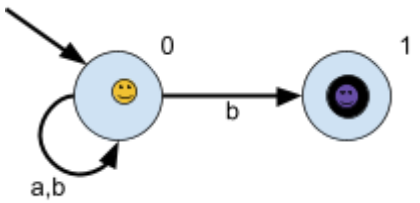
- Example:



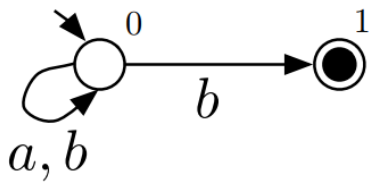
- As you can see, it's slightly different from a DFA:
  - There are two 'b' transitions in state '0'
  - There are no transitions in state '1'

- Let's walk through this NFA to get a feel for how NFAs work:

Step	Description	String so far	Illustration
1	So far, we haven't gone anywhere. The arrow that seems to come from nowhere indicates where we are initially, so we start at state '0'. The smiley faces indicate different "paths" that the NFA is going through. A different coloured face is a different path, and their journey is displayed on the right, for example the green path has just started on 0, so a green '0' is shown on the right.	ε 0	
2	Let's try inputting 'a' and see where that leads us. There's only one transition we can go down if we input 'a', so no new simultaneous paths are created; we just stick with our original one. The 'a' path just goes from 0 to 0, so our green path goes from '0' to '0'.	a $0 \rightarrow 0$	

3	Let's try inputting a 'b' and see where that leads us. There are two paths that 'b' can take us, from 0 to 0 or from 0 to 1. Here, our green path splits into two, one red and one blue. The red one picks the path that stays on 0, and the blue path picks the one that goes to 1. Right now, there exists a path that is on a final state: the blue one. Therefore, if we left the string as just ab, this NFA will accept that string.	ab $0 \rightarrow 0 \rightarrow 0$ $0 \rightarrow 0 \rightarrow 1$	
4	Let's try inputting another 'b' and see where that leads us. The red and blue paths will both need to go through a 'b' transition. Since the blue path doesn't have a blue transition where it is, it'll 'die'. The red path needs to go down a 'b' transition, too. There are two transitions that the red path can go down, the one that goes from 0 to 0 and the one that goes from 0 to 1. Like last time, the red path splits into two different paths: a yellow path and purple path. The yellow path takes the transition from 0 to 0, therefore the yellow path is still on 0. The purple path takes the transition from 0 to 1, therefore the purple path is now on 1. The NFA should accept this string too, since the purple path is on a final state.	abb $0 \rightarrow 0 \rightarrow 0 \rightarrow 0$ $0 \rightarrow 0 \rightarrow 0 \rightarrow 1$	1. When blue path dies  2. When red path splits 

- Now, let's formally construct an NFA:
  - $M = (Q, \Sigma, \Delta, s, F)$
- where:
  - Q is the set of all states
  - Σ is an alphabet
  - $\Delta$is the transition function $Q \times \Sigma \rightarrow 2^Q$, which basically means input a state and a letter from the alphabet and get a set of all states we could be in (see 'Subset construction')
  - s is the start state
  - F is the set of final states
- Let's formally construct the NFA as shown above:

$M = (Q, \Sigma, \Delta, s, F)$ $Q = \{0, 1\}$ $\Sigma = \{a, b\}$ $\Delta(0, a) = \{0\}$ $\Delta(0, b) = \{0, 1\}$ $\Delta(1, a) = \emptyset$ $\Delta(1, b) = \emptyset$ $s = 0$ $F = \{1\}$	

- There is some notation that is worth noting:
- When you see something like:

$$0 \xrightarrow{ab} 2$$

- This means that the NFA will go from 0 to 2 if you input ‘a’, and then ‘b’. There should be some intermediate state, either ‘0’, ‘2’, or even ‘1’, that you enter when you input ‘a’ and leave when you input ‘b’.

Subset construction

- It’s easy to convert DFAs into NFAs, because DFAs are a subset of NFAs.
- But how do you convert NFAs to DFAs?
- You use subset construction. Basically, each state is a subset of states that we could’ve gone to in our NFA counterpart.
- For example, in our NFA, if we’re in state 0 and by inputting ‘b’ we can either go to 0 or go to 1, in the DFA we’ll actually go to the state {0,1} because we can go to either 0 or 1 in the NFA.

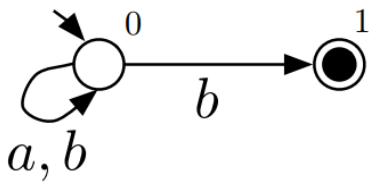
- Let’s define an NFA M_N, then let’s create a DFA M_D which is based on M_N:
  - $M_N = (Q, \Sigma, \Delta, s, F)$
  - $M_D = (Q', \Sigma, \delta, s, F')$
    - $Q' = 2^Q$
    - $\delta(X, \sigma) = \bigcup_{q \in X} \Delta(q, \sigma)$
    - $F' = \{X \mid \exists f \in F. f \in X, X \in Q'\}$

- This may look like an alien language to you now, but I’ll go through each line and explain it in plain English, then I’ll apply subset construction to the example above.
- First things first, what’s going on up there?

Line in weird maths language	**Line in plain English**
$M_N = (Q, \Sigma, \Delta, s, F)$	Here, we create an NFA with the parameters Q, Σ, Δ, s and F.

$M_D = (Q', \Sigma, \delta, s, F')$	Here, we create a DFA with the parameters Q', Σ, δ, s and F'. The parameters Q', δ and F' are all dependent on the parameters for the NFA M_N.
$Q' = 2^Q$	Q is the set of all states, like $\{0, 1\}$. However, 2^Q is the powerset of Q, so it's the set of all subsets of Q, so for example, $\{\emptyset, \{0\}, \{1\}, \{0, 1\}\}$ Because of this, the states in the DFA is all the subsets of Q, so you could have a state $\{0\}$, a state $\{0, 1\}$ or even a state $\emptyset$.
$\delta(X, \sigma) = \bigcup_{q \in X} \Delta(q, \sigma)$	From before, we know that δ is a transition function that defines how we go from one state to the other. Therefore, the inputs must be a **state** and a **letter**. X is our state input. It's actually a set, which makes sense, because the states in our DFA are all subsets of Q. σ is our letter input. Nothing strange here; it's just a letter within the alphabet Σ. Don't be too confused with the $\bigcup_{q \in X}$ that comes after. This simply means that, for each NFA state in our input X, we're going to apply the NFA transition function on that state with our input letter σ and put the result in our state output. More formally, $\bigcup_{q \in X}$ is a bit like the sum in maths $\sum_{i=0}^{k} f(k)$, except instead of adding all the terms, it unions all the terms. Remember that the transition function also outputs a state. Since the states of our DFA are subsets of Q, the DFA transition function δ also outputs a subset of Q. For example, let's say we input $\delta(\{0, 1\}, a)$. The output will be $\Delta(0, a) \cup \Delta(1, a)$. I'll go over this in more detail in our example below, but the point is that this transition function just transitions all the states in X and unions them all together into one big output.
$F' = \{X \mid \exists f \in F. f \in X, X \in Q'\}$	The set of final states F' is a subset of Q'. For a DFA state to be a final state, it needs to contain an NFA state that is in F. For example, if 0 is a non-final state and if 1 is a final state in the NFA, then in the DFA, {0} would not be a final state, but {0,1} and {1} would be final states. That's basically what the left expression gets; a subset of Q' where all elements contain at least one final state from F.

- Now that we've got the formalities out the way, we can work on an example. Let's convert the NFA above into a DFA.
- As said above, we'll define our DFA as $M_D = (Q', \Sigma, \delta, s, F')$ and our NFA as $M_N = (\{0, 1\}, \{a, b\}, \Delta, 0, \{1\})$, so we'll go through this step-by-step by explaining how we get each parameter for the DFA M_D:



<table>
<tr><th>What we're defining</th><th>How we did it</th></tr>
<tr><td>Q'</td><td>The value of Q is $\{0, 1\}$, so we just need to get the powerset of $\{0, 1\}$, or the 'set of subsets'. This is:

$Q' = \{\emptyset, \{0\}, \{1\}, \{0, 1\}\}$</td></tr>
<tr><td>Σ</td><td>This is just the same as before, nothing new here:

$\Sigma = \{a, b\}$</td></tr>
<tr><td>δ</td><td>First of all, let's create a transition table. This is a table where the input letters go on the top, the states go on the left, and you can look up transition results by looking up a state and a letter:
<table>
<tr><th></th><th>a</th><th>b</th></tr>
<tr><td>$\emptyset$</td><td></td><td></td></tr>
<tr><td>$\{0\}$</td><td></td><td></td></tr>
<tr><td>$\{1\}$</td><td></td><td></td></tr>
<tr><td>$\{0, 1\}$</td><td></td><td></td></tr>
</table>
Now we work through this table from top to bottom. First of all, there's the empty set state. You can't go from any state from the empty set state; it's practically a death state. So just input all empty set states here.
<table>
<tr><th></th><th>a</th><th>b</th></tr>
<tr><td>$\emptyset$</td><td>$\emptyset$</td><td>$\emptyset$</td></tr>
<tr><td>$\{0\}$</td><td></td><td></td></tr>
<tr><td>$\{1\}$</td><td></td><td></td></tr>
<tr><td>$\{0, 1\}$</td><td></td><td></td></tr>
</table>
Now we're moving onto the singleton states. For $\{0\}$ and a, just have a look at the NFA and see what states you'd end up at if you were at 0 and you took the paths labelled 'a'. As you can see, you'd just end up back at 0, so we just input $\{0\}$ there. As for $\{0\}$ and b, you can see that you can go to 0 or 1 using a 'b' path on 0, so we can input $\{0, 1\}$there. Continue this until we're done with all the singleton states.
<table>
<tr><th></th><th>a</th><th>b</th></tr>
<tr><td>$\emptyset$</td><td>$\emptyset$</td><td>$\emptyset$</td></tr>
<tr><td>$\{0\}$</td><td>$\{0\}$</td><td>$\{0, 1\}$</td></tr>
<tr><td>$\{1\}$</td><td>$\emptyset$</td><td>$\emptyset$</td></tr>
</table>
</td></tr>
</table>

<table>
<tr><td></td><td>
<table>
<tr><td>$\{0, 1\}$</td><td></td><td></td></tr>
</table>

Now we're faced with a state with more than one element! Remember the definition from before: we need to perform the transition function on all the elements, then union the results together. For example, on $\{0, 1\}$ and a, we need to union the results of doing $\{0\}$ and a, and $\{1\}$ and a. Just above the $\{0, 1\}$ and a cell, we can see the results of those. We just need to union those together, by which I mean, union $\{0\}$ and $\emptyset$ together. When we do that, we get $\{0\}$, so in this transition table, when you input a on the state $\{0, 1\}$, you get $\{0\}$. Repeat this for all the states and inputs until you finish the table.
<table>
<tr><td></td><td>a</td><td>b</td></tr>
<tr><td>$\emptyset$</td><td>$\emptyset$</td><td>$\emptyset$</td></tr>
<tr><td>$\{0\}$</td><td>$\{0\}$</td><td>$\{0, 1\}$</td></tr>
<tr><td>$\{1\}$</td><td>$\emptyset$</td><td>$\emptyset$</td></tr>
<tr><td>$\{0, 1\}$</td><td>$\{0\}$</td><td>$\{0, 1\}$</td></tr>
</table>

Once you're done, that's pretty much it! This transition table is your new transition function for your DFA.

This video goes into more detail and I strongly suggest it if you still struggle with this.
</td></tr>
<tr><td>s</td><td>It's pretty much the same as the original, but you encapsulate it in a set:

$s = \{0\}$</td></tr>
<tr><td>F'</td><td>Get all the elements of Q':

$\{\emptyset, \{0\}, \{1\}, \{0, 1\}\}$

Now get rid of the ones that do not have final states in them:

$\{\emptyset, \{0\}, \{1\}, \{0, 1\}\}$

So therefore, your new F' is:

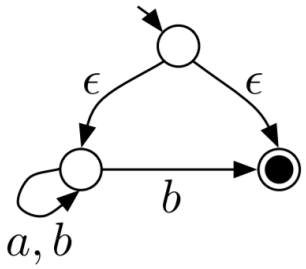
$F' = \{\{1\}, \{0, 1\}\}$</td></tr>
</table>

ϵ-moves

- An epsilon-move (ϵ-move) is a type of transition that you can use without needing to "spend" or "consume" any symbols.
- It's basically a free move.
- An NFA that uses epsilon-moves are called ϵNFAs.

- They're formally defined like this:
  - $M_\epsilon = (Q, \Sigma, \theta, s, F)$
  - where
  - $\theta: Q \times (\Sigma + \{\epsilon\}) \rightarrow 2^Q$
- This is basically the same as an NFA, except transitions now support epsilon-moves.

- Here is an example of an ϵNFA:



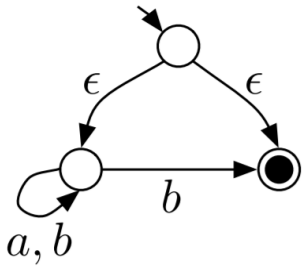
- From the starting state, you can go to the left-most state and the right-most final state, all without consuming any symbols.

- You can also convert ϵNFAs to NFAs. Don't worry! It's not as long as subset construction.
- You only need to edit the transition function and the set of final states.
- With an ϵNFA M_ϵ we can create an NFAM_N like so:
    - $M_\epsilon = (Q, \Sigma, \theta, s, F)$
    - $M_N = (Q, \Sigma, \Delta, s, F')$
        - $\Delta(q, \sigma) = \{q' \mid q \overset{\epsilon}{\Rightarrow}\overset{\sigma}{\rightarrow} q'\}$
        - $F' = \{q \mid \exists f \in F. q \overset{\epsilon}{\Rightarrow} f\}$

- Your next line will be "Is this going to be translated into plain English again?"
- Please see below for a translation into plain English.

Line in weird maths language	**Line in plain English**
$M_\epsilon = (Q, \Sigma, \theta, s, F)$	We're just defining an ϵNFA with these parameters.
$M_N = (Q, \Sigma, \Delta, s, F')$	We're just defining an NFA with these parameters. These parameters will be based off of the parameters for M_ϵ.
$\Delta(q, \sigma) = \{q' \mid q \overset{\epsilon}{\Rightarrow}\overset{\sigma}{\rightarrow} q'\}$	When you input a state and a symbol, the output will be all the possible states it can reach using epsilon-moves and that symbol.
$F' = \{q \mid \exists f \in F. q \overset{\epsilon}{\Rightarrow} f\}$	The new set of final states is a superset of the old set of final states, meaning we still keep the final states the same, but we add more. We add the states that can epsilon-move towards a final state.

- Now I'll go over an example, by converting that ϵNFA above into an NFA:



<table>
<tr><th>What we're defining</th><th>How we did it</th></tr>
<tr><td>Δ</td><td>
First, let's create that transition table again. Just for example's sake, I'm calling the initial state 0, the left-most state 1 and the final state 2:

<table>
<tr><td></td><td>a</td><td>b</td></tr>
<tr><td>0</td><td></td><td></td></tr>
<tr><td>1</td><td></td><td></td></tr>
<tr><td>2</td><td></td><td></td></tr>
</table>

If you were at state 0 and had an input 'a', what states could you get to? Well, you could go down the left epsilon-move to get to state 1 and spend the 'a' on going back to state 1. That's pretty much it, so here, we'd input just state 1.

If you were at state 0 and had an input 'b', what states could you get to? You could go down the left epsilon-move and use 'b' to go to state 1 or use your 'b' to get to state 2. If you went down the right epsilon-move, you wouldn't be able to use 'b', so you can ignore it. Seems like you can get to states 1 and 2, so we put that in the transition table.

We keep going like this until we fill out the transition table:

<table>
<tr><td></td><td>a</td><td>b</td></tr>
<tr><td>0</td><td>$\{1\}$</td><td>$\{1, 2\}$</td></tr>
<tr><td>1</td><td>$\{1\}$</td><td>$\{1, 2\}$</td></tr>
<tr><td>2</td><td>$\emptyset$</td><td>$\emptyset$</td></tr>
</table>

This transition table is our new transition function!
</td></tr>
<tr><td>F'</td><td>
For this one, you could look at the final states in the graph, and work your way back up the epsilon-moves, looking for non-final states that can epsilon-move to the final state.

As you can see in the graph above, if you start at state 2 (the final state) and work your way up the epsilon-move, you'll find yourself at state 0. Therefore, we can make state 0 a final state. However, you cannot go up to state 1 from state 2, so state 1 cannot be a new final state.

Obviously, we keep the old final states from the ϵNFA:

$F' = \{0, 2\}$
</td></tr>
</table>

Regular expressions and Kleene's theorem

Defining regular expressions

- A string s matches a regular expression α whenever $s \in L(\alpha)$
- Regular expressions can be defined using atomic and compound patterns

Atomic patterns

- There exist the following atomic patterns, which do not need to be broken down further:

Pattern	**Matched by …**
$L(\sigma) = \{\sigma\}$	… the single symbol σ from the alphabet: Σ
$L(\varepsilon) = \{\varepsilon\}$	… the empty string
$L(\emptyset) = \{\}$	… nothing
$L(\#) = \Sigma$	… any single symbol from the alphabet Σ
$L(@) = L(\#^*) = \Sigma^*$	… any string formed by the symbols in Σ (including the empty string), i.e. any string in Σ^*

Compound patterns

- Compound patterns are defined recursively, based on the atomic patterns.

Pattern	**A String will match if ...**
$L(\alpha + \beta) = L(\alpha) \cup L(\beta$	… it matches α or β
$L(\alpha \cap \beta) = L(\alpha) \cap L(\beta$	… it matches both α and β
$L(\alpha\beta) = L(\alpha)L(\beta)$	… it can be broken down into two parts such that the first part matches α and the second part matches β
$L(\sim \alpha) = \sim L(\alpha)$	… it does not match α
$L(\alpha^*) = L(\alpha)^*$	… it matches 0 or more repetitions of α.
$L(\alpha^+) = L(\alpha)^+$	… it matches 1 or more repetitions of α.

Examples

- $\Sigma = \{a, b, i, t, v\}$

  All strings that end in “vita”: $(a + b + i + t + v)^* vita$ or more elegant: $@vita$
- $\Sigma = \{a, b\}$

  All strings with an even number of a’s: $b^*(ab^*ab^*)^*$
- $\Sigma = \{a, b\}$

  All strings with an odd number of a’s: $b^*(ab^*ab^*)^*ab^*$

Kleene’s theorem

- If α is a regular expression then $L(\alpha)$ is a regular language.

    - All strings that match a regular expression form a language. This language is regular.
- If L is a regular language then $L = L(\alpha)$ for some regular expression α.
    - If a language is regular, then there exists a regular expression that matches all strings in that language (and not more/less).
- Therefore, we can conclude that finite automata and regular expressions describe the same languages; they are equally powerful.

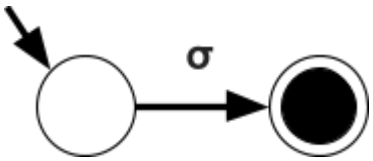
- In order to prove this theorem, we need to show that we can convert a regular expression to a finite automaton and vice versa.

Reg exp to εNFA

- We will convert a regular expression to a εNFA
- Basically, we're going to split our regular expression into "building blocks", then we're going to convert those "building blocks" into automata individually.
- With each regex "building block", we need to show that we can build an automata out of it. In other words, for each possible regex "building block", we need to show that it's language is regular.
- Once we've shown that all our building blocks are regular, we can use said building blocks to build an automata out of any regex expression.

- We can prove we can convert by induction, as regular expressions are built inductively.
- This means that we can build any regular expression by combining our atomic patterns, which results in a compound pattern. Think of the atomic patterns as our basic building blocks:
    - First we check if our basic building blocks are regular.
    - Then we assume that the blocks we use to build compound patterns are regular.
    - Then we check if the compound patterns are regular.
    - If they are regular we have completed the proof.

- Let's start with checking the base cases, these are our basic building blocks, represented by atomic regular expressions:
    - $L(\sigma)$is regular:



    - $L(\varepsilon)$is regular:

- $L(\emptyset)$is regular:

- These are all atomic regular expressions we have to check, as # and @ are redundant, meaning that they can also be represented by compound patterns.

- Now, assume that the building blocks we use to build compound patterns are regular. Then we will check if the compound patterns are regular:
  - $L(\alpha + \beta) = L(\alpha) \cup L(\beta)$
    - We have already proved above that the union of two regular languages is regular.
  - $L(\alpha\beta) = L(\alpha)L(\beta)$
    - It was also proved above that the concatenation of two languages are regular.
  - $L(\alpha^*) = L(\alpha)^*$
    - Again, we have proved above that kleene star is regular.
- These are all compound regular expressions we have to check as all other compound regular expressions are redundant, meaning that they can also be represented by other compound patterns.

- Now we have completed the proof that from any regular expression we can construct an automaton.

εNFA to reg exp

- First of all, let's simply state what we want to do. We want to prove that there is always a way to convert an εNFA to a regular expression.

- We will construct an NFA $M = (Q, \Sigma, \Delta, s, F)$
  - Q is the set of states
  - Σ is the alphabet
  - Δ is the transition function
  - s is the start state
  - $F$is the set of final states

- This is what we want: the regular expression $\alpha^{X}_{u,v}$ which describes all possible paths
  - starting at the state u
  - ending at the state v
  - passing through 0 or more intermediary states, which are in the set X.
- It's important to note that we do not need to pass through any intermediate states in X, but we have the opportunity to pass through as many as we want.

- If we can define the regex $\alpha^{X}_{u,v}$ mathematically, then we've shown that we can convert an εNFA to a regex, thereby finishing our proof.
- We shall prove this by induction! In case you've forgotten, proof by induction goes like this:
    1. Base case
    2. Inductive step
- Yes, there will be some weird maths language, but don't worry; I'll translate it for you. It sounds scary at first, but once you see the proof, it's not that bad:

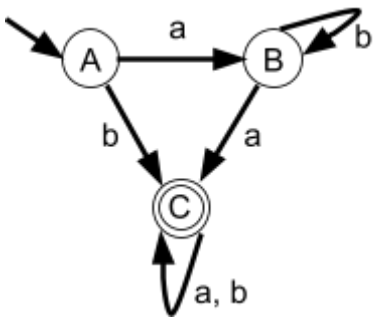
Step	**Weird maths language**	**Normal English + pretty pictures**
Base case	Let $a_1, a_2, a_3 ...$ be all symbols such that $\delta(u, a_i) = v$ for $1 \leq i \leq k$ $1.\ if\ u \neq v\ and\ k > 0$ $\alpha^{\emptyset}_{u,v} = \{a_1 + a_2 + a_3 + ...\}$ $2.\ if\ u \neq v\ and\ k = 0$ $\alpha^{\emptyset}_{u,v} = \emptyset$ $3.\ if\ u = v\ and\ k > 0$ $\alpha^{\emptyset}_{u,v} = \{\varepsilon + a_1 + a_2 + ... + \varepsilon\}$ $4.\ if\ u = v\ and\ k = 0$ $\alpha^{\emptyset}_{u,v} = \{\varepsilon\}$	Firstly, our base case. The simplest case is going from the initial state to the final state without going through any intermediate states. This is what we're defining on the left here; from top to bottom, we're defining 1. If the initial state isn't the same as the final state and there are transitions from initial state to final state 2. If the initial state isn't the same as the final state and there are **no** transitions from initial state to final state 3. If the initial state is the final state and there are transitions from the initial state to the final state 4. If the initial state is the final state and there are **no** transitions from the initial state to the final state All those $a_1, a_2, a_3 ...$ symbols are all transitions that go straight from the initial state to the final state.
Inductive step	$\alpha^{X+\{q\}}_{u,v} = \alpha^{X}_{u,v} + \alpha^{X}_{u,q}(\alpha^{X}_{q,q})^{*}\alpha^{X}_{q,v}$	First, we have a proposition (assumption). We propose that $\alpha^{X}_{u,v}$ exists and works. Then, we define a $\alpha^{X+\{q\}}_{u,v}$ out of $\alpha^{X}_{u,v}$. Once we do that, we've defined our inductive step. By doing this, we show that if there exists a regex from u to v through intermediate states X, there also exists a regex from u to v with

		intermediate states X + {q}, where 'q' is any other state in the automata. This means we can keep adding on states from our base case $\alpha_{u,v}^{\emptyset}$ until we've filled in all the states in our automata.

- As you can see from the proof, we can just keep adding on new states to our X. That means we can add our initial state and final state to this regular expression, and then append all the intermediate states onto X and it'll work.

- That's great, but how do you actually convert an εNFA to a regular expression? We've proved that we can do it, so how do we actually do it?
- Below you'll see an example of how to actually convert an εNFA into a regular expression. It involves breaking down the regex you need by taking out intermediate states, until you get something trivial, then working your way back up again.

- EXAMPLE:
- Given the following NFA



- We need to find a regular expression that starts at state A, ends at state C and passes through any of A,B,C zero or more times.

- Therefore, we are looking for $\alpha_{A,C}^{\{A,B,C\}}$
    - The starting state u is going to be A
    - The ending state v is going to be C
    - The set of states which we can pass through X is equal to Q

- Now, we are going to rewrite this regular expression using the formula above
    - We are going to pull out one of the states from X
    - It is best to choose a state that reduces the paths we can go a lot
- I'm going to pull out B. This gives:

$$\alpha_{A,C}^{\{A,B,C\}} = \alpha_{A,C}^{\{A,C\}+\{B\}} = \alpha_{A,C}^{\{A,C\}} + \alpha_{A,B}^{\{A,C\}} (\alpha_{B,B}^{\{A,C\}})^{*} \alpha_{B,C}^{\{A,C\}}$$

- We can decompose this further into

$$\alpha_{A,C}^{\{A,C\}} = \alpha_{A,C}^{\{A\}+\{C\}} = \alpha_{A,C}^{\{A\}} + \alpha_{A,C}^{\{A\}}(\alpha_{C,C}^{\{A\}})^{*}\alpha_{C,C}^{\{A\}}$$

$$\alpha_{A,B}^{\{A,C\}} = \alpha_{A,B}^{\{C\}+\{A\}} = \alpha_{A,B}^{\{C\}} + \alpha_{A,A}^{\{C\}}(\alpha_{A,A}^{\{C\}})^{*}\alpha_{A,B}^{\{C\}}$$

$$\alpha_{B,C}^{\{A,C\}} = \alpha_{B,C}^{\{A\}+\{C\}} = \alpha_{B,C}^{\{A\}} + \alpha_{B,C}^{\{A\}}(\alpha_{C,C}^{\{A\}})^{*}\alpha_{C,C}^{\{A\}}$$

- It is up to you how far you decompose.
  - If you are able to see what the regular expression is right away, you don't need to decompose. However, if you barely decompose, the chances that you miss a path are high.
  - We could decompose what we have got now even further, but I don't think that it is necessary to do so as the regular expressions for these are fairly easy to find.

- Let's take a look at the first decomposed regular expression:

$$\alpha_{A,C}^{\{A,C\}} = b + b(a+b)^{*}(a+b) = b(a+b)^{*}$$

- Any string that starts with b or that starts with b followed by one or more mixed a's and/or b's will be matched. This can be simplified into a string starting with b followed by zero or more mixed a's and/or b's.

- Let's take a look at the second decomposed regular expression:

$$\alpha_{A,B}^{\{A,C\}} = a + \varepsilon(\varepsilon)^{*}a = a + a = a$$

- Any string that contains exactly one a will be matched.

- Let's take a look at the third decomposed regular expression:

$$\alpha_{B,C}^{\{A,C\}} = a + a(a+b)^{*}(a+b) = a(a+b)^{*}$$

- This is very similar to the first one. It will match a string starting with a followed by zero or more mixed a's and/or b's.

- As you might have noticed, I didn't decompose the blue part. The reason for that is that the blue regular expression starts and ends in the same state, meaning that it can only take transitions to the state itself (if exists):
- $\alpha_{B,B}^{\{A,C\}} = b$
- Any string that contains exactly one b will be matched.

- Now, let's put together the regular expressions from the coloured parts:

- $\alpha_{A,C}^{\{A,B,C\}}$ =b(a+b)* + ab*a(a+b)*
- This is the regular expression that is represented by this NFA - we are done!

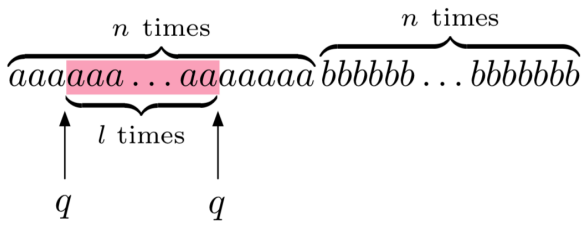
- Often, there is more than one regular expression that is represented by an automaton, meaning that there is no single correct solution. For example, the regular expression (ab*a + b)(a + b)* is correct as well.

Limitations of regular languages

- Is every language regular? No!

- For example, look at the language $\{a^n b^n \mid n \in \mathbb{N}\}$
  - Our automaton has finitely many states.
  - It's going to scan the string from the left to the right, starting with the a's.
  - Then, when it's going to scan the b's, it has to remember how many a's there were previously.
  - Since n can be infinitely big, there is no way to do this with our finite number of states!

- We are going to prove by contradiction that the language $\{a^n b^n \mid n \in \mathbb{N}\}$ is not regular.
  - Suppose we have an automaton M that accepts this language. Since the number of states it has is finite, we are going to say that it has k states.
  - Next, we are going to choose n such that $n > k$. We can always find an n that is greater than k because n is not bounded, i.e. can be infinitely large.
  - Now, our automaton is going to start to scan the n a's from the left to the right. Since we have less states than the number of a's, there must be a state we are going to enter at least twice (pigeonhole principle)! We are going to call this state q. Between the first time we enter q and the second time we enter q we are going to scan l a's. After the second time we entered q our automaton is going to continue to the final state.



  - When the automaton enters state q for the first time, it goes on a path that leads it to q again. Then, after arriving at q for the second time, it goes on a path that leads it to the final state.
  - Here is the problem: Why arrive at q, and then take a path that leads to q again? Why not take the path that goes to the final state upon arriving at q for the first time? This means that our automaton would accept a string with

$n - l$ number of a's. However, now the number of a's is not anymore equal to the number of b's and here we have our contradiction!

Pumping (🎃) Lemma for regular languages - contrapositive form

- We can use the contrapositive form of the pumping lemma for regular languages to show that a language is not regular. We cannot use the pumping lemma to show that a language is regular!

- It works by splitting up the language into parts, then "pumping" a subpart until you get a string that isn't in the language. This is similar to what we have just done before.

- The different steps of the proof can be understood as a game versus a demon.

1. The demon chooses a number k such that $k > 0$.
   - We cannot choose a value for k, we only know that it is greater than 0.

2. We choose a string $xyz \in L$ such that $|y| \geq k$.
   - We can choose any string xyz that is within the language. However, we have to make sure that, no matter what k is, our string xyz is always in the language. We do not need to cover the whole language with our string!
   - x and z can be empty, there is no restriction concerning them.
   - The length of the y part must be at least k. A way to make sure that this is always the case is to let the y part be equal to something to the power of k.
   - $|y| > 0$ due to the lower bound of k.
   - As a general tip, try to make y as trivial as possible, like in the following example:
   - For instance, we could let $y = a^k$ to make sure that y is at least k long (a is a symbol from the alphabet).

3. The demon splits the y part of the string into uvw such that $|v| > 0$, i.e. $v \neq \varepsilon$. The whole string would be $x(uvw)z$.
   - You cannot choose how the string is going to be split! All you know is that the v part of the string is not going to be empty.
   - Therefore, you have to do a general case and define uvw such that every way of splitting the string is possible and we do not introduce any restrictions.
   - u and w can be empty, there is no restriction concerning them.
   - For instance, if $y = a^k$, then $v = a^l$ and $1 \leq l \leq k$ because v cannot be empty. Furthermore, the remaining part $uw = a^{k-l}$. If we add the two parts, we are going to get back to $y = a^k$. Also, we haven't made any assumptions about how the string is going to be split.

4. We pick an $i \geq 0$ such that the string $xuv^iwz \notin L$.If this is the case we have proven that the language is not regular!
   - We can pick any i we want as long as it is equal to or greater than 0.

- Our goal is to choose an i such that the resulting string is outside of the language.
- For instance, $v = a^{l}$ and $uw = a^{k-l}$. If we now choose $i = 0$, we are going to get $uv^{0}w = a^{k-l}$. We have now removed l a's compared to what we had as uvw in the step before, which was $uvw = y = a^{k}$. Also we know that $1 \leq l$, meaning that we removed at least one a!

5. You **MUST** say "Therefore L is not regular", if you don't, you will lose marks.

- If this still doesn't make any sense, look at a few examples below, and then read the general case above again.

Example	**Description**
$\{a^{n}b^{n} \mid n \in \mathbb{N}\}$	1. The demon chooses k such that $k > 0$ 2. Now choose a string $xyz \in L$ and $\lvert y \rvert \geq k$. - I'll do it like $(\varepsilon)(a^{k})(b^{k})$, so $x = \varepsilon$, $y = a^{k}$, $z = b^{k}$ and $xyz \in L$. - For every value of k I will get a specific string which is in the language of $a^{n}b^{n}$. - It is ok that I cannot get the empty string ($a^{0}b^{0}$ which is in the language) because I do not need to cover the whole language. 3. The demon splits your y into a uvw such that $v \neq \varepsilon$. - Now, we have to do the splitting in a general case. - Since v cannot be empty, then $v = a^{l}$ and $1 \leq l \leq k$. - Since $uvw = y = a^{k}$, then $uw = a^{k-l}$. 4. We pick an $i \geq 0$ such that the string $xuv^{i}wz \notin L$. - Now for this example, choose $i = 0$. - Then $xuv^{0}wz = a^{k-l}b^{k} \notin L$ as the number of b's will be bigger than the number of a's, no matter what value k is. 5. Therefore L is not regular
$\{a^{n!} \mid n \in \mathbb{N}\}$	1. The demon chooses k such that $k > 0$ 2. Now choose a string $xyz \in L$ and $\lvert y \rvert \geq k$. - I'm going to choose $x = a^{(k+1)!-k}$, $y = a^{k}$, $z = \varepsilon$, so $xyz = (a^{(k+1)!-k})(a^{k})(\varepsilon) = a^{(k+1)!} \in L$. 3. The demon splits your y into a uvw such that $v \neq \varepsilon$. - $v = a^{l}$, as it cannot be empty we have $1 \leq l \leq k$. - The remaining part $uw = a^{k-l}$ because $uvw = y = a^{k}$

	4. We pick an $i \geq 0$ such that the string $xuv^{i}wz \notin L$. - We pick $i = 2$. - Then $xuv^{2}wz = a^{(k+1)!-k}a^{\ k-l}a^{\ 2l} = a^{(k+1)!+l} =\notin L$ because the power we have got here is not going to be a factorial! - Let me prove why this is not going to be a factorial: - $(k + 1)!$ is going to be some factorial $\geq 2!$. - The next higher factorial is going to be $(k + 2)!$. - However, our power $(k + 1)! + l$ is going to be in the middle of these two factorials: $(k + 1)! < (k + 1)! + l < (k + 2)!$.Therefore, it cannot be a factorial! - If you still have some doubts, try it out with the smallest k we could have, $k = 1$. Then choose the largest l we can choose, $l = 1$. So we have $2! < 2! + 1 < 3!$ which holds. For any choice of $k > 1$ the next higher factorial is going to grow even faster, therefore this will always hold. 5. Therefore L is not regular
$\{a^{n} \mid n\ is\ prime\}$	1. The demon chooses k such that $k > 0$ 2. Now choose a string $xyz \in L$ and $\lvert y\rvert \geq k$. - I'm going to choose $x = \varepsilon,\ y = a^{p},\ z = \varepsilon$ where p is going to be the next prime number $\geq k$. Since the number of primes is infinite this will always work, no matter how large k is. - Therefore, $xyz = a^{p} \in L$. 3. The demon splits your y into a uvw such that $v \neq \varepsilon$. - The length of $uvw$is going to be p, so $\lvert uvw\rvert = p$. - Since $x = z = \varepsilon$ we have $\lvert xuvwz\rvert = p$. - That's all info we need to continue our proof, you will see why in the next step. 4. We pick an $i \geq 0$ such that the string $xuv^{i}wz \notin L$. - We will pick $i = p + 1$. - I'm going to show that the length of $xuv^{p+1}wz$ is not going to be equal to a prime number, therefore it is outside of the language. - $\lvert xuv^{i}wz\rvert = \lvert xuv^{p+1}wz\rvert = \lvert xuv^{1}wz\rvert + \lvert v^{p}\rvert = p + \lvert v^{p}\rvert = = p * (1 + \lvert v\rvert)$, which is not going to be a prime number since we also have a factor $(1 + \lvert v\rvert)$ here but prime numbers should only be divisible by 1 or itself (p). - Therefore, $xuv^{p+1}wz \notin L$.

	5. Therefore L is not regular
$\{mm^R \mid m \in \Sigma^*,$ m^R *is its reverse*$\}$ and $\lvert\Sigma\rvert \geq 2$	1. The demon chooses k such that $k > 0$ 2. Now choose a string $xyz \in L$ and $\lvert y\rvert \geq k$. - I'm going to choose $x = \varepsilon$, $y = e^k$, $z = dde^k$ where $d, e \in \Sigma$ and $d \neq e$ - $xyz = e^k dde^k \in L$ - The demon splits your y into a uvw such that $v \neq \varepsilon$. - $v = e^l$ - $uw = e^{k-l}$ - $1 \leq l \leq k$ 4. We pick an $i \geq 0$ such that the string $xuv^iwz \notin L$. - Pick $i = 0$ - Now $xuv^0wz = e^m dde^k$ and $m \neq k$because we deleted at least one e by letting $i = 0$. - Therefore $xuv^0wz \notin L$ 5. Therefore L is not regular

Automata theory: Context free languages

Pushdown Automata (PDA)

- Pushdown automata are an extension to the automata we have seen before.
- We will take an εNFA and add a stack to the control unit.
- The stack does not have a size limit, i.e. is infinitely big.

- More formally, a PDA is a 7 tuple $M = (Q, \Sigma, \Gamma, \delta, s, \perp, F)$
  - Q is the set of states
  - Σ is the input alphabet
  - Γ is the stack alphabet (the set of things we can put on the stack)
  - δ is the transition relation (more detail see below)
  - $s \in Q$ is the start state
  - $\perp$ is the initial stack symbol
  - $F \subseteq Q$ is the set of final states

Transition relation

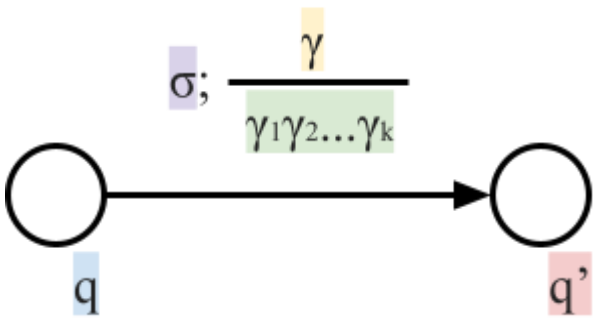
- Now, let's take a closer look at the transition relation δ:

  $\delta \subseteq (Q \times (\Sigma \cup \{\varepsilon\}) \times \Gamma) \times (Q \times \Gamma^*)$
  - Q is the state the PDA is currently in.

- $\Sigma \cup \{\varepsilon\}$ is the symbol we are currently reading. This symbol could be any symbol from our alphabet or epsilon, which means that we can do epsilon moves by reading nothing!
- Γ is the symbol which is currently on top of the stack. When we make a transition we are going to pop (remove) this symbol from the top of the stack.
- Q is the resulting state after we have made the transition.
- Γ^* is going to be 0 or more symbols from our stack alphabet that we are going to push on the stack when we make the transition.

- Let's look at how one tuple of our transition relation looks like:
  $\delta \ni ((q,\sigma,\gamma), (q',\gamma_1\gamma_2 \dots \gamma_k))$
  Here, we are in the state q, we will read the symbol σ, and we see γ on top of the stack. We will then pop (remove) γ off the stack, go to the next state q' and push (add) $\gamma_1\gamma_2 \dots \gamma_k$ on the stack. We will start pushing with γ_k and end with γ_1, such that we now have γ_1 on top of the stack.
- Keep in mind that if γ is not on the top of the stack, we cannot perform this transition. Think of it as like a "second input".

- Graphically, the transition looks like this:



Configuration

- A configuration is a complete description of our PDA at a certain point in time. It is represented as an element of:
  Configurations $\subseteq (Q \times \Sigma^* \times \Gamma^*)$
  - Q is the current state.
  - Σ^* is the whole part of the input we still have left to scan/read. Note that this is not only the next symbol.
  - Γ^* is the whole stack content. Note that this is not only the symbol on top of the stack; it's the entire thing.

- Therefore, every configuration will be of the form
  Configurations $\ni (q, \sigma_1\sigma_2 \dots \sigma_k, \gamma_1\gamma_2 \dots \gamma_k \perp)$

- We can define relations between those configurations, which tell us how to go from one configuration to the next configuration. We have to distinguish between two cases: The case when we consume a symbol from the input, and the case where we don't consume any symbol and take an epsilon move.

- Case 1: We consume a symbol. We write the relation as:
  $(q,\sigma s,\gamma g)\rightarrow(q',s,hg)$
  when there exists such an element in δ
  $((q,\sigma,\gamma),(q',h)) \in \delta$

- Case 2: We don't consume a symbol and take an epsilon move. We write the relation as:
  $(q,s,\gamma g)\rightarrow(q',s,hg)$
  when there exists such an element in δ
  $((q,\varepsilon,\gamma),(q',h)) \in \delta$

Acceptance

- A PDA can accept a string either by empty stack or by final state.
- We can turn a PDA that accepts by final state into a PDA that accepts by empty stack and vice versa.

By final state

- A PDA will accept a string by final state if we can go from the start configuration to a final configuration which contains a final state and where we don't have any string left to scan/consume. Between those two configurations there can be 0 or more other intermediary configurations which we pass through.

- Formally, this is written as:
  $(s,x,\perp)\Rightarrow(f,\varepsilon,g)$
  - We will start at state s with the whole input string x still remaining to scan, and our stack contains the initial stack symbol only.
  - We will go through as many intermediary states as we want to.
  - We will end in a state f where we don't have anything remaining to scan, and our stack content is g.

- Therefore, the language accepted by the PDA is
  $L(M) = \{x \mid \exists f \in F.\ (s,x,\perp)\Rightarrow(f,\varepsilon,g)\}$

By empty stack

- A PDA will accept a string by empty stack if we can go from the start configuration to a final configuration where the stack is completely empty and where we don't have any string left to scan/consume. Between those two configurations there can be 0 or

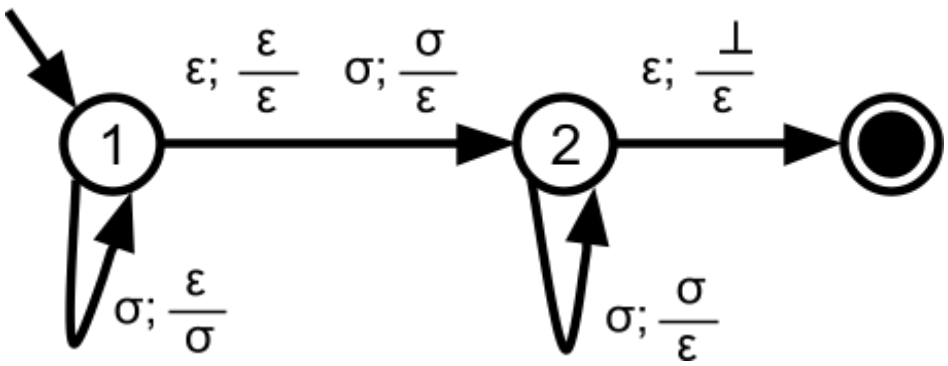
more other intermediary configurations which we pass through.

- Formally, this is written as:
  $(s,x,\perp) \Rightarrow (q,\varepsilon,\varepsilon)$
  - We will start at state s with the whole input string x still remaining to scan, and our stack contains the initial stack symbol only.
  - We will go through as many intermediary states as we want to.
  - We will end in a state q where we don't have anything remaining to scan, and our stack is empty.
  - "Stack is empty" means that we do **not** have $\perp$ or any other symbol on the stack

- Therefore, the language accepted by the PDA is
  $L(M) = \{x \mid \exists q \in Q \,.\, (s,x,\perp) \Rightarrow (q,\varepsilon,\varepsilon)\}$

Example PDA accepting palindromes

- A Palindrome is a string where the reverse of that string is equal to the original string.
  - Examples: abba, abdba, aaaa
- A PDA that will accept all palindromes by final state (and also empty stack) looks like this:



where $\sigma \in \Sigma$, i.e. σ is any symbol from our alphabet Σ.

- Let's start from scratch and build a PDA that looks like this.

- We are going to differentiate between palindromes with an even number of symbols and those with an odd number of symbols

- Our strategy for constructing the PDA that accepts palindromes with an even number of symbols will be the following:
  - We scan the word until we reach the middle of it.
  - Then we scan the second half of the word, expecting the symbols from the first half of the word in reverse order.
  - If this is the case, our word is a palindrome.

- Now, we can assign a state for every step of our strategy

    - We will be in state 1 when we scan the first half of the word.
    - We will be in state 2 when we scan the second half of the word.
    - We will be in the final state when we have finished scanning the whole word and the reverse of the first half is equal to the second half of the word.

- Let's think about our transitions for each of our states:
    - When we are in state 1
        - We will stay in state 1 and keep adding the current symbol to the stack if we haven't reached the middle of the word.
        - We will make a transition to state 2 when we have reached the middle of the word, without changing the stack.
    - When we are in state 2
        - We will stay in state 2 and keep popping the current symbol from the stack as long as the current symbol is equal to the symbol at the top of the stack.
        - We will make a transition to the final state if our stack contains only the initial stack symbol, and remove the initial stack symbol.

- However, there is still one problem: Palindromes with an odd number of symbols, such as "abdba". How to deal with them? We can throw away the symbol in the middle! For this example, it means that we throw away "d" and we will check the remaining string "abba". We have already defined a PDA for checking palindromes with an even number of symbols above.

- In order to integrate the "throw away the middle symbol" action into our existing PDA we will add one transition from state 1 to 2 where we consume a symbol but don't change the stack contents.

- Some questions you might ask yourself are:
    - How does the PDA know when it has reached the middle of the word?
    - How does the PDA know that it has arrived at the middle symbol of an odd string, which can be thrown away?
- The answer is that the PDA is going to guess. The PDA can do this because it is non-deterministic, similar to NFAs we have seen before. In other words, the PDA can take any of the transitions that are possible.
- If we have a palindrome as a string and we process this palindrome, there will be one combination of transitions where the PDA will go into the final state at the end.

- Let's simulate our PDA on the string "abcba", which is a palindrome:

$$
\begin{aligned}
&(0, abcba, \perp)\\
\rightarrow &(0, bcba, a\perp)\\
\rightarrow &(0, cba, ba\perp)\\
\rightarrow &(1, ba, ba\perp)\\
\rightarrow &(1, a, a\perp)\\
\rightarrow &(1, \varepsilon, \perp)
\end{aligned}
$$

$\rightarrow(2, \quad \varepsilon, \quad \varepsilon)$
The PDA has reached state 2 and will therefore accept.

- Let's simulate our PDA on the string "abcde", which is not a palindrome:
  $(0, abcde, \quad \perp)$
  $\rightarrow(0, bcde, \ a \ \perp)$
  $\rightarrow(0, \ cde, ba \ \perp)$
  $\rightarrow(1, \quad de, ba \ \perp)$
  → X at this point our computation dies because there is no transition we could take (d ≠ b). Therefore the PDA will reject.

Context free grammars

- A CFG, consisting of terminal and non-terminal symbols, is used to generate strings (sentences) by following its production rules.
  - The terminal symbols are the symbols which will form the string we are going to generate.
  - The non-terminal symbols are going to be replaced by other symbols. These other symbols can be terminals, non-terminals or a mix of both.
  - There exist so called "productions" which tell us how to replace the non-terminal symbols.
  - We have to keep replacing non-terminal symbols until we arrive at a string that consists solely of terminal symbols. This process is called "derivation".
  - The final string is also called a sentence.

- Formally, a CFG is a quadruple (N, Σ, P, S)
  - N is a finite set of nonterminal symbols
  - Σ is a finite set of terminal symbols
  - $P \subseteq N \times (N + \Sigma)^*$ are the productions
  - $S \in N$ is the start nonterminal

- To define production rules …
  - we will use the shorthand $A \rightarrow \alpha$
    to mean $(A, \alpha) \in P$
  - we will use the shorthand $A \rightarrow \alpha_1 | \alpha_2 | \ldots | \alpha_k$
    to mean $A \rightarrow \alpha_1$, $A \rightarrow \alpha_2, \ldots, A \rightarrow \alpha_k$

- To express that β is derivable from α we will write $\alpha \Rightarrow \beta$ or $\alpha \overset{k}{\Rightarrow} \beta$
  - Without k: This means that given the string α, we have applied some production rules and arrived at string β.
  - With k: This means that given the string α, we have applied exactly k production rules and arrived at string β.

- The language generated by a CFG G is the set of all its sentences, i.e. the set of all strings that can be generated by using its production rules.

  - $L(G) = \{x \in \Sigma^* \mid S \Rightarrow x\}$
  - The language generated is going to be equal to the set of all strings x that are derivable from the start non-terminal.
  - We can apply as many production rules as we want, as long as we end with a string (sentence) that solely consists of terminals.
- This language L(G), generated by some CFG G, is called context-free (CFL).

- If it's all still a bit too confusing for you, look at the table below, where we convert context-free languages into context-free grammars:

Language	**Converting it to a grammar**
$\{a^n b^n \mid n \in \mathbb{N}\}$	- Every string with the same number of a's followed by the same number of b's will be in the language. - We have shown that this language is not regular, however, it is context free as we can come up with a CFG for it. - The CFG is $S \rightarrow aSb \mid \varepsilon$ - We can produce 0 a's and b's, i.e. the empty string, by applying only the second production rule: $S \Rightarrow \varepsilon$ - We can produce 1 a and 1 b by applying the first production rule once and the second production rule last: $S \Rightarrow aSb \Rightarrow ab$ - We can produce 2 a's and 2 b's by applying the first production rule 2 times and the second production rule last: $S \Rightarrow aSb \Rightarrow aaSbb \Rightarrow aabb$ - Therefore, we can produce n a's and n b's by applying the first production rule n times and the second production rule last. - We achieve this through pumping aSb in the middle of the string every time we want to add an a and a b.
$\{a^n b^n \mid n > 0\}$	- $S \rightarrow aSb \mid ab$
$\{a^n b^m c^{2n} \mid n, m > 0\}$	- $S \rightarrow aScc \mid aBcc$ - $B \rightarrow bB \mid b$
$\{a^n b^m c^{2n} \mid n, m \geq 0\}$	- $S \rightarrow aScc \mid B$ - $B \rightarrow bB \mid \varepsilon$
L(G) is the set of all palindromes over the alphabet $\Sigma = \{a, b, c\}$	- $S \rightarrow aSa \mid bSb \mid cSc \mid a \mid b \mid c \mid \varepsilon$

L(G) is the set of all balanced parentheses	- $S \to (S) \mid () \mid SS$

$\{yb^{2n}x \mid n \geq 0,$
$y, x \in \{a\}^*,$
$\lvert yx \rvert = n\}$
 	
<u>English, please</u>
So you have a bunch of 'a's, then a bunch of 'b's, then another bunch of 'a's.
 	
There needs to be twice as many 'b's as 'a's.
 	
Examples:
ε
abb
abbbba
bbbbaa
	- Simplify: $L(G) = \{a^k b^{2(k+d)} a^d \mid k + d \geq 0\}$
	  - We know that y and x are just going to be repetitions of the letter a.
	  - We let $n = k + d$, then we can use those for the number of times we repeat the first a and the last a. We put k and d in the corresponding exponents to denote these repetitions.
	- Simplify further:
	$L(G) = \{a^k b^{2k} b^{2d} a^d \mid k \geq 0,\ d \geq 0\}$
	  - We use k and d to denote repetitions of the letters, so they can't be negative. For example, what would a^{-1} mean? It doesn't make sense, as we can't repeat the letter a minus one times.
	  - We split $b^{2(k+d)}$ into $b^{2k} b^{2d}$. No magic here, just exponent rules.
	- We can now split this into two grammars which we will then concatenate, because CFL are closed under concatenation.
	  - One will be for the first half: $\{a^k b^{2k} \mid k \geq 0\}$
	  - $L \to aLbb \mid \varepsilon$
	  - One will be for the second half: $\{b^{2d} a^d \mid d \geq 0\}$
	  - $R \to bbRa \mid \varepsilon$
	- Now, the final grammar will be the concatenation of the grammar for the left and the right part:
	  - $S \to LR$
	  - $L \to aLbb \mid \varepsilon$
	  - $R \to bbRa \mid \varepsilon$

Chomsky Normal Form

- A CFG is in Chomsky normal form (CNF) when all productions are of the form
  - $A \to BC$
    Two non-terminals are derivable by one non-terminal
  - $A \to \alpha$
    One terminal is derivable by one non-terminal
- Note that we don't have ε here, therefore we cannot generate the empty string.

- Chomsky normal form is harder to convert into a PDA, but it's used in things like the CYK algorithm which parses context-free grammars.

Language	CFG in Chomsky Normal Form
$L(G) = \{a^n b^n \mid n > 0\}$	- $S \to XB \mid AB$ - $A \to a$ - $B \to b$ - $X \to AS$
$L(G) = \{a^n b^m c^{2n} \mid n, m >$	- $S \to XC' \mid YC'$ - $X \to AS$ - $Y \to AB'$ - $A \to a$ - $B' \to B'B \mid b$ - $B \to b$ - $C' \to CC$ - $C \to c$

Greibach Normal Form

- A CFG is in Greibach normal form (GNF) when all productions are of the form
  - $A \to \alpha B_1 B_2 \ldots B_k$ where $k \geq 0$

    One terminal followed by k non-terminals is derivable by one non-terminal
- Note that we don't have ε here, therefore we cannot generate the empty string.

- Greibach normal form makes it easier to show that there is a PDA for every CFG, and can also be used with recursive descent parsers.

Language	CFG in Greibach Normal Form
$L(G) = \{a^n b^n \mid n > 0\}$	- $S \to aSB \mid aB$ - $B \to b$
$L(G) = \{a^n b^m c^{2n} \mid n, m > 0$	- $S \to aSC' \mid aBC'$ - $B \to bB \mid b$ - $C' \to cC$ - $C \to c$

Removing epsilon and unit productions

- A production of the form $A \to \varepsilon$ is called an epsilon production
  - Assume we have another production $B \to \alpha A \beta$
  - We can get rid of the epsilon production and refine this production
  - Our new refined and only production will be $B \to \alpha\beta$

Video with example: Simplification of CFG (Removal of Null Productions)

- A production of the form is $A \to B$ is a called a unit production

- Assume we have another production $B \rightarrow \alpha$
- We can get rid of this production and refine the unit production
- Our new refined and only production will be $A \rightarrow \alpha$ (Instead of A → B → α, we are removing the step in the middle)

Helpful video with example: Simplification of CFG (Removal of Unit Productions)

- Since we can remove both epsilon and unit productions, we can conclude that for every CFG there exists another CFG that accepts the same language, except that it cannot accept epsilon:
  $L(G') = L(G) - \{\varepsilon\}$
  Where G' is the grammar G but without any epsilon or unit productions

Removing left-recursive productions

- **This is only required for conversion to GNF and therefore not examinable**
- A left-recursive production looks like the following:
  $A \rightarrow A\alpha \mid \beta$
- This rule generates a string starting with exactly one β and ending with one or more α

- It is left recursive because when we want to generate more α, we are going to prepend them to the string, i.e. we are going to add them on the left side of the string.
- In other words, we are generating the string from the right to the left, starting with the rightmost (last) symbol.
- Every left recursive rule must have the part | β because it allows us to break out of our recursion, i.e. it allows us to end "spamming" α, by replacing the non-terminal A with β at some point in our derivation.

- We want to eliminate the left recursion in $A \rightarrow A\alpha \mid \beta$
  - We know this rule produces $\beta\alpha^n$ where $n \geq 1$
  - We can easily create α^n with this new rule:
    - $D \rightarrow \alpha D \mid \alpha$
  - Now, we need to add another rule to produce the β in front:
    - $C \rightarrow \beta D$
  - So in total we need these two rules to eliminate the left-recursion:
    - $C \rightarrow \beta D$
    - $D \rightarrow \alpha D \mid \alpha$

- In general, we can have multiple alternatives that are left-recursive, and multiple alternatives to break out of our left-recursion. We have to get rid of all of them.
  - Our left-recursive production is in the form
    $A \rightarrow A\alpha_1 \mid A\alpha_2 \mid \ldots \mid A\alpha_k \mid \beta_1 \mid \beta_2 \mid \ldots \mid \beta_l$
  - For every alpha, we need the following new rule:
    - $D \rightarrow \alpha_k D \mid \alpha_k$

- For every beta, we need the following new rule:
  - $C \rightarrow \beta_l D$
- So all in all, we need the following rules for every alpha, beta:
  - $C \rightarrow \beta_l D$
  - $D \rightarrow \alpha_k D \mid \alpha_k$

- What? You still don't understand?
- No, don't feel bad! Have a look at a few examples below, they might help:

With left-recursion	**Without left-recursion**
$A \rightarrow A\alpha \mid \beta$	$A \rightarrow \beta A'$ $A' \rightarrow \alpha A' \mid \alpha$
$E \rightarrow E + T \mid T$	$E \rightarrow TE'$ $E' \rightarrow + TE' \mid + T$
$A \rightarrow AB\alpha \mid Aa \mid a$	$A \rightarrow aA'$ $A' \rightarrow B\alpha A' \mid aA' \mid B\alpha \mid \alpha$
$A \rightarrow AC \mid Aad \mid bc \mid C$	$A \rightarrow bcA' \mid CA'$ $A' \rightarrow CA' \mid adA' \mid C \mid ad$

- You don't get it? Yare yare daze... it's alright. There are just some topics that are really hard to master. We won't judge.
- What better last-resort revision technique to fall back on than Indian YouTubers?
- For this topic, I recommend this one. Sure, he leaves his answers with epsilon productions, but all the examples above are from this video and it's really easy to follow.

Converting between CFG, CNF and GNF

- For any context-free grammar, there exists a corresponding CNF and GNF that produces the same language, except epsilon.

Converting a CFG to CNF

1. Remove all epsilon and unit productions like described above

2. For all $\sigma \in \Sigma$ add a new nonterminal N_σ to our set of non-terminals N
   - We will now have a new non-terminal symbol for each of our symbols in the alphabet
   - We will use these later when we define some new productions

3. In all productions, replace any terminal σ with the the non-terminal N_σ
   - We will replace all terminal symbols in every production by the new non-terminal symbols we have generated in the step before

4. Add productions $N_\sigma \to \sigma$ to our set of productions P
   - We will need those productions to produce the terminals, since we replaced all of them with our new N_σ non-terminals.

5. While there exists a production $A \to B_1 B_2 \ldots B_k$ with $k > 2$, i.e. while there exists a production that derives more than 2 non-terminals and therefore cannot be in CNF
   - Remove this production from P
   - Add a new non-terminal C to N
     - It has to have a different name in every iteration of this "while-loop"
   - Add the following two productions to P
     - $A \to B_1 C$
     - $C \to B_2 \ldots B_k$

- Now our grammar is in CNF.
- If this is a little hard to take in, try following through the example below:

Step	Progress	Description
0	$S \to ASB$ $A \to aAS \mid a \mid \varepsilon$ $B \to SbS \mid A \mid bb$	We haven't done anything yet, I just wanted to show you the grammar first before we start converting it. It's on the left.
1	Remove $A \to \varepsilon$ $S \to ASB \mid SB$ $A \to aAS \mid a \mid aS$ $B \to SbS \mid A \mid bb \mid \varepsilon$ Remove $B \to \varepsilon$ $S \to ASB \mid SB \mid AS \mid S$ $A \to aAS \mid a \mid aS$ $B \to SbS \mid A \mid bb$ Remove $B \to A$ $S \to ASB \mid SB \mid AS \mid S$ $A \to aAS \mid a \mid aS$ $B \to SbS \mid aAS \mid a \mid aS \mid bb$ Remove $S \to S$ $S \to ASB \mid SB \mid AS$ $A \to aAS \mid a \mid aS$	In this step, we're simply removing the ε-productions and the unit productions, as detailed in the previous section. First, we remove the ε-production in A, but then it shifts it to B. So then, we remove the ε-production in B. We've gotten rid of all the ε-productions, but now we've got to get rid of all the unit productions. We get rid of the unit productions in B and S. For the one in B, we just replace it with A's contents, and for the one in S, we just remove it, because $S \to S$ is trivial.

	$B \rightarrow SbS \mid aAS \mid a \mid aS \mid bb$	
2	$a \;=>\; A_0$ $b \;=>\; B_0$	In this step, we create a non-terminal for every terminal that exists. I call these A_0 and B_0.
3	$S \rightarrow ASB \mid SB \mid AS$ $A \rightarrow A_0AS \mid A_0 \mid A_0S$ $B \rightarrow SB_0S \mid A_0AS \mid A_0 \mid A_0S \mid B_0B_0$	In this step, I convert all terminals into their respective non-terminals that I've defined in the last step. At first, this doesn't make much sense, until the next step...
4	$A_0 \rightarrow a$ $B_0 \rightarrow b$	In this step, I set up productions such that the non-terminals can produce their respective terminals. Now the previous step makes sense!
5	$S \rightarrow ASB$ $S \rightarrow A\,X_0$ $X_0 \rightarrow SB$ $A \rightarrow A_0AS$ $A \rightarrow A_0X_1$ $X_1 \rightarrow AS$ $B \rightarrow SB_0S$ $B \rightarrow SX_2$ $X_2 \rightarrow B_0S$ $B \rightarrow A_0AS$ $B \rightarrow A_0X_3$ $X_3 \rightarrow AS$	In this step, I split up productions that have more than 2 non-terminals. I create non-terminals called X_n so I don't run out of names and so that I know that any non-terminal called X is just used for shortening. Every red highlighted production is a production I need to shorten, and the blue highlighted production underneath is the shortened version.
Finish	$S \rightarrow AX_0 \mid SB \mid AS$ $A \rightarrow A_0X_1 \mid A_0S \mid a$ $B \rightarrow SX_2 \mid A_0X_3 \mid A_0S \mid B_0B_0 \mid a$ $X_0 \rightarrow SB$ $X_1 \rightarrow AS$ $X_2 \rightarrow B_0S$ $X_3 \rightarrow AS$ $A_0 \rightarrow a$ $B_0 \rightarrow b$	This isn't really a step. I've just compiled all the productions and put them into one place, so you can see the finished CNF result! This grammar accepts the same language as the one at the start, except now it's in a different form: Chomsky Normal Form.

Converting a CFG to GNF

This is not examinable according to Gennaro, but still worth reading to give further insight into context-free grammars.

1. Convert the CFG into CNF.

2. Re-order the rules such that the start rule is at the top, the next derivable rules are below and so on. In other words it should be the case that for every possible derivation we are going down in the list of rules we have, but never up.
   - This will not always be possible, but try to order them as much as possible according to this schema.
   - You can't do anything wrong here, the purpose of this ordering is to minimise the amount of work you will have to do.

3. Replace all non-terminals with non-terminals of the form $A_1, A_2, \ldots, A_k$ where k is equal to the number of all non-terminals. Start with the first rule at the top and work down, otherwise we cannot benefit from the ordering we did in the last step.

4. Check if for every production $A_k \rightarrow A_j X$, it is true that $k < j$. Check the rules in ascending order, i.e. start with the rule where $k = 0$. If we find one rule where this does not hold, we must modify this rule immediately before we continue to check other rules. We have to deal with the following two cases:
   - $k > j$ in $A_k \rightarrow A_j X$
     - Replace A_j with whatever A_j produces, i.e. derive A_j
     - If for the resulting rule still $k > j$, keep replacing until this is not the case anymore.
   - $k = j$ (left-recursive rule), so $A_k \rightarrow A_k X$
     - Eliminate left recursion according to description above.

5. Now, we are taking a closer look at every production starting with a non-terminal, i.e. every production which is not in GNF yet. For all of those rules it must hold that $A_k \rightarrow A_j X$ where $k < j$. Starting with the rule where $k =$the highest number and working in descending order, derive A_j.The result of deriving $A_j$will be either a terminal symbol or two non-terminal symbols:
   - If we get another non-terminal, repeat this step with the current rule, i.e. derive the left-most non-terminal again, check again, and so on.

- If we get a terminal symbol, our production must be in GNF, meaning that it starts with a terminal followed by some non-terminals. This means that this rule is OK, we can leave it as it is and continue to the next rule.

6. Rename all non-terminals back to their old names.

7. There might exist some rules which are unreachable from the start symbol. Remove those rules.

- Now, our grammar is in GNF. Congrats if you are not confused at this point.
- If you need to visualise this problem (or if you're masochistic), an example of this is provided below:

Step	Progress	Description
0	$S \to Ab$ $A \to aS \mid a$	Here, we haven't done anything yet! This is the grammar we want to convert to GNF. Right now, it's not in any normal form.
1	$S \to AB$ $A' \to a$ $A \to A'S \mid A'$ $B \to b$	In this step, we convert the grammar into CNF using the method described above that you should have read.
2	$S \to AB$ $A \to A'S \mid A'$ $A' \to a$ $B \to b$	In this step, we reorder the steps so that, when going through productions, you go from the top and work your way down; you don't go back up. Sometimes, this isn't always completely possible, for example the production $A \to A'S \mid A'$ uses S, which is further up.
3	$A_1 \to A_2A_4$ $A_2 \to A_3A_1 \mid A_3$ $A_3 \to a$ $A_4 \to b$	In this step, we replace all non-terminals with A_k, where k is the position of the non-terminal from the top to the bottom.
4	$A_1 \to A_2A_4$ $A_2 \to A_1 \mid A_3$ $A_3 \to a$ $A_4 \to b$ ---------- $A_1 \to A_2A_4$ $A_2 \to A_2A_4 \mid A_3$ $A_3 \to a$ $A_4 \to b$	In this step, we look for any productions $A_k \to A_jX$ where $k > j$. There doesn't actually exist any productions in the example where this is true, so I've created a sub-example to demonstrate this step. This won't affect the example we're working on. That's why the text looks like that. In the first block, we notice the production $A_2 \to A_1 \mid A_3$ where this condition is true. So we replace A_1 in that production with its derivation. Next, we notice that, in the production

	---------- $A_1 \to A_2A_4$ $A_2 \to A_3A_5$ $A_3 \to a$ $A_4 \to b$ $A_5 \to A_4A_5 \mid A_4$ ---------- $A_1 \to A_2A_4$ $A_2 \to A_5A_4 \mid A_3$ $A_3 \to a$ $A_4 \to b$ $A_5 \to b \mid a$	$A_2 \to A_2A_4 \mid A_3$, $k = j$, which means that we have left recursion. Therefore, we introduce another non-terminal to rectify this called A_5. Finally, on the last block, we notice that $k > j$ in $A_5 \to A_4 \mid A_3$. Therefore, we convert both non-terminals into their derivations, which happen to be single terminals. With this, we've completed this step.
5	$A_1 \to A_2A_4$ $A_2 \to A_3A_1 \mid A_3$ $A_3 \to a$ $A_4 \to b$ ---------- $A_1 \to A_2A_4$ $A_2 \to aA_1 \mid A_3$ $A_3 \to a$ $A_4 \to b$ ---------- $A_1 \to A_2A_4$ $A_2 \to aA_1 \mid a$ $A_3 \to a$ $A_4 \to b$ ---------- $A_1 \to aA_1A_4 \mid aA_4$ $A_2 \to aA_1 \mid a$ $A_3 \to a$ $A_4 \to b$	In this step, we look at all the productions that are not in GNF, and we substitute productions until all the productions are in GNF. We start with the highest rule (which is the furthest down in our list) and work in descending order. For example, in the first block, $A_2 \to A_3A_1$ is not in GNF. So we substitute A_3's production into it, making the first part 'a'. Next, $A_2 \to A_3$ is not in GNF, so we substitute the same thing in that, and we get $A_2 \to a$. After that, the only production that is not in GNF is $A_1 \to A_2A_4$. So we substitute A_2 into that one, and we get $A_1 \to aA_1A_4 \mid aA_4$. There are no longer any more productions that are not in GNF. We're now in Greibach normal form! But we're not done yet; our non-terminal symbols are all A_k. We need to change them back.
6	$S \to aSB \mid aB$ $A \to aS \mid a$ $A' \to a$ $B \to b$	In this step, we've just renamed our non-terminal symbols back to normal.
7	$S \to aSB \mid aB$ $B \to b$	In this step, we're just getting rid of any productions that are unreachable / unused.

		In this example, the productions for A and A' are never used, so they are deleted.

PDA and CFG conversions

- PDAs and CFG have the same power, therefore we can convert a PDA to a CFG and vice versa.

CFG to PDA

- We will do this by first converting a CFG into GNF, and then converting GNF into a one state PDA.

- Assume that our Grammar is in GNF. Then all productions will look like:
  $A \rightarrow \alpha B_1 B_2 ... B_k$ where $k \geq 0$
  One terminal followed by k non-terminals is derivable by one non-terminal

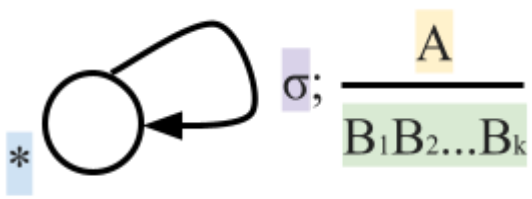
- The PDA we are going to construct will only have one state and will accept by empty stack.
- We are going to call the one and only state $*$.
- Formally, this means that the PDA will accept the string x when there exists a derivation:
  $(*,x,\perp) \Rightarrow (*,\varepsilon,\varepsilon)$

- Therefore, the PDA we will construct is going to be of the form:
  $M = (\{*\}, \Sigma, N, \delta, *, S, \emptyset)$

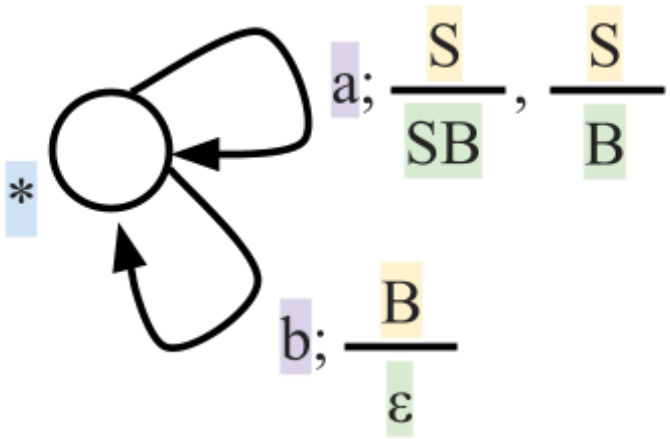
	representing	is defined as	because
$\{*\}$	set of states	a singleton set containing the state $*$	we will only have a single state in this PDA
Σ	input alphabet	terminal symbols of CFG	these are the symbols, produced by the grammar, which our PDA is going to scan
N	stack alphabet	non-terminal symbols of CFG	this is what we will push on/pop off the stack
δ	transition relation	see below	see below
$*$	start state	the state $*$	we only have a single state, so we don't have much choice where to

			start.
S	initial stack symbol	the start production of CFG	we won't have anything on the stack when we start scanning our string
∅	final states	empty set	our PDA will accept by empty stack, therefore we don't need any final states.

- Now, we still need to define the transition relation.
- We are going to add an element to the relation for every production we have in our grammar.
- Therefore, for each $A \rightarrow \sigma B_1 B_2 \ldots B_k$, add a transition:
  - Formally:
    $((*, \sigma, A), (*, B_1 B_2 \ldots B_k))$
  - Graphically:



- Example for the grammar of the language $L(G) = \{a^n b^n \mid n > 0\}$
  - $S \rightarrow aSB \mid aB$
  - $B \rightarrow b$
- The resulting PDA will have
  - A set of states $\{*\}$
  - The input alphabet $\{a, b\}$
  - The stack alphabet $\{S, B\}$
  - The start state $*$
  - The initial stack symbol S
  - The transition relation as shown in the following graphic:



- If we want to test if our PDA accepts the same language as our grammar, we can try to match every derivation of our grammar with a transition in our PDA
- We will do this by giving the leftmost derivation, which means that productions are only applied to the leftmost non-terminal in every step.

Grammar derivations	**PDA transitions**
S	$(*, aabb, S)$
$\Rightarrow aSB$	$\rightarrow(*, abb, SB)$
$\Rightarrow aaBB$	$\rightarrow(*, bb, BB)$
$\Rightarrow aabB$	$\rightarrow(*, b, B)$
$\Rightarrow aabb$	$\rightarrow (*, \varepsilon, \varepsilon)$

PDA to CFG

- This proof is pretty much just the previous proof, but in reverse!
- The steps we will go through are as follows:
    1. Prove that one-state PDAs can be converted to a CFG
    2. Prove that any PDA can be converted to a one-state PDA
- As you can see, the law of transitivity show that by doing this, we will prove that any PDA can be converted into a CFG.

1. First, we will prove that one-state PDAs can be converted to a CFG.
- The obvious thing to do first is to start off with a PDA:
    - $(\{q\}, \Sigma, \Gamma, \delta, q, \perp, \emptyset)$
- Now we're going to create a grammar (in GNF) like this:
    - $G = (\Gamma, \Sigma, P, \perp)$
    - $\forall\, ((q,\ c,\ A),\ (q,\ B_1 \ldots B_k)) \in \delta,\ A \rightarrow cB_1 \ldots B_k \in P$

- This is just the previous proof in reverse, so if you understood that, you'll get this.
- If you don't, I'll translate that maths language above for you:

Line in weird maths language	**Line in plain English**
$(\{q\}, \Sigma, \Gamma, \delta, q, \perp, \emptyset)$	This one's nice and simple; I'm just defining a PDA with only one state, q. Σ is the alphabet that the PDA accepts Γ is the set of symbols accepted onto the stack δ is the transition function $\perp$ is the initial stack symbol Where it says $\emptyset$ just means that there is no final state;

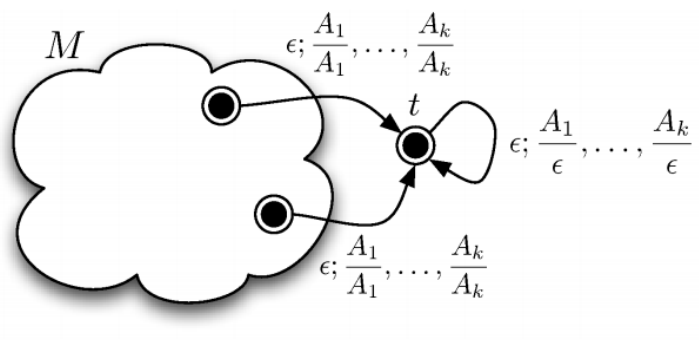
	this PDA will accept by empty stack.
$G = (\Gamma, \Sigma, P, \perp)$	Again, I'm just defining a grammar. Γ is the set of non-terminal symbols (notice how it's the same as the set of symbols accepted onto the PDA stack) Σ is the set of terminal symbols (notice how it's the same as the alphabet that the PDA accepts) P is the set of productions $\perp$ is the start non-terminal (notice how it's the same as the initial stack symbol in the PDA)
$\forall\ ((q,\ c,\ A),\ (q,\ B_1 \ldots B_k)) \in \delta,$ $A \rightarrow cB_1 \ldots B_k \in P$	For every transition that exists in the PDA, there is also a production in the grammar that takes in the input stack symbol, and outputs the input symbol followed by all the stack symbols put into the stack.

- Just by simply reversing the previous proof, we can show that a single-state PDA can be converted to a CFG.
- But not every PDA is a single-state PDA. To complete the proof, we need to convert any PDA into a single-state PDA.

2. Now, we will prove that any PDA can be converted to a single-state PDA.

- To convert a PDA to a single-state PDA, we need to store the state information inside the stack (since we're not going to be moving to any state).
- Before we do that, let's change the PDA a little bit.

- Let's change it so that when the PDA reaches a single final state, it can clear it's entire stack. Basically, we're changing it so the PDA only has one final state.



- Second, we're going to introduce a new type of stack symbol. It looks like this:
  - $\langle pAq \rangle$
- where $p, q \in Q$ and $A \in \Gamma$
- All the new stack symbols will be in a new set called Γ'
- We do this so we can store state information on the stack.

- We're also going to change Q to just {*}, where * is the name of the only state in the PDA.
- So, basically, our PDA looks like this:
  - $M' = (\{*\}, \Sigma, \Gamma', \delta', *, \langle s \perp t \rangle, \emptyset)$
  - *(where s is the start and t is the end)*

- What is δ'? Well, it's defined as follows:

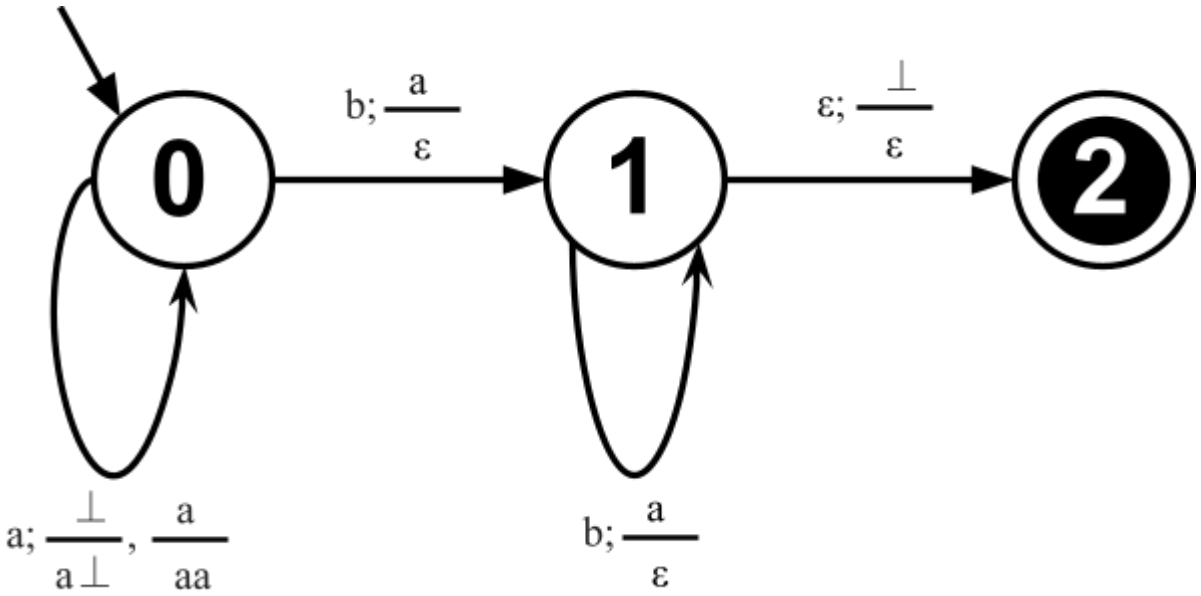
- $for\ each\ transition\ ((p, \sigma, A), (q_0, B_1 B_2 \dots B_k)) \in \delta$
- $include\ ((*, \sigma, \langle pAq_k \rangle), (*, \langle q_0 B_1 q_1 \rangle \langle q_1 B_2 q_2 \rangle \dots \langle q_{k-1} B_k q_k \rangle))\ in\ \delta'$
- $for\ all\ possible\ combinations\ of\ q_1, q_2, \dots, q_k \in Q$

- ... and when $k = 0$, then:

- $for\ each\ transition\ ((p, \sigma, A), (q_0, \varepsilon)) \in \delta$
- $include\ ((*, \sigma, \langle pAq_0 \rangle), (*, \varepsilon))\ in\ \delta'$

- To some people (normal people), this is just maths gibberish; what does this actually mean?
- Remember that PDAs are also nondeterministic. Here, each guess the PDA makes, the stack "predicts" what transitions it'll go through in the future. If the prediction is correct, it'll pass. If the prediction is wrong, that guess will "die".
- In other words, at each stage, we nondeterministically guess the remainder of the computation of the automaton and then verify it.
- This is just a way of encoding the non-deterministic nature of the states into the stack.

- This is really hard to visualise just by reading the descriptions, so let's go through an example.
- We have a PDA here that maps out the language $\{a^n b^n \mid n > 0\}$:



- Before we pick out transitions, let's have a look at all the possible stack symbols:

$\langle 0 \perp 0 \rangle$	$\langle 0a0 \rangle$
$\langle 0 \perp 1 \rangle$	$\langle 0a1 \rangle$
$\langle 0 \perp 2 \rangle$	$\langle 0a2 \rangle$
$\langle 1 \perp 0 \rangle$	$\langle 1a0 \rangle$
$\langle 1 \perp 1 \rangle$	$\langle 1a1 \rangle$
$\langle 1 \perp 2 \rangle$	$\langle 1a2 \rangle$
$\langle 2 \perp 0 \rangle$	$\langle 2a0 \rangle$
$\langle 2 \perp 1 \rangle$	$\langle 2a1 \rangle$
$\langle 2 \perp 2 \rangle$	$\langle 2a2 \rangle$

- You can think of each of these as like a possible transition that the PDA could take.
- For example, $\langle 1a1 \rangle$ could refer to the transition going from state 1 to 1, accepting a on the stack.
- Yes, there are some transitions that do not exist, like $\langle 2a2 \rangle$ or $\langle 0 \perp 2 \rangle$, but they could exist. The stack doesn't know what transitions exist, so it guesses with all possible combinations of transitions. The guesses that predict transitions that do exist will live and are more likely to predict everything correctly, and the guesses that predict transitions that do not exist end up dying.

- Let's pick out a transition from this and convert it to its single-state equivalent.
- Let's pick the transition that goes from state 0 to 0 with $\perp$ on the stack:
  - $((0, a, \perp), (0, a \perp))$
- Now let's convert it:
  - $((*, a, \langle 0 \perp 0 \rangle), (*, \langle 0a0 \rangle \langle 0 \perp 0 \rangle))$
  - $((*, a, \langle 0 \perp 0 \rangle), (*, \langle 0a1 \rangle \langle 1 \perp 0 \rangle))$
  - $((*, a, \langle 0 \perp 0 \rangle), (*, \langle 0a2 \rangle \langle 2 \perp 0 \rangle))$

  - $((*, a, \langle 0 \perp 1 \rangle), (*, \langle 0a0 \rangle \langle 0 \perp 1 \rangle))$
  - $((*, a, \langle 0 \perp 1 \rangle), (*, \langle 0a1 \rangle \langle 1 \perp 1 \rangle))$

- $((*, a, \langle 0 \perp 1\rangle), (*, \langle 0a2\rangle \langle 2 \perp 1\rangle))$

- $((*, a, \langle 0 \perp 2\rangle), (*, \langle 0a0\rangle \langle 0 \perp 2\rangle))$
- $((*, a, \langle 0 \perp 2\rangle), (*, \langle 0a1\rangle \langle 1 \perp 2\rangle))$
- $((*, a, \langle 0 \perp 2\rangle), (*, \langle 0a2\rangle \langle 2 \perp 2\rangle))$

- So if we compute that transition, our single-state PDA will compute 3 parallel guesses, depending on what's on the stack.

- Let's pick out a simpler transition and convert that:
  - $((1, b, a), (1, \varepsilon))$
- Since this transition will only pop from the stack and not push, it'll only generate one transition relation for our single-state PDA:
  - $((*, b, \langle 1a1\rangle), (*, \varepsilon))$
- So in our single-state PDA, if it sees $\langle 1a1\rangle$ on the stack, that means the guess' prediction was correct and the guess will live. If it does not see that on the stack, the guess' prediction was wrong and will die.

- Now that we've defined these, let's try inputting a string into this multi-state PDA, and see how it's single-state equivalent works by comparing the two.

Multi-state PDA	**Single-state PDA guesses**	**Description**
$(0, aabb, \perp)$	$(*, aabb, \langle 0\perp 2\rangle)$	We haven't really done anything yet. The initial stack symbol in a single-state PDA is always: $\langle [initial\ state] \perp [final\ state]\rangle$ This is possible because there's only one initial state and, like what we've defined before, only one final state.
$(0, abb, a\perp)$	$(*, abb, \langle 0a0\rangle \langle 0\perp 2\rangle)$ $(*, abb, \langle 0a1\rangle \langle 1\perp 2\rangle)$ $(*, abb, \langle 0a2\rangle \langle 2\perp 2\rangle)$	We've inputted an 'a' as input, and now our single-state PDA has started guessing! As you've (hopefully) read above, three guesses will be made in this situation; a $\langle 0\perp 2\rangle$ was read from the stack and an 'a' input was read, and now the following three guesses have been established. These three guesses go like this: What if we go from state 0 to state 0 using 'a' on the stack, then go from state 0 to state 2 using floor? What if we go from state 0 to state 1 using 'a' on the stack, then go from state 1 to state 2 using floor? What if we go from state 0 to state 2 using 'a' on the stack, then go from state 2 to state 2 using floor? I admit, some of these guesses are a little (really) dumb, but can you blame them?

		The stack doesn't know what transitions exist; it doesn't know that there isn't a transition from state 0 to state 2. So what does it do? It guesses. It guesses every single combination so that if it gets something wrong, it can fall back on another guess. If all the guesses are wrong, then there's no way the PDA can accept the string.
$(0, bb, aa\perp)$	$(*, bb, \langle 0a0\rangle\langle 0a0\rangle\langle 0\perp 2\rangle)$ $(*, bb, \langle 0a1\rangle\langle 1a0\rangle\langle 0\perp 2\rangle)$ $(*, bb, \langle 0a2\rangle\langle 2a0\rangle\langle 0\perp 2\rangle)$ $(*, bb, \langle 0a0\rangle\langle 0a1\rangle\langle 1\perp 2\rangle)$ $(*, bb, \langle 0a1\rangle\langle 1a1\rangle\langle 1\perp 2\rangle)$ $(*, bb, \langle 0a2\rangle\langle 2a1\rangle\langle 1\perp 2\rangle)$ $(*, bb, \langle 0a0\rangle\langle 0a2\rangle\langle 2\perp 2\rangle)$ $(*, bb, \langle 0a1\rangle\langle 1a2\rangle\langle 2\perp 2\rangle)$ $(*, bb, \langle 0a2\rangle\langle 2a2\rangle\langle 2\perp 2\rangle)$	Wow! Look at all those guesses! We've inputted another 'a' into our PDAs, and now, we're exponentially making guesses. None of our guesses have died yet. Because we're pushing more than we're popping, we're making more guesses about what we want to do next; there are more possibilities now. The transition here is slightly different, because we're not using floor $\perp$, but the concept is still the same. What? You want to see how this transition is converted to it's single-state PDA equivalent? Well, alright, but only because you asked so nicely: $(\,(0, a, a),\ (0, aa)\,)$ Converts to: $(\,(*, a, \langle 0a0\rangle),\ (*, \langle 0a0\rangle\langle 0a0\rangle)\,)$ $(\,(*, a, \langle 0a0\rangle),\ (*, \langle 0a1\rangle\langle 1a0\rangle)\,)$ $(\,(*, a, \langle 0a0\rangle),\ (*, \langle 0a2\rangle\langle 2a0\rangle)\,)$ $(\,(*, a, \langle 0a1\rangle),\ (*, \langle 0a0\rangle\langle 0a1\rangle)\,)$ $(\,(*, a, \langle 0a1\rangle),\ (*, \langle 0a1\rangle\langle 1a1\rangle)\,)$ $(\,(*, a, \langle 0a1\rangle),\ (*, \langle 0a2\rangle\langle 2a1\rangle)\,)$ $(\,(*, a, \langle 0a2\rangle),\ (*, \langle 0a0\rangle\langle 0a2\rangle)\,)$ $(\,(*, a, \langle 0a2\rangle),\ (*, \langle 0a1\rangle\langle 1a2\rangle)\,)$ $(\,(*, a, \langle 0a2\rangle),\ (*, \langle 0a2\rangle\langle 2a2\rangle)\,)$
$(1, b, a\perp)$	$(*, bb, \langle 0a0\rangle\langle 0a0\rangle\langle 0\perp 2\rangle)$ $(*, bb, \langle 0a1\rangle\langle 1a0\rangle\langle 0\perp 2\rangle)$ $(*, bb, \langle 0a2\rangle\langle 2a0\rangle\langle 0\perp 2\rangle)$ $(*, bb, \langle 0a0\rangle\langle 0a1\rangle\langle 1\perp 2\rangle)$ $(*, bb, \langle 0a1\rangle\langle 1a1\rangle\langle 1\perp 2\rangle)$ $(*, bb, \langle 0a2\rangle\langle 2a1\rangle\langle 1\perp 2\rangle)$ $(*, bb, \langle 0a0\rangle\langle 0a2\rangle\langle 2\perp 2\rangle)$ $(*, bb, \langle 0a1\rangle\langle 1a2\rangle\langle 2\perp 2\rangle)$ $(*, bb, \langle 0a2\rangle\langle 2a2\rangle\langle 2\perp 2\rangle)$	Oh no! The guesses! They're dying! We've taken the transition: $(\,(0, b, a),\ (1, \varepsilon)\,)$ ...which converts into: $(\,(*, b, \langle 0a1\rangle),\ (*, \varepsilon)\,)$ So, what the single-state PDA is doing is, it's looking for a $\langle 0a1\rangle$ on the stack. If it's there, the guess is right, it just pops it off and doesn't push anything else on. If it's not there, the guess is wrong and the guess dies.

	---------- $(*, b, \langle 1a0 \rangle \langle 0\perp 2 \rangle)$ $(*, b, \langle 1a1 \rangle \langle 1\perp 2 \rangle)$ $(*, b, \langle 1a2 \rangle \langle 2\perp 2 \rangle)$	As you can see, 6 guesses died, leaving only 3 guesses from each coloured partition. The top guesses show which ones survived and which ones died from the previous configuration (the gothy red and black ones are the ones that died). The bottom 3 guesses shows the top 3 living guesses after their 'b's have been inputted and stacks have been popped.
$(1, \varepsilon, \perp)$	$(*, b, \langle 1a0 \rangle \langle 0\perp 2 \rangle)$ $(*, b, \langle 1a1 \rangle \langle 1\perp 2 \rangle)$ $(*, b, \langle 1a2 \rangle \langle 2\perp 2 \rangle)$ ---------- $(*, \varepsilon, \langle 1\perp 2 \rangle)$	We've taken the transition: $(\ (1, b, a),\ (1, \varepsilon)\ )$ ... which converts to: $(\ (*, b, \langle 1a1 \rangle),\ (*, \varepsilon)\ )$ So, like before, it looks for a $\langle 1a1 \rangle$ on top of the stack. If it's there, we live. If it's not there, we die. There's only one guess here with a $\langle 1a1 \rangle$! This guess has correctly predicted every move that the PDA has done. Congratulations, guess! So now, we pop off $\langle 1a1 \rangle$ and continue. The existence of this guess is enough to accept the inputted string, right? Almost. There's one formality we need to take care of first: we need to move to state 2, the final state, to properly conclude this. Will this last-standing guess be able to conquer the final step by predicting the final transition from state 1 to state 2? Spoilers: [redacted]
$(2, \varepsilon, \varepsilon)$	$(*, \varepsilon, \varepsilon)$	It should be needless to say that this final transition looks for a $\langle 1\perp 2 \rangle$ to pop, and finds one within the final guess. We can now accept by empty stack. This string is within the language of this PDA, and this single-state PDA is equivalent to this multi-state PDA!

- If it helps to visualise it better, here's the successful guess from start to finish:
  - $(*, aabb, \qquad\qquad \langle 0\perp 2 \rangle)$

- $(*, abb, \langle 0a1\rangle \langle 1\bot 2\rangle)$
- $(*, bb, \langle 0a1\rangle \langle 1a1\rangle \langle 1\bot 2\rangle)$
- $(*, b, \langle 1a1\rangle \langle 1\bot 2\rangle)$
- $(*, \varepsilon, \langle 1\bot 2\rangle)$
- $(*, \varepsilon, \varepsilon)$

Proving closure

Union

- CFLs are closed under union
  - This means that the language of the union of two CFGs is also going to be context-free

- We are going to create a grammar that is the union of two other grammars.

- We can assume that the non-terminals of our two grammars are going to be distinct, meaning that we don't have non-terminals that appear in both grammars.
  - If we have, we are just going to rename them, it won't change the grammar.

- Let's introduce a new start symbol and a new production for this new start symbol.
  - The new production is going to link our new start symbol to the old start symbols from our two grammars like this:
    $S \rightarrow S_1 \mid S_2$
    Where S_1 is the start symbol of the first grammar and S_2 is the start symbol from the second grammar

- As for the other non-terminals and the other productions, we are just copying them from what we have in our two other grammars.

- Now we can generate everything that the first grammar is able to produce by deriving S_1 and we can also generate everything that the second grammar is able to produce by deriving S_2.

Concatenation

- CFLs are closed under concatenation
  - This means that the language of the concatenation of two CFGs is also going to be context-free

- Proving this is exactly the same as proving the union, just that our start production is going to be different:
  $S \rightarrow S_1 S_2$
  Where S_1 is the start symbol of the first grammar and S_2 is the start symbol from the second grammar

- Now we can generate everything that the first grammar is able to produce by deriving S_1 and then we can generate everything that the second grammar is able to produce by deriving S_2.

Kleene star

- CFLs are closed under kleene star
  - This means that the language of 0 or more repetitions of a CFG is also going to be context-free.

- We are going to create a grammar that represents the kleene star of another grammar.

- Let's introduce a new start symbol and a new production for this new start symbol.
  - The new production is going to look like this:
    $S \rightarrow SS_1 \mid \varepsilon$
    Where S_1 is the start symbol of the other grammar.

- As for the other non-terminals and the other productions, we are just copying them from what we have in the other grammar.

- Now we can produce 0 or more repetitions of everything that the other grammar is able to produce by deriving SS_1 or ε from S.

Intersection with regular languages

- CFLs are closed under the intersection with regular languages
  - This means that the language of a CFG intersected with a regular language is going to be context-free.
  - In contrast, the intersection of two CFL is not always context-free.

- Idea: Product construction with PDA and DFA.
  - The slides don't go into more detail than that, but I will.

  - Let's give our automata names, because that makes them easier to work with. Let's call our DFA M_D and our PDA M_P.
  - DFAs are just weak PDAs, so we can convert our DFA M_D into a PDA. It's basically just going to be a PDA that doesn't use it's stack and doesn't use any non-deterministic features. Let's call this M_P', where $L(M_P') = L(M_D)$.

  - So now we have two PDAs, M_P' and M_P. We need to perform product construction on these, so that we can construct a PDA that accepts strings accepted by **both** M_P' and M_P (because that's what *intersection* means).

- Let's call this product construct-ed PDA M_P''. Normally, product construction with PDAs don't work, because they interfere with each other on the stack (they have to share a stack). However, this case is different. How do we know that M_P and M_P' don't interfere with each other on the stack?
- We know they don't interfere because M_P' doesn't even use the stack! Remember, it's just a DFA calling itself a PDA; it doesn't even use the stack. Heck, it can't even use any non-deterministic features, or any epsilon-moves. The stack is completely free for M_P to use.

- Because we've defined a PDA that supports the intersection of regular languages, we've shown that it's closed. We can convert this PDA into a context-free grammar, showing that the language represented by this PDA is context-free, therefore showing that the intersection with regular languages is closed.

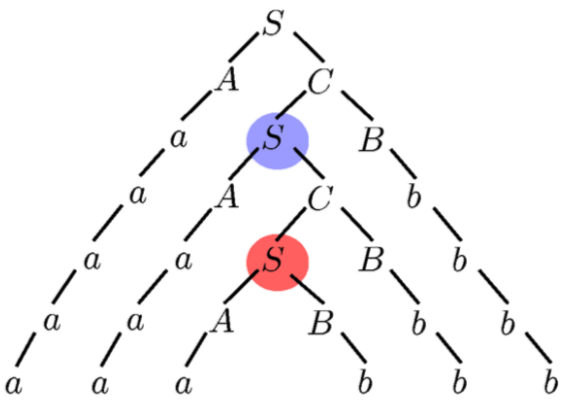
Limits of context-free languages

- Are all languages context free? No!
  - E.g. the language $\{a^n b^n c^n\}$ is not context-free, meaning that we cannot construct a PDA that accepts or a CFG that produces this language. In fact, we would need a Turing machine to accept this language, which we will see later.

- Assume our grammar is in CNF
  - Remember that we can convert any grammar to CNF if it isn't
- I'm going to use the following CNF for further explanations:
  - $S \to AC \mid AB$
  - $C \to SB$
  - $A \to a$
  - $B \to b$

    Which represents the language $\{a^n b^n \mid n \geq 1\}$

- If we start deriving a string using our CNF, we can draw the process of deriving the string as a binary tree.
- For instance, let's derive $a^3 b^3$ and draw the derivations as a binary tree:



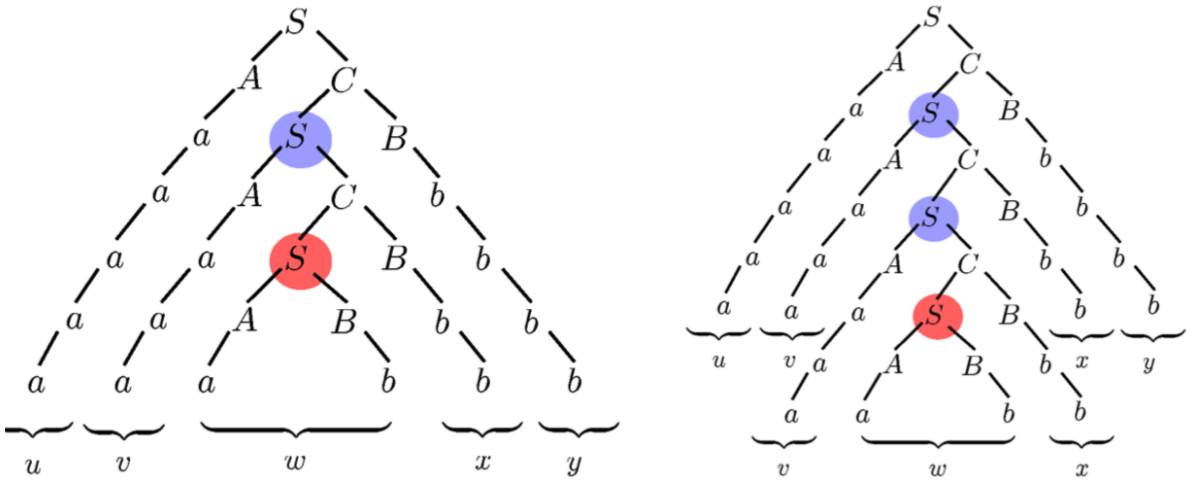
  - We can see that in the middle path of the tree, we have a long path of non-terminals. We will have this in every derivation which is long enough (we'll define what long enough means below).
  - We can also see that there are some non-terminals that are repeated.

- If we go one step down in the tree, we are going to at most double its width
  - This is because our grammar is in CNF, which allows us to derive at most 2 non-terminals in a single rule.
- Therefore, the derivation tree of a string of size 2^k must have a depth of at least k

- Suppose that there are n non-terminals. In any derivation tree of a string of length at least 2^{n+1} there must be a path where some non-terminal appears at least twice, i.e. is repeated.

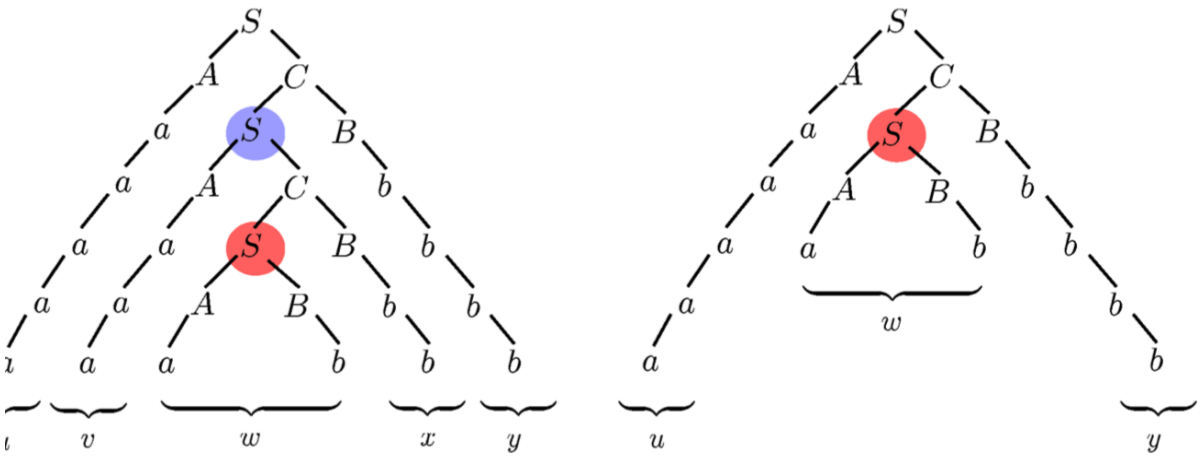
- The derivation tree of a string of size 2^{n+1} must have a depth of at least $n + 1$.
- However, we only have n non-terminals, therefore we need to repeat at least one of our n non-terminals.

- If we see a repeated non-terminal, we can start pumping this non-terminal into the tree.



- Here, we split our tree into $uvwxy$ where the tree generated by the blue S covers vwx and the tree generated by the red S covers w only. Everything outside of the blue S will be covered by u and y.
- Now, we are copying the subtree of the blue S and paste it into the position of the red S .
- Every time we do this, we are essentially adding another part v and another part x to our tree.

- We can also replace the subtree of the blue S with the subtree of the red S. By doing this we are taking v and x out of the tree completely.



- This act of ‘removing’ and ‘pumping in’ v and x forms the basis for the pumping lemma for context-free languages.

Pumping (🎃) Lemma for context-free languages - contrapositive form

- We can use the contrapositive form of the pumping lemma for context free languages to show that a language is not context free. We cannot use the pumping lemma to show that a language is context free!

- It works by splitting up the language into parts, then "pumping" a subpart until you get a string that isn't in the language. This is similar to what we have just done before.

- The different steps of the proof can be understood as a game versus a demon.

1. The demon chooses a number k such that $k \geq 0$.
    - We cannot choose a value for k, we only know that it is greater than or equal to 0.

2. We choose a string $z \in L$ such that $|z| \geq k$.
    - We can choose any string z that is within the language. However, we have to make sure that, no matter what k is, our string z is always in the language. We do not need to cover the whole language with our string!
    - The length of z must be at least k. A way to make sure that this is always the case is to have at least a part in z that is equal to something to the power of k.
    - As a general tip, try to make z as trivial as possible, like in the following example:
    - For instance, we could let $z = a^k b^k c^k$ to make sure that z is at least k long (a, b, c are symbols from the alphabet).

3. The demon splits z into $uvwxy$ such that $|vx| > 0$, i.e. $vx \neq \varepsilon$, and $|vwx| \leq k$.
    - You cannot choose how the string is going to be split!
    - What you do know is that the vx part of the string is not going to be empty. However, one of v or x can still be empty.
    - You also know that the length of the vwx is at most k.
    - You can think of uvw as a window you can slide across what you defined as z .
    - Therefore, we have to list all possible cases in which $uvwxy$ can be split and proceed with each of them to the next step.

    - For instance, if $z = a^k b^k c^k = uvwxy$, we have to consider the following cases:
        i. Case 1: vx contains only one repeated letter: Either only a's, or only b 's, or only c's
            - e.g. $uvwxy$=aaaaabbbbbccccc
        ii. Case 2: either v or x (but not both) is a word that contains more than one kind of symbol.
            - e.g. $uvwxy$=aaaaabbbbbccccc
        iii. Case 3: v and x do not have any symbols in common.

- e.g. $uvwxy$=$aaaaabbbbbccccc$

iv. It cannot be the case that vwx contains all three letters because the length of vwx is at most k.

4. We pick an $i \geq 0$ such that the string $uv^iwx^iy \notin L$.If this is the case we have proven that the language is not context-free!
   - We can pick any i we want as long as it is equal to or greater than 0.
   - Our goal is to choose an i such that the resulting string is outside of the language.
   - We have to do this for every case we defined in the previous step.
   - Here is how to do it for the example cases above:
     i. Let $i = 2$
     ii. Case 1: vx contains only one repeated letter: then uv^2wx^2y is going to have either too many of $a's$, $b's$ or $c's$
     iii. Case 2: either x or v contains two different symbols: then uv^2wx^2y is going to have too many of two of $a's$, $b's$ or $c's$.
     iv. Case 3: v does not contain the same symbol as x. then uv^2wx^2y is going to have too many of two of $a's$, $b's$ or $c's$.

5. You **MUST** say "Therefore L is not context-free", if you don't, you will lose marks.

- If this still doesn't make any sense, look at a few examples below, and then read the general case above again.

Example	**Solution**
$L = \{a^n b^n c^n\}$	1. The demon chooses a number k such that $k \geq 0$. 2. We choose a string $z \in L$ such that $\|z\| \geq k$. - Let's choose $z = a^k b^k c^k$. - We can see that $\|z\| \geq k$. 3. The demon splits z into $uvwxy$ such that $\|vx\| > 0$, i.e. $vx \neq \varepsilon$, and $\|vwx\| \leq k$. - Let's consider all the different cases: - Case 1: vx contains only one repeated letter: Either only a's, or only b's, or only c's - Case 2: either v or x (but not both) is a word that contains more than one kind of letter. - Case 3: v and x do not have any letters in common. 4. We pick an $i \geq 0$ such that the string $uv^iwx^iy \notin L$. - pick $i = 2$, then $uv^2wx^2y \notin L$ - Case 1:

	i. Our resulting word will have one letter repeated more often than the other two letters. - Case 2 i. Our resulting word is not going to be of the form a*b*c*. - Case 3: i. Our resulting word is going to have more of two of the letters but the number of occurrences of the third letter remains unchanged. 5. Therefore, L is not context-free
$L = \{a^{4n} b^{n} a^{4n}\}$	1. The demon chooses a number k such that $k > 0$. 2. We choose a string $z \in L$ such that $\lvert z \rvert \geq k$. - Let's choose $z = a^{4k} b^{k} a^{4k}$. - We can see that $\lvert z \rvert \geq k$. 3. The demon splits z into $uvwxy$ such that $\lvert vx \rvert > 0$, i.e. $vx \neq \varepsilon$, and $\lvert vwx \rvert \leq k$. - Let's consider all the different cases: - Case 1: vx contains only one repeated letter: Either only a's, or only b's. - Case 2: either v or x (but not both) is a word that contains more than one kind of letter. - Case 3: v and x do not have any letters in common. 4. We pick an $i \geq 0$ such that the string $uv^{i}wx^{i}y \notin L$. - Pick $i = 2$, then $uv^{2}wx^{2}y \notin L$ - Case 1: i. We are changing the number of either the a's or the b's. Therefore, the number of a's won't be eight times the number of b's anymore. - Case 2 i. Our resulting word is not going to be of the form a*b*a*. - Case 3: i. We are changing the number of either the first a's and the b's or the b's and the second a's. Therefore, the number of the first a's won't match with the number of the second a's anymore. 5. Therefore, L is not context-free
$L = \{a^{2^{n}} \mid n \geq 0\}$	1. The demon chooses a number k such that $k > 0$. 2. We choose a string $z \in L$ such that $\lvert z \rvert \geq k$. a. Let's choose $z = a^{2^{k}}$. b. We can see that $\lvert z \rvert = 2^{k} \geq k$.

	3. The demon splits z into $uvwxy$ such that $\lvert vx \rvert > 0$, i.e. $vx \neq \varepsilon$, and $\lvert vwx \rvert \leq k$. a. This time we only have to consider one case as we only have one letter a in our window vwx. b. We know that we have at least 1 and at most k a's in our window, which we are going to pump in the next step. 4. We pick an $i \geq 0$ such that the string $uv^i wx^i y \notin L$. a. pick $i = 2$ b. $\lvert uv^2 wx^2 y \rvert = \lvert uvwxy \rvert + \lvert vx \rvert = 2^k + l$ where we define l to be $\lvert vx \rvert$ c. Define the bounds of l to be $1 \leq l \leq k$ i. This is because $\lvert vx \rvert > 0$ and $\lvert vwx \rvert \leq k$ when $w = \varepsilon$ d. I know that $uv^2 wx^2 y \notin L$because its length, $2^k + l$, cannot be expressed as some number 2^m. e. This is because $2^k + l$ is going to be strictly between $2^k$and 2^{k+1}: $2^k < 2^k + l < 2^{k+1}$ f. If you have doubts, check the bounds of l again and also try to plug in k and l (imagine you are the demon) 5. Therefore, L is not context-free

Computability theory

Turing machines

- With DFAs and NFAs, we had states and transitions.
- With PDAs, we had states, transitions and a stack.
- Now, with Turing machines, we have states, transitions and a “tape”, which is basically a linked list storing the input string at each node.
- With this new data structure at our disposal, we can do even more things than PDAs.
- In fact, according to the Church-Turing thesis, Turing machines (TMs, not to be confused with Pokémon TMs) are just as powerful as algorithms.

- However, TMs do not cover all languages. They’re not **that** good. See Undecidable Problems for languages not covered by TMs.

- So, what exactly are TMs? I’ll go through them informally and then formally.

1. Informally

- They’re basically like PDAs, but instead of a stack, you get a tape, where you can move left or right on each transition.
- You can think of the tape as like a doubly linked list, where you can traverse left and right.
- The input string is on the tape, and we start at the far left and usually work our way to the right.

- Here is an example of a tape in the middle of a TM execution:

$\vdash$	a	$\sqcup$	c	$\dashv$

- The $\vdash$ symbol means the ‘start’, it’s a left marker.
- The $\sqcup$ symbol means ‘empty’, there’s nothing here.
- The $\dashv$ symbol means the ‘end’, it’s a right marker.
- The part that’s shaded in red is the current position we’re on, and the other blue parts are all the other positions.

2. Formally

- Formally, a TM is a 9-tuple:
- $M = (Q, \Sigma, \Gamma, \vdash, \sqcup, \delta, s, t, r)$
- where
  - Q is the finite set of states
    - |Q| ≥ 3 : accept, reject and start states as well as other states
  - Σ is the input alphabet

- Γ is the tape alphabet ($\Sigma \subset \Gamma$: the input alphabet is a proper subset of the tape alphabet)
- $\vdash \in \Gamma - \Sigma$ is the left endmarker (left endmarkers are part of the tape alphabet, but not the input alphabet)
- $\sqcup \in \Gamma - \Sigma$ is the blank symbol (blanks are part of the tape alphabet, but not the input alphabet)

- $\delta: (Q - \{t, r\}) \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}$is the transition function
  - In English, it's in the form $\delta(a, b) \rightarrow (c, d, e)$ where
    - a is a possible state that is **not** the accept / reject state
    - b is a tape symbol that must be at our current position on the tape
    - c is a state to go to
    - d is a tape symbol to write to the tape
    - e is either left L or right R, to traverse the tape
- s is the start state
- t is the accept state
- r is the reject state

- When performing transitions, we cannot overwrite $\vdash$ and we cannot move left if we're at the left endmarker.
- We can write element in Γ onto the tape (including $\vdash$ and $\sqcup$).

Decidability

Recursive and R.E languages

- A **total turing machine** is a TM that halts on every input (does not loop infinitely)
- A turing machine that isn't total can hang (like Microsoft Visual Studio)
- A language is **recursive** if there exists a total Turing machine that accepts it.
- A language is **recursively enumerable** (r.e) if there exists a Turing machine that accepts all of the strings in that language.
- Remember that all recursive languages are recursively enumerable: R $\subset$ R.E

Proof: Recursive Sets Closed under Complement

- **Recursive sets are closed under complement** (if L is recursive, ~L is also recursive)
- Remember how we proved DFAs are closed under complement? It's the same ordeal here.
- Let's say we have a language 'A' that is accepted by a TM 'M'. If we can construct a TM for ~A, then we can prove this lemma.
- Just swap the accept state and the reject state. This way, ~A will accept $\Sigma^* - A$.
- In other words, whenever A accepts something, ~A will reject it, and whenever A rejects something, ~A will accept it. That's literally the definition of a complement!

- Lemma: If L and ~L are r.e, then L is recursive
- Let's say that the TMs 'M1' accepts 'L' and 'M2' accepts '~L'.
- If we can construct a total TM 'N' that simulates both M1 and M2, then we can show that this lemma is true.

- Why the existence of N proves this lemma:
- M1 accepts everything in L, so it never hangs for strings in L.
- M2 accepts everything in ~L, so it never hangs for strings that are not in L.
- When 'N' simulates M1 and M2, that means if M1 accepts, N accepts, and if M2 accepts, N rejects, so that the language of N is the same as M1.
- But M1 never hangs for L, and M2 never hangs for anything not in L. If 'N' will halt when either M1 or M2 halts, that means 'N' will never hang for L **and** never hang for anything not in L. But that just means 'N' will halt on any string!
- Therefore 'N' won't hang on **any** string, making the language L recursive.

- Constructing N:
- We need to create a TM 'N' that simulates both M1 and M2.
- Remember subset construction back in Automata theory? Well, this is similar to that.
- Basically, we have two tapes, stuck to each other. The top tape is M1, and the bottom tape is M2. $a_i \in L$ and $b_i \in$~L

$\vdash$	a_1	a_2	a_3	a_4
$\vdash$	b_1	b_2	b_3	b_4

- But how do we know what position M1 and M2 are at? Currently, we can only select a_i and b_i together, where i is the same number for both 'a' and 'b', like a_1, b_1 or a_4, b_4.
- That's why we introduce a 'hat':

$\widehat{\vdash}$	a_1	a_2	a_3	a_4
$\widehat{\vdash}$	b_1	b_2	b_3	b_4

- Now we know what position M1 and M2 are on! For example, if our simulation of M1 is on the tape element a_1 and our simulation for M2 is on the tape element b_4, we can represent that on our tape like so:

$\vdash$	$\widehat{a_1}$	a_2	a_3	a_4
$\vdash$	b_1	b_2	b_3	$\widehat{b_4}$

- Now that our tape is all set up, all we have to do is set the instructions for the TM to follow:

1. Scans the tape to find a symbol with a hat in the "upper" section of the tape. Then, according to the transitions of M1, it performs M1's move.
2. If M1 accepts then N accepts;
3. Scans the tape to find a symbol with a hat in the "lower" section of the tape. Then, according to the transitions of M2 , it performs M2's move.
4. If M2 accepts then N rejects;
5. Go back to step 1

- The states work just like subset construction; we have tuples as states, in the form:
- (s,k)
- Where s is a state in M1 and k is a state in M2.

- To complete this formal proof, all we need now is a short statement to show that 'N' is total:
- "Any x is either in L or $\Sigma^* - L$: $x \in L \cup \Sigma^* - L$ So it is accepted by either M1 or M2. So N will eventually accept or reject, thus N is total and so L is recursive."
- Just like that, the proof is done!

- <u>Lemma: If L is r.e and not recursive, ~L is not r.e</u>
- Let's say TM 'M1' accepts 'L'. If 'L' is r.e and not recursive, then that means 'M1' loops on some strings that are not in L (which are instead in ~L)
- Let's try to construct a TM 'M2' that accepts ~L. This Turing Machine would have to accept all strings that are not in L.
- However, as stated before, 'L' is not recursive, so 'M1' loops on some strings that are not in L. Because ~L is every string that is not in L, M2 would have to loop on some strings that are in ~L, meaning that M2 can never fully accept ~L, and no Turing Machine can ever accept ~L, making it not recursively enumerable.

(Semi) Decidable properties

- First of all, let's define what a **property** is. A property of a string is a predicate that tells you something about the string. It is either true or false.
- A few examples are:
    - The string has length 2 : {w | #w = 2}
    - There is a letter 'g' in the string : {w | 'g' ∈ w}
    - There is an even number of letters in the string : {w | #w mod 2 = 0}
    - The string contains the substring "jojo" : {w | "jojo" ∈ w}

- A property is **decidable** if all strings with that property form a language that is recursive.
- For example: the property that the given string is equal to "duwang" (because all finite languages are recursive)

- A property is **semi-decidable** if all strings with that property form a language that is recursively enumerable.
- For example: the property that the given string is an encoding for a Turing Machine that halts (you can simulate the TM and accept if it halts, but if it loops, then your simulation will loop, making this recursively enumerable but not recursive).

- Remember:
    - semi-decidable ⇔ recursively enumerable
    - decidable ⇔ recursive

Universal Turing machines

Multiple tapes

- Are TMs with multiple tapes more powerful than TMs with only one tape?
- No! Remember the section "Recursive and R.E languages" where we constructed a dual-tape TM out of a single-tape TM?
- We can apply the same logic, but with any number of tapes.
- If we can make a multi-tape TM out of a single-tape TM, that means multi-tape TMs are just as powerful as single-tape TMs.

Simulating Turing machines

- Turing machines are so powerful that they can simulate other Turing machines! Think of it like a virtual machine, like VMware or VirtualBox.
- These Turing machines are called Universal Turing machines (UTM).

Weird maths language	**Plain English**
There exists a TM U such that $L(U) = \{M\#x \mid x \in L(M)\}$ Where $M\#x$ is an encoding of the TM 'M', followed by a '#', followed by an encoding of 'x' in M's input alphabet.	Like all Turing machines, they have a string input. Universal Turing machines only accept strings that fit this pattern: $M\#x$ ... which is an encoding of the TM 'M', a separating character (in this case #), and the encoding of the input string 'x'. First of all, the UTM checks if M#x is a valid encoding. If it isn't, it'll reject it straight away. If the TM 'M' accepts 'x', then the UTM will accept M#x If the TM 'M' rejects 'x', then the UTM will reject M#x The UTM does this by simulating M on 'x'. If it finds that M accepts, then the UTM will accept. If it finds that M rejects, then the UTM will reject.

- We use UTMs without even thinking about it, for example interpreters (for Java, Python, Ruby etc.) and virtual machines.

Encoding Turing machines

- So how do you actually encode a TM and an input into M#x?
- Well, the details aren't really important, but there's a method from the Kozen book that converts all TM parameters into numbers, and then encodes them into one big number like this:
- $0^n 1 0^m 1 0^k 1 0^s 1 0^t 1 0^r 1 0^u 1 0^v$
- where:
  - $\{0, 1, 2, 3, ..., n - 1\}$is the set of states
  - $\{0, 1, 2, 3, ..., m - 1\}$is the tape alphabet where the first k numbers are the input alphabet
  - Start, accept and reject states are s,t and r respectively
  - Blank symbol is u and the endmarker is v
- So you use the number of zeroes to determine what state / alphabet character the encoding is referring to.

- To encode the transitions, this encoding could be followed up by:
- $0^p 1 0^a 1 0^q 1 0^b 1 0$
- ... which is the encoded form of $\delta(p, a) = (q, b, L)$, or
- $0^p 1 0^a 1 0^q 1 0^b 1 00$
- ... which is the encoded form of $\delta(p, a) = (q, b, R)$

Constructing a UTM

- First of all, a UTM checks if the input string M#x is of the right encoding.
- The rules for encoding could be anything; there's lots of ways to do it. One of which is defined in the previous section.

- Second, we use three tapes to store:
  - Description of M
  - Contents of M's tape
  - M's current state and position on the tape

- Third, the UTM will loop through the following steps:
  1. looks at M's current state and head position (tape 3);
  2. reads the tape contents at the correct position (tape 2);
  3. reads the relevant transition (tape 1);
  4. simulates transition, updating tape, state and head position;
  5. accepts if M accepts, rejects if M rejects.

Halting problem

- The Halting problem goes like this:
- **Can you come up with a TM that takes in an encoding M#x and accept if M halts on x and rejects if M loops on x?**
- Or, more formally...
- **Is the set** $HP = \{ M\#x \mid M\ halts\ on\ x \}$ **recursive?**
- No.

Proof by contradiction

- Here's an intuitive proof before we move onto diagonalisation.
- Ever heard of the Pinocchio paradox?
- What happens when Pinocchio says "My nose will grow?"
- If it's a lie, it won't grow. But if it doesn't grow, Pinocchio will be lying, so it will grow. But then he's telling the truth, so it won't grow, and we're just going around in circles.
- This proof is just like that.

- Let's say we have a TM called K that takes in an encoding M#x and accepts if it halts, and rejects if it loops. Basically, K solves the HP.

- Let's create another TM called P that takes in an encoding of M, and runs K on M#M. It goes into an infinite loop if K accepts, and halts if K rejects.
- What if we run P on P? Will it loop or halt?
- If it loops, that means P halts. But if P halts, that means it has to loop. We're going around in circles again; there is a contradiction!

- If that's a bit too wordy for you, here's some pseudocode showing this:

```
P on M:
     call K on M#M
     if K says halt --> go into infinite loop
     if K says doesn't halt --> halt

P(P) = ?
```

- Really good computerphile video explaining it via contradiction: https://www.youtube.com/watch?v=macM_MtS_w4

Proof by diagonalisation

- We can also show how P is going to behave via diagonalisation
- We can define the output of our TM K, which is supposed to decide the HP, by constructing a table
- If K exists then we will be able to list the halting behaviour of all TMs in this table

	ε	0	1	00	01	10	11 ...
M_{ε}	H	L	L	H	L	H	L
M_{0}	L	L	H	H	L	H	L
M_{1}	H	L	L	L	L	L	L
M_{00} ...	L	H	H	L	H	L	H

- Where the left column is all the possible Turing machines and the top row is all the possible inputs.

- We can create the TM P by taking a diagonal snippet of this table:

	ε	0	1	00	01	10	11 ...
M_{ε}	H	L	L	H	L	H	L
M_{0}	L	L	H	H	L	H	L
M_{1}	H	L	L	L	L	L	L
M_{00} ...	L	H	H	L	H	L	H

- Then, we flip all of the cells. Our TM P behaves like this:

	ε	0	1	00	01	10	11 ...
P	L	H	H	H	H	L	H

- This new TM P can't fit anywhere on our original table
- The TM K cannot exist because the TM P wasn't anywhere on our table

- Examples on how this table entry is equal to P:

  P on (M_{ε}):

  Call K on $M_{\varepsilon}\#\epsilon$

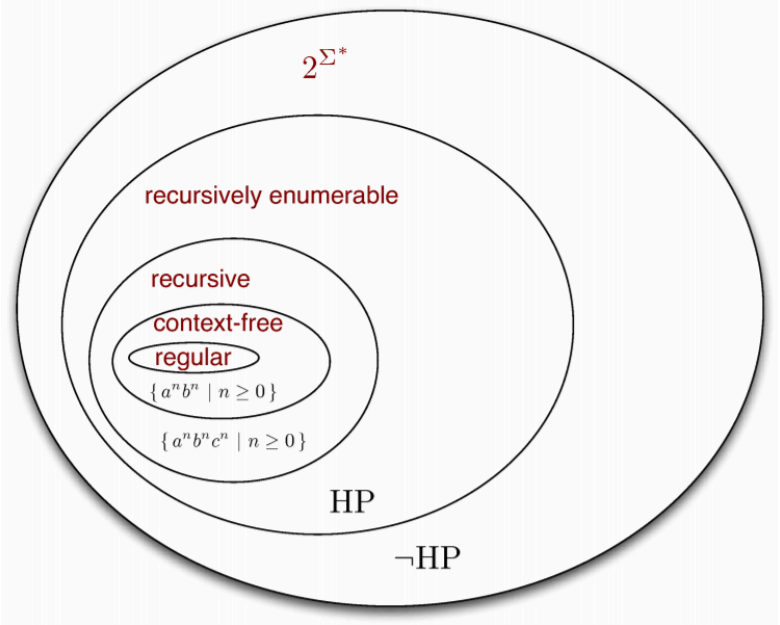
  According to the table K returns H

  Therefore we will go into an infinite loop

  P on (M_{0}):

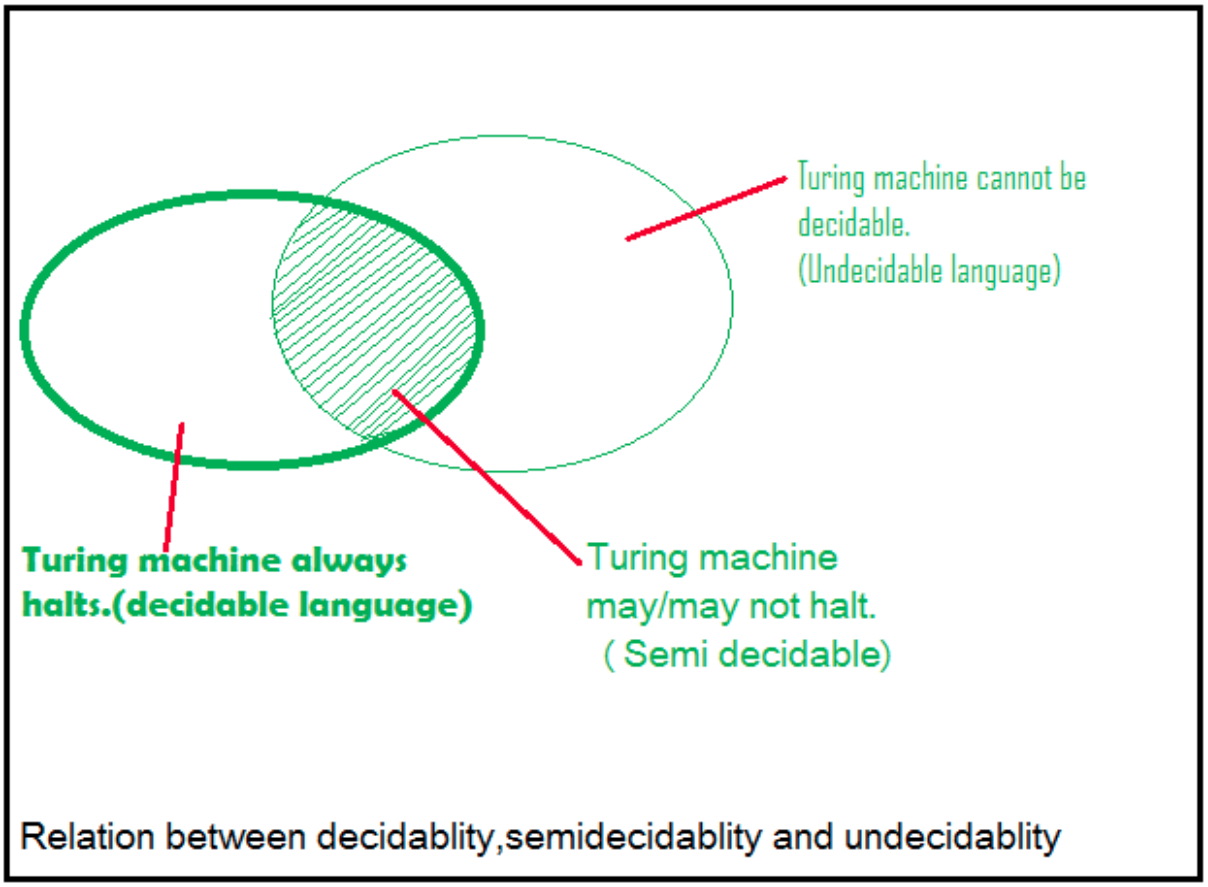
Call K on M_0#0
According to the table K returns L
Therefore we will halt

- So we've proved that there's no total Turing machine for this. However, the set HP is recursively enumerable.
- Why? Because if the TM simulates M#x and it halts, the TM will accept this encoding. On all input strings that the TM accepts, the TM will accept it and not loop (because that's the definition of an accepting string; it won't loop).
- So HP is recursively enumerable, but not recursive. That also means ~HP is not recursively enumerable (see proof in Recursive and R.E languages).
- If ~HP is not recursively enumerable, there is no way of knowing if a Turing machine will loop on an input string or not.



Decidable / Undecidable problems

- A **decidable** problem is a decision problem for which there exists a total TM that decides it.
    - The total TM will accept for all yes/true instances and reject for all no/false instances of the problem.
    - Therefore, it will never loop for any input.
- An **undecidable** problem is a decision problem for which there does not exist a total TM that decides it.
    - However, there might exist a TM that accepts yes/true instances and loops or rejects on no/false instances of the problem, making it **semi-decidable**.



- Ever heard of the membership problem? It goes like this:
- $MP = \{M\#x \mid x \in L(M)\}$
- Basically, can you make a TM that decides if a TM M will accept an input x?
- Sounds simple, right? Well, it's actually impossible. It's an *undecidable* problem.

- Why is the membership problem undecidable?
- Because if it was decidable, you'd be able to solve the Halting problem, which we know cannot be solved because of our proof from before!

  If we had a TM K that solved the membership problem, we could create a TM N that would solve the Halting problem.
- Let's construct a new TM N with input M#x:
  - Constructs a new TM M' which will change our existing machine M: All reject states become accept states, and all other states stay the same as before. Basically, M' will accept iff M halts on x.
  - Simulates K on M'#x. If K accepts, then make N accept. If K rejects, then make N reject.
- Here, N solves the Halting problem because it'll accept if M halts on x and reject if M loops on x.
- We know that the Halting problem is undecidable, so that means the membership problem must be undecidable.

- By doing this, we've shown that the membership problem is just as hard as the Halting problem, because we can use the membership problem to solve the Halting problem.
- This is called a reduction, at which we will take a closer look soon.

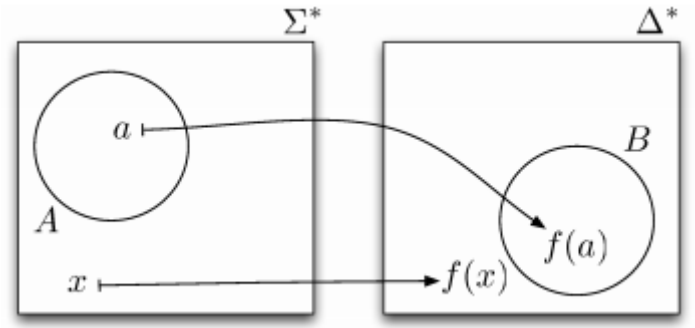
Reductions

- "Reduction" is an operation between two languages.
- It is used to show the decidability of a language.

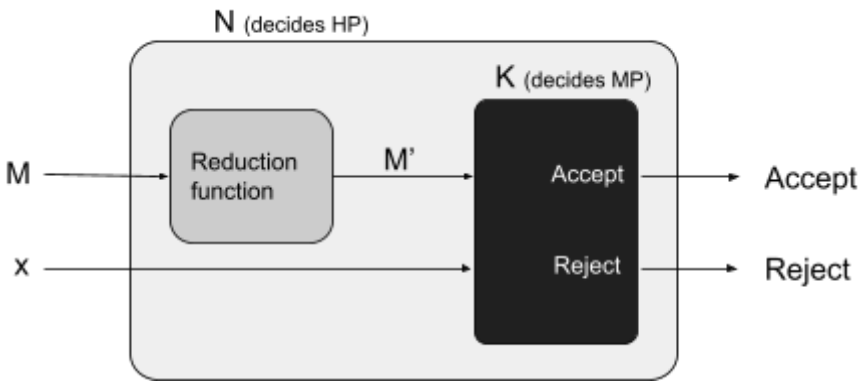
- For example, in the previous section we "reduced" the Halting problem to the membership problem, thus showing that the membership problem is undecidable (because we know the Halting problem is undecidable).
- To represent this, we say that HP ≤ MP and, for our previous example, HP ≤ A.

- Informally, a reduction is where you map one language to another language, so if you solve the second language with a TM, you can use that solution to solve the first language.
- Like what we did with MP and HP; we reduced HP to MP, so if we solve MP, we can solve HP.

- Formally, a function $f: \Sigma^* \to \Delta^*$ is <u>computable</u> when there exists a total TM 'K' that when started with $x \in \Sigma^*$ on its tape, eventually halts with $f(x) \in \Delta^*$ on its tape.
- A <u>reduction</u> of $A \subseteq \Sigma^*$ to $B \subseteq \Delta^*$ is a computable function $f: \Sigma^* \to \Delta^*$ s.t. $x \in A \Leftrightarrow f(x) \in B$
- So a reduction function maps the inputs of one TM to another TM.

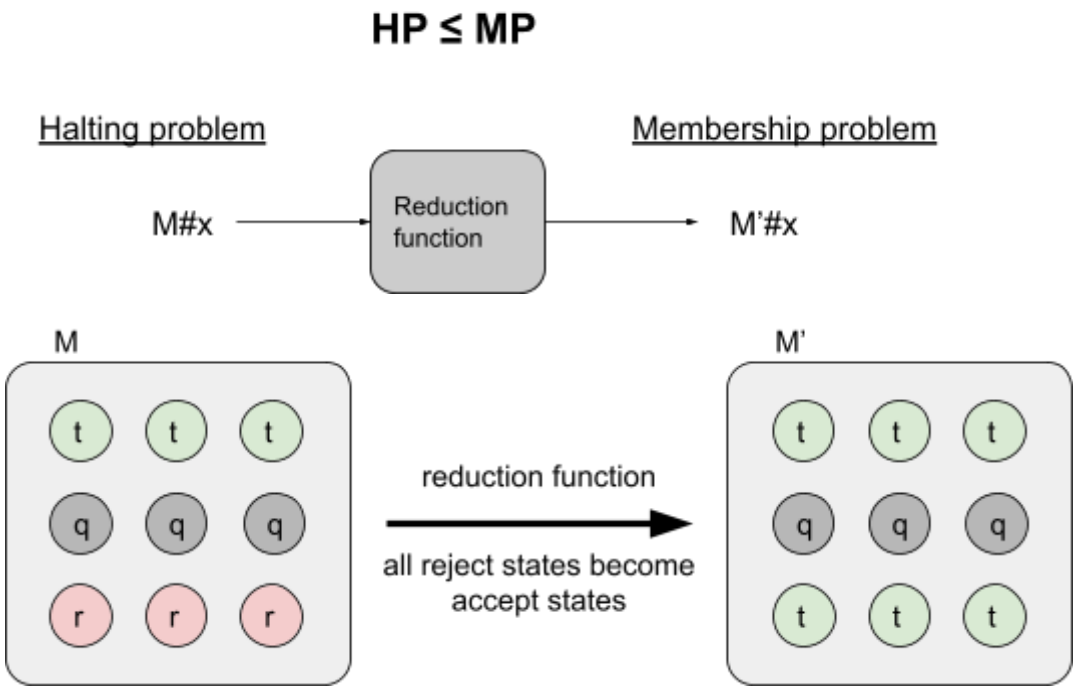


- For example, remember the previous section when we showed how to solve the Halting problem if we had a TM K that could solve the membership problem by creating a TM N that could solve the Halting problem?
- Here's a diagram of what the TM N actually does:



- What N actually does is it takes in a Halting problem input, and uses the reduction function to turn it into a membership problem input.
- That's all reduction is! It's just a function converting input of one problem into an equivalent other.

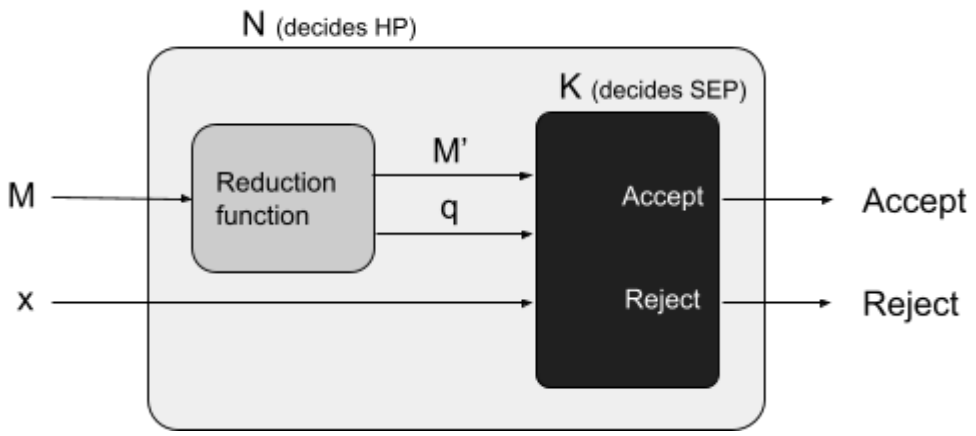
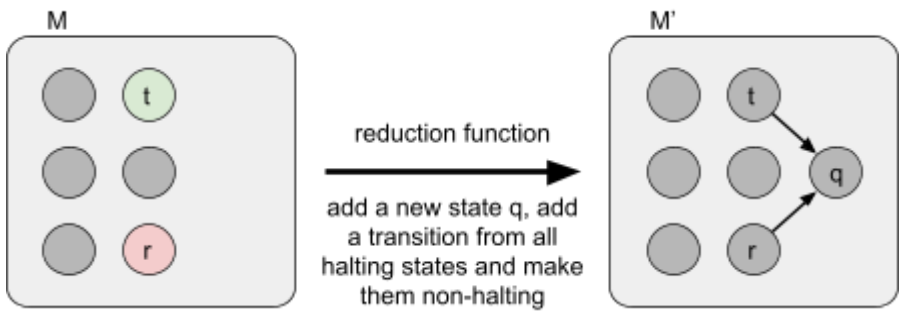
- Let's pluck out that reduction function and take a further look:

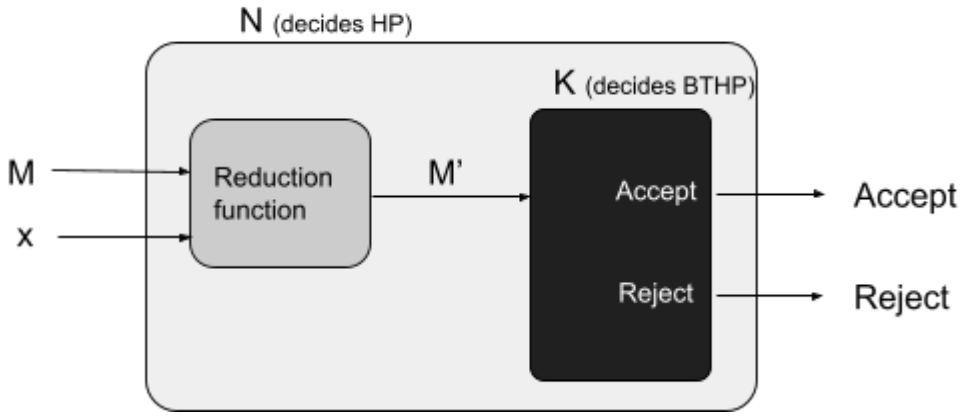
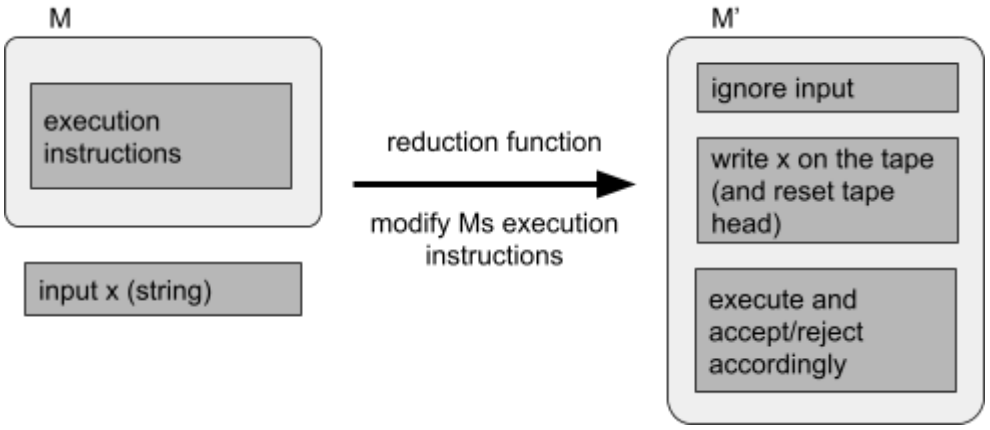


- Let's say we have our TM for the Halting problem N and our TM for the membership problem K again.
- If we run N on M#x, it should yield the same result as running K on M'#x.

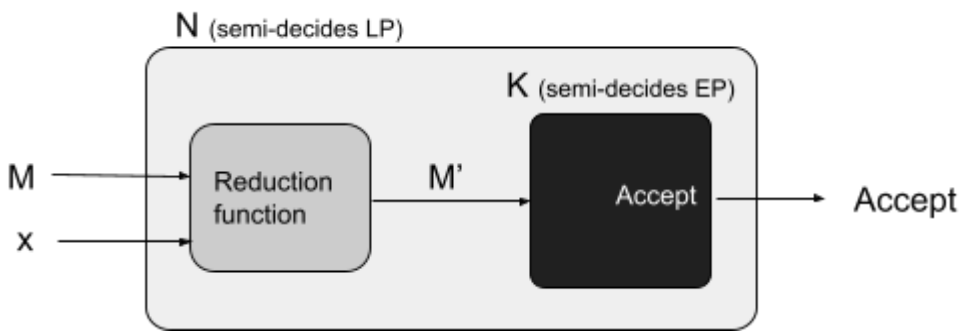
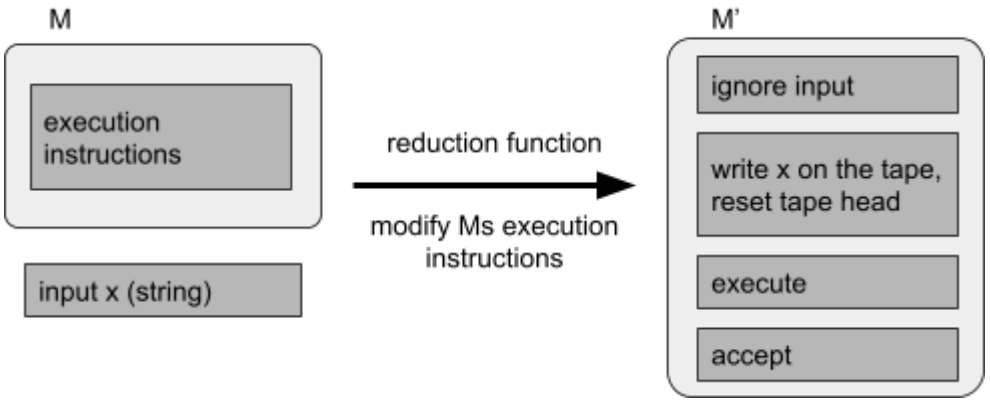
- In summary, just think this:
- If you see A ≤ B, then you can convert the problem A into the problem B.

- Let's look at some more examples

<table>
<tr><td rowspan="5">State Entry Problem (SEP)</td><td>Decide whether a TM M’ enters a given state q on input x</td></tr>
<tr><td>Assume that we already have a TM K that decides the SEP. Create a TM N which decides the HP using a reduction from the HP to the SEP.</td></tr>
<tr><td>N on inputs M and x (e.g. as an encoding M#x):
1. Construct a new TM M’ from M
a. Add a new state q which is not in M
b. From all halting states add a new transition to q
c. Make all halting states non-halting states
2. Simulate K on inputs M', q, x (e.g. as an encoding M’#q#x)
3. Accept if K accepts, reject if K rejects</td></tr>
<tr><td> 
</td></tr>
<tr><td> 
</td></tr>
<tr><td>Since we know that the HP is undecidable, the SEP must also be undecidable</td></tr>
<tr><td rowspan="3">Blank Tape Halting Problem (BTHP)</td><td>Decide whether a TM M’ halts on the blank tape (no input).
Decide whether a TM M’ halts on input epsilon.</td></tr>
<tr><td>Assume that we already have a TM K that decides the BTHP. Create a TM N which decides the HP using a reduction from the HP to the BTHP.</td></tr>
<tr><td>N on inputs M and x (e.g. as an encoding M#x):
1. Construct a new TM M’ from M
a. Ignore any input
b. Write x on the blank tape and reset the tape head</td></tr>
</table>

<table>
<tr><td rowspan="4"></td><td>c. Execute M with x on the tape and return the result
2. Simulate K on input M’
3. Accept if K accepts, reject if K rejects</td></tr>
<tr><td> 
</td></tr>
<tr><td> 

If you are confused about M’:
● M’ is a special version of M
● M’ has the same instructions as M, however, it has some instructions prepended to the other instructions
○ We will ignore any input given to M’
○ We need to write x on the tape and reset the tape head to the start
● So all in all we have a TM M’ which has x hard coded in its instructions</td></tr>
<tr><td>Since we know that the HP is undecidable, the BTHP must also be undecidable</td></tr>
<tr><td rowspan="2">Emptiness Problem (EP)</td><td>Semi-decide whether the language of a TM M’ is the empty set</td></tr>
<tr><td>Assume that we already have a TM K that semi-decides the EP. Create a TM N which semi-decides the LP (complement of HP) using a reduction from the LP to the EP.</td></tr>
</table>

<table>
<tr><td rowspan="4"></td><td>N on inputs M and x (e.g. as an encoding M#x):
1. Construct a new TM M’ from M
a. Ignore any input
b. Write x on the blank tape and reset the tape head
c. Execute M with x on the tape
d. Accept if the execution ends
2. Simulate K on input M’
3. Accept if K accepts</td></tr>
<tr><td> 
</td></tr>
<tr><td> 

If you are confused about M’:
● Similarly to the last example M’ will ignore any input and write x to the tape, so again our TM M’ comes with the x hard-coded
● Now, if M loops on x, then M’ will also loop and never accept. Therefore, L(M’) is the empty set.
● If M doesn’t loop on x, then M’ will accept and its language is clearly not equal to the empty set</td></tr>
<tr><td>Since we know that the LP is not semi-decidable, the EP cannot be semi-decidable as well.</td></tr>
<tr><td rowspan="3">Same Language Problem (SLP)</td><td>Decide whether TMs M1 and M2 accept the same language.</td></tr>
<tr><td>Assume that we already have a TM K that decides the SLP. Create a TM N which decides the HP using a reduction from HP to SLP.</td></tr>
<tr><td>N on inputs M and x (e.g. as an encoding M#x):</td></tr>
</table>

	1. Construct a new TM M1:
	  a. Simulate M on x
	  b. If simulation halts accept (no matter what the input is)
	2. Construct a new TM M2:
	  a. Accept (no matter what the input is)
	3. Simulate K on inputs M1 and M2
	4. Accept if K accepts, reject if K rejects

	This works because
	● M2 always accepts everything, so its language is Σ^*
	● M1 either accepts nothing or everything
	  ○ If the simulation halts, i.e. if M halts on x, it will accept everything, so its language is Σ^*
	  ○ If the simulation doesn't halt, then it will loop and run forever. Therefore its language is the empty set.
	● We can see that the language of M1 and M2 is the same if and only if M halts on x
	● So when we ask K if M1 and M2 have the same language it will accept if and only if M halts on x and reject otherwise
	● We can use this answer to solve the HP, which is a contradiction because we know that the HP is undecidable

	Since we know that the HP is not decidable, the SLP must be undecidable as well.

Properties of reductions

- <u>If A ≤ B and B is r.e. then so is A. Equivalently, if A is not r.e. then B is not r.e.</u>
- An example of this reduction rule is MP ≤ HP.
- If A ≤ B, then there's a reduction function from A to B: $f: A \rightarrow B$
- If B is r.e, then there exists a TM 'M' that accepts B.
- To prove A is r.e, we just need to define a TM that accepts A.
- This is that TM 'N':
  - On an input 'x', compute f(x)
  - Run 'M' on f(x)
  - If that accepts, make N accept
  - If that rejects, make N reject

- If A is not r.e, then there is no TM that accepts it.
- Let's just say a TM exists for B, called 'M'.
- Using the same TM we defined before (N), we can now define a TM to accept A, if the TM 'M' actually exists.
- This is a contradiction, so a TM for B cannot exist if A is not r.e., and therefore B is not r.e.

- If A ≤ B and B is recursive then so is A. Equivalently, if A isn't recursive then neither is B
- It's the same proof as before, except now, the TM 'M' halts on all inputs.
- If we follow through the same steps as before, our TM 'N' will not halt on any inputs either, because it uses 'M'. This means that A is recursive when B is.

- The same goes for B not being recursive if A isn't.
- If A is not recursive, then it's either r.e or not r.e. We've already proved the case for when A is not r.e, so let's prove the case for when A is r.e.
- Let's say that B is recursive and has a TM 'M' that halts on all inputs.
- If we have 'M', we can create a total TM 'N' that halts on all inputs and determines A.
- Therefore, B cannot be recursive if A is not recursive.

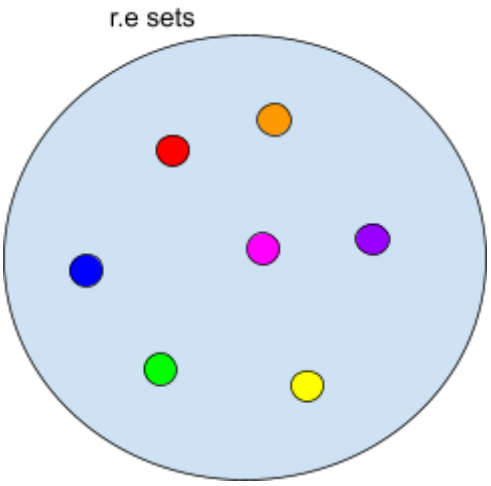
Rice's theorem

- Does Theory of Computing depress you?
- Well, this theorem is about to make it worse.

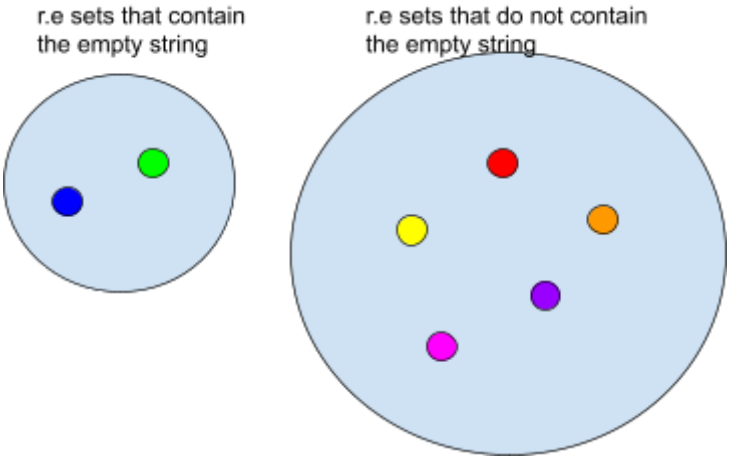
- Rice's theorem states that every non-trivial property of r.e sets is undecidable.

- So, what does this actually mean?
- Every language with a non-trivial property is constructed by a Turing Machine which loops.
- A r.e set is actually the same as a TM, but we're just referring to a TM by its language. By saying r.e set instead of TM, we ignore things like the TM's states, tape etc. and we only focus on the language it recognises.
- A property of an r.e set is a property of the TM's language. It's a predicate that tells you about what the TM recognises.
- A non-trivial property is a property that isn't always true or always false.
- So what Rice's theorem means is that if you partition the set of all r.e sets into two using some condition, both partitions will be undecidable.

- Let's visualise this. We have the set of all r.e sets:



- What if I were to split this set into two, using some property?
- Let's say the property is this: "the r.e set must contain the empty string".



- Now we know our sets aren't recursive anymore! This is due to Rice's theorem.
- Why are they not recursive?
- Remember the "Decidable / Undecidable problems" section where we proved that it's impossible to have a total TM that tells if M accepts the empty string?

- It doesn't stop at accepting the empty string. Any predicate you can think of that splits the set of all r.e sets are undecidable:
    - Accepts any string at all?
    - Accepts every string?
    - Accepts a given finite language?
    - Accepts a given regular language?
    - Accepts a context-free language?
    - Accepts a recursive language?
    - Accepts a language with the word "davidtyoember" (😍) in it?

- You may be thinking "Why keep using the term 'r.e set'? Why not just say any non-trivial property of a Turing machine is undecidable?"
- There's a reason why it's only a property of a TM's language and not a property of the TM itself.

- The following are properties of TMs, but not r.e sets:
    - M has less than 400 states (this isn't a property of an r.e set because 1. it queries nothing about the language and 2. you could find two TMs that recognise the same set, but one having less than 400 states and one having more than or equal to 400 states)
    - M halts on all inputs (this isn't a property of an r.e set because 1. it, again, queries nothing about the language and 2. you could have two TMs that recognise the same set, one halting on all inputs and one which doesn't)
- So if the concept of r.e sets is a little weird to you, then don't worry; just think about it in terms of TMs and their languages. Instead of a property of a r.e set, think of a property of a TM's language.

- Sure, because of Rice's theorem, there are lots of properties that are undecidable, like:
    - membership problem for TMs: does a given TM accept a given string?
    - emptiness problem for TMs: does a given TM accept any strings at all?
    - equivalence problem for TMs: given two TMs, do they accept the same language?
- ... but there are properties that are decidable:
    - membership problem for DFAs/NFAs: does a given DFA/NFA accept a given string?
    - emptiness problem for DFAs/NFAs: does a given DFA/NFA accept any strings at all?
    - equivalence problem for DFAs: given two DFAs, do they accept the same language?
    - membership problem for CFGs
    - emptiness problem for CFGs
- They're all decidable because simulations of NFAs, DFAs and CFGs are all decidable; you can't loop with any of those because with each transition, a bit of the string is being spent.

- If Rice's theorem depressed you, don't worry. You've come this far, and there's only one topic left to go. You can do it! I believe in you!

Complexity theory

- We know that Turing machines and algorithms are equivalent. However, how do we measure and compare the complexity of algorithms?
- Complexity theory is concerned with measuring how the time/space required by an algorithm/problem increases as the input gets larger
- You should remember Big-O notation from Algorithmics (I hope).
- In this module, we use Big-O and Big-Θ, and we use the classes P and NP to categorise problems.

Big-O and Big-Θ

- To define complexity in an algorithm or a problem, we use Big-O and Big-Θ notation.
- The complexity of a problem is the complexity of the best algorithm that currently exists for this problem.

Big-Θ

- **Informally (English):**
  - If an <u>algorithm</u> $f(n)$ is $\Theta(g(n))$ for some equation $g(n)$, that means that the algorithm's complexity scales with $g(n)$ and can only scale by $g(n)$, it can do no better than that.
  - For example, the average-case complexity of quick sort is $\Theta(n\ log(n))$.

  - If a <u>problem</u> $f(n)$ is $\Theta(g(n))$ for some equation $g(n)$, that means there exists no algorithm that can solve the problem $f(n)$ at a complexity better than $g(n)$.
  - In other words, it means $\Theta(g(n))$ is the complexity of the best known algorithm for $f(n)$.
  - For example, the sorting problem has the complexity $\Theta(n\ log\ n)$ because no sorting algorithm exists that does better than $n\ log\ n$ (except sleepsort, but that doesn't count).

- **Formally (weird maths language)**:
  - We say that $f(n)$ is $\Theta(g(n))$ for some $g(n)$ if there exist reals:
    - c > 0
    - d > 0
    - integer M > 0
  - with
    - $dg(n) \leq f(n) \leq cg(n)$ for all $n \geq M$.
  - We say that $f(n)$ and $g(n)$ have the same rate of growth.

Big-O

- **Informally (English):**

- If an algorithm $f(n)$ is $O(g(n))$ for some equation $g(n)$, that means that the algorithm's complexity scales, at worst, with $g(n)$, but it could do better in certain cases.
- For example, the worst-case complexity of bubble sort is $O(n^2)$.

- If a problem $f(n)$ is $O(g(n))$ for some equation $g(n)$, that means our best algorithm can solve this problem with complexity $g(n)$. There might exist an algorithm that does better, but we haven't discovered it yet.
- In other words, it means $O(g(n))$ is the complexity of the best known algorithm for $f(n)$.
- For example, if we had an algorithm that had a worst-case of $O(n\ log\ n)$, then that means the algorithm will, at worst, perform $n\ log\ n$, but may perform better on average-case or best-case.

- **Formally (weird maths language):**
  - We say that $f(n)$ is $O(g(n))$ for another function $g(n)$ if there exist reals:
    - c > 0
    - integer M > 0
  - with
    - $f(n) \leq cg(n)$ for all $n > M$.
  - We say that g(n) is an asymptotic upper bound for f(n)

The class P

- The class P is the set of all decision problems that can be solved by a deterministic Turing machine in polynomial time.
  - Decision problem → For all inputs we receive a "yes" or "no" answer
  - Deterministic Turing machine → A Turing machine that cannot make any choices in terms of the next move
    - This is the model of a Turing machine that we have seen so far
    - We will see a non-deterministic Turing machine later
  - Solved in polynomial time → The problem is feasible

- Some examples that are in P are
  - Searching an unordered list
    - Use sequential search, which is $O(n)$
  - Searching an ordered list
    - Use binary search, which is $O(log(n))$
  - Testing whether a list is sorted
    - Go through the list sequentially and compare each element with the element before, which is $\Theta(n)$

- An example that is not in P is: Given a first-order statement about non-negative integer variables (with only symbols 0, 1, +, = and logical operators allowed), is it true?

- We are going to look at some equation with non-negative integer variables, i.e. variables that can be assigned natural numbers
- On the left and right hand side of the equation we can only
  - use the variables
  - use the "+" operation
  - use the numbers "0", "1"
- We will use quantifiers to form the first-order logic statement, which is going to be either true or false
  - We can use the quantifiers "for all" and/or "there exists"
  - We must define one quantifier for each variable
- Problem instances are:
  - $\forall x \exists y (x + y = x + 1)$
    - For all x there exists y such that the equation holds
  - $\exists x \forall y \exists z (x + z = x + y + z)$
    - There exists x then for all y there exists z such that the equation holds
- This problem is known to be decidable, but infeasible
  - This means that we can always say whether this equation holds or not
  - However, this cannot be done in polynomial time with a deterministic machine

Feasible and infeasible problems

- A problem is said to be feasible if it has an algorithm with time complexity ≤ some polynomial f(n).
  - Examples: $\Theta(n)$, $\Theta(n\, log(n))$, $\Theta(\sqrt{n})$, $\Theta(n^{6})$
  - These are problems with polynomial time complexity
- Otherwise, a problem that can be solved in finite time but has no polynomial time algorithm is said to be infeasible
  - Examples: $\Theta(2^{n})$, $\Theta(1.3^{n})$, $\Theta(n!)$
  - These are problems with complexity greater than polynomial time (exponential and factorial time complexity)

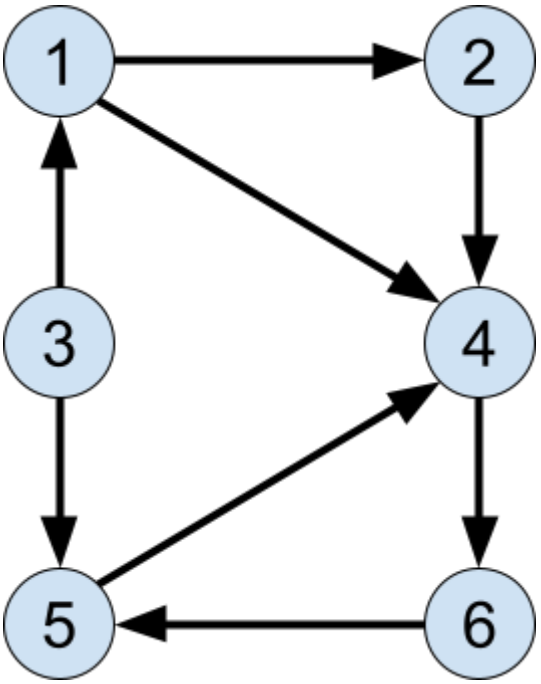
Decision problems

- A computational problem is called a decision problem if the output for any problem instance is either "yes" or "no".
  - A problem instance is a specific input for a problem
  - For example, the numbers "1", "4", "5", "8" are problem instances for an algorithm that determines whether a number is an even number or not
  - This algorithm is a decision problem because given a number this algorithm can say "yes this number is even" or "no this number is not even"
- We can classify the problem instances into positive/"yes" instances and negative/"no" instances

    - For example, “4” and “8” are positive instances and “1” and “5” are negative instances of the “is this number even” problem.
- The “yes” instances are forming a language. Therefore, solving the decision problem is the same as deciding this language
    - For example, the positive instances of the “is this number even” problem form the language of all even numbers.
    - $L_{\text{even numbers}} = \{..., -4, -2, 0, 2, 4, ...\}$

The PATH problem

- Let PATH be the following decision problem: Given a directed graph, does there exist a directed path connecting two given nodes?
- For example, given the following graph:



- Does there exist a path from 1 to 6?
    - Yes, 1 → 2 → 4 → 6
- Does there exist a path from 3 to 4?
    - Yes, 3 → 5 → 4
- Does there exist a path from 5 to 1?
    - No, from 5 we can only reach 4 and 6
- Does there exist a path from 1 to 3?
    - No, from 1 we can reach all nodes but 3

- A polynomial time algorithm for PATH is the following:
    - On input (G, s, t), where G is the graph, s is the start node and t is the goal node
    - Place a mark on node s
    - Repeat the following until no additional nodes are marked
        - For each edge from node a to b of G, if a is marked then also mark b

    - If t is marked then accept, otherwise reject
- This algorithm runs in polynomial time
    - The graph has n nodes and m edges
    - For each node n we have to mark and check at most m other nodes which are directly connected (via an edge) to n
    - Therefore, this algorithm runs in $O(n * m)$ time

The HAMPATH problem

- Let HAMPATH be the following decision problem: Given a directed graph, does there exist a directed path connecting two given nodes that goes through each node exactly once?
- This is an extension of PATH: We do not only have to find a path from one node to another node - the path also needs to go through each node exactly once.
- The best known algorithm for this problem is exponential, however, we do not know if there exists a polynomial time algorithm.
- Given a candidate for a HAMPATH, we can verify it in polynomial time.

Regular and context-free languages

- Every regular language is in P
    - If a language is regular then there exists a DFA that accepts the language
    - A Turing machine can simulate a DFA
    - It will have the input string and a symbol that marks the end of the string (end marker) on the tape
    - It will always move right after consuming a symbol and change state in the same way as the DFA would change state
    - When it reaches the end marker and it is in a state that corresponds to the accept state of the DFA it will accept, otherwise it will reject
    - Therefore, the Turing machine will execute exactly n steps (where n is the length of the input string) so it's time complexity is $\Theta(n)$

- Every context-free language is in P
    - For every context-free language there exists a corresponding context-free grammar.
    - For every context-free grammar, there exists another grammar which accepts the same language in Chomsky normal form.
    - Grammars in Chomsky can be parsed using the CYK (Cocke-Younger-Kasami) algorithm (citation), which parses in cubic time, which is polynomial.
    - One could create a Turing machine which applies the CYK algorithm to a given context-free grammar, and that Turing machine would accept said context-free language in cubic time.

The class NP

- The class NP is the set of all decision problems that can be solved by a non-deterministic Turing machine in polynomial time
- NP stands for non-deterministic polynomial

- Anything in P is also in NP. If a problem can be solved in polynomial time by a deterministic machine, then also by a non-deterministic one.

- Let's look at some NP problems which are believed not to be in P

Non-deterministic Turing machine

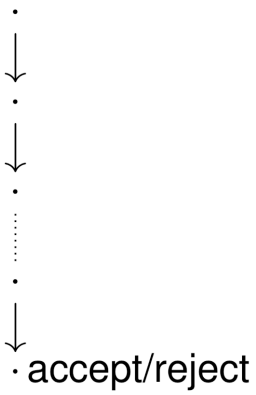
- A non-deterministic TM is of the same form as a deterministic TM:
- $M = (Q, \Sigma, \Gamma, \vdash, \sqcup, \delta, s, t, r)$

- However, the transition function is defined differently
  - $\delta: (Q - \{t, r\}) \times \Gamma \rightarrow P(Q \times \Gamma \times \{L, R\})$ is the transition function
  - Note that the output of the function is now a power set of configurations
  - This means that given a configuration, there are many possible other configurations in which the TM can be after making the transition
  - Therefore, the Turing machine becomes non-deterministic.

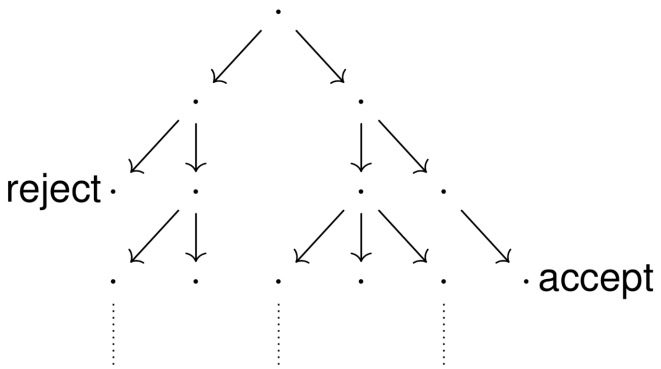
- We can think of the non-deterministic TM as a TM that makes all possible transitions from a given configuration in parallel
- Therefore, the computation of a non-deterministic TM is a tree, whose branches are all traversed in parallel
- In contrast, the computation of a deterministic TM is a linear path, as there is exactly one configuration after making a transition.
- Non-deterministic TMs have the same expressibility as deterministic TMs. We can simulate a non-deterministic TM with a multi-tape deterministic TM.
  - The deterministic TM has four tapes:
    - One to remember the input.
    - One to simulate the current state of the branch.
    - One to remember the branch being simulated on.
    - One to remember the branches simulated.
  - The deterministic TM accepts an input when one of the branches accepts, and rejects otherwise.

- A non-deterministic TM is a **decider** when all its branches halt on all inputs, similar to a total TM.

deterministic machine:



non-deterministic machine:



- Since we now have a tree of configurations, we need to re-think the condition when the non-deterministic TM accepts/rejects.
- Reminder: A deterministic TM
    - accepts/rejects as soon as it reaches a accept/reject configuration
    - loops if it never reaches such a configuration
- A non-deterministic TM
    - accepts if **some** branch of the computation leads to the accept state
    - rejects if **all** branches of the computation lead to the reject state
    - loops if none of those two conditions are ever met
        - For example, the tree can be infinite and without any accept state

- If all branches halt on all inputs, i.e. if the computation tree is finite, the non-deterministic TM is called a decider
    - This is similar to a deterministic total TM, which always halts on any input
    - The decider accepts an input if there exists a sequence of choices, where at least one ends in the accepting state.
    - Otherwise all choices will end in the reject state and the decider will reject the input

- Non-deterministic TMs are **<u>not</u> more powerful** than deterministic TMs as we are going to see shortly

- For any non-deterministic TM there exists a deterministic TM accepting the same language
    - A non-deterministic TM N can be simulated by a 3-tape deterministic TM D
    - D will try all branches of N's computation using breadth first search
        - If we use depth first search our simulation can get stuck in an infinite loop in case of an infinite tree
        - There might exist an accept state on another branch which we are unable to reach because of the infinite loop
        - In order to avoid this we have to use breadth first search
    - The 3 tapes are used as follows

        - One tape stores the input
        - One tape used to simulate a branch up to a given depth
        - One tape used to remember the branch being simulated
    - If D ever finds the accept state in the tree it will accept. Otherwise D will loop, or reject if all branches evaluate to a reject state.

- For any non-deterministic TM that halts on all branches (= decider) there exists a total TM deciding the same language
    - We can modify the proof from above such that if N halts on all branches then D halts
    - Therefore, we will introduce a fourth tape where we remember the branches that halt. We are not going to explore these branches again
    - If there is no branch left to explore, reject

- Therefore, we can conclude that any non-deterministic TM has an equivalent deterministic TM

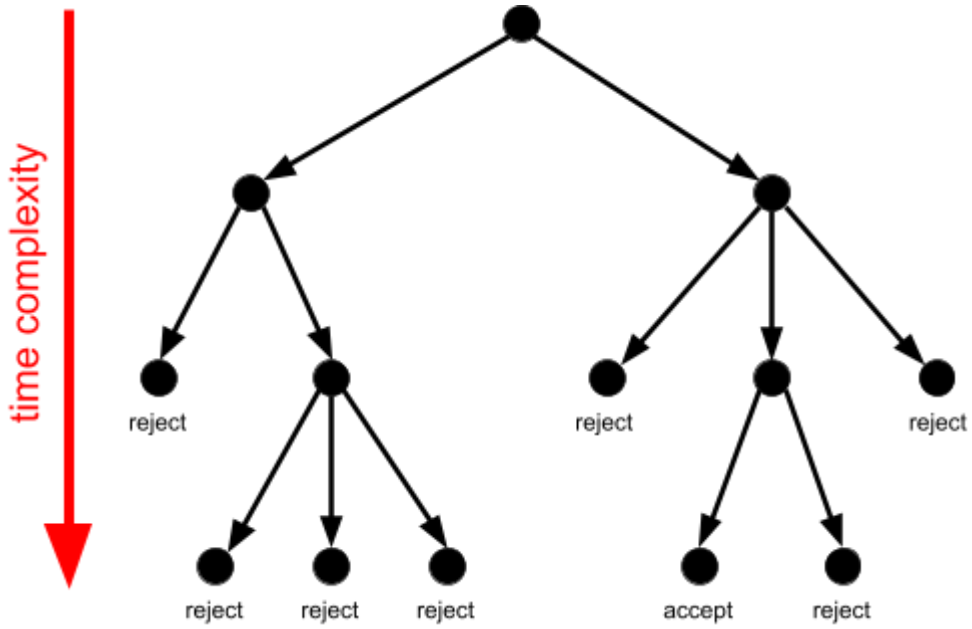
Time complexity of Turing machines

- Let's think about the time complexity of Turing machines

- First, we will define the following:
    - The size of a problem instance = the number of symbols on the input tape
    - The algorithm cost (measure of time taken) = number of steps the Turing machine takes

- The time complexity of a decider is a function f(n), where f(n) is the maximum number of steps the machine takes on any branch of its computation, on any input of length n.
    - In other words, to determine the time complexity of a decider, we are going to take a look at the computation tree for some input of length n

- The time complexity is equal to the depth of this tree



- Every $t(n)$ time multi-tape deterministic TM has an equivalent $O(t^2(n))$ time single-tape deterministic TM
  - If a multi-tape TM runs in $t(n)$time, it can only use $t(n)$ tape cells on each tape
  - We can construct a single-tape TM that simulates the multi-tape TM
  - Before simulating a single step of the multi-tape TM we need to find the position of the tape heads. These positions will be marked using special symbols on the tape.
  - Therefore, we are going to scan the tape from left to right, which takes an extra $O(t(n))$ steps. The tape heads might be anywhere on the tape but cannot be beyond the $t(n)$ tape cells, which is why we use big O notation here.
  - All in all, we have $t(n)$ steps to simulate, and for each step we can find the tape heads in $O(t(n))$ time, so all in all we require $O(t^2(n))$ time for the whole simulation.
  - The following is an example of a single-tape TM simulating a 3-tape TM. The position of the heads is marked with a "^" symbol:

⊢	⊢	a	**b̂**	c	d	e
	⊢	h	i	j	**k̂**	l
	⊢	**v̂**	w	x	y	z

- Every $t(n)$ time non-deterministic Turing machine has an equivalent $2^{O(t(n))}$ time deterministic Turing machine
  - Each branch of a $t(n)$ time non-deterministic TM N has length of at most $t(n)$.

    - Remember: the time complexity of a non-deterministic TM is defined as the depth of the computation tree
    - Therefore, the length of the longest branch is equal to the time complexity
- Each node in the computation tree of N can have up to b children.
    - This is also known as the branching factor of the tree.
- Therefore, there are at most $b^{t(n)}$ leaves, and $O(b^{t(n)})$ nodes in the computation tree of the non-deterministic TM N
    - A tree with branching factor b and depth d cannot have more than b^{d} nodes
    - A tree with branching factor b and depth d cannot have more than b^{d-1} leaves. Furthermore, it also cannot have more than b^{d} leaves, which is a less restrictive statement.
- We can construct a multi tape deterministic TM to simulate N. This will take $O(t(n)*b^{t(n)})$ time.
    - We need to simulate the path from the root to each of the leaves. Since we have at most $b^{t(n)}$ leaves we need to simulate at most $b^{t(n)}$ paths.
    - Each of the paths we are going to simulate is no longer than $t(n)$
    - Therefore the simulation will take no longer than $t(n)*b^{t(n)}$ time
- $t(n) * b^{t(n)}$ is $2^{O(t(n))}$ (see tutorial 8 question 2)
    - The big O notation is an upper bound and hides a constant c, therefore we have to show that there exists some $c > 0$ and $M \in \mathbb{N}$ such that $t(n) * b^{t(n)} \leq 2^{c*(t(n))}$ for all $n \geq M$
    - The proof for this is just below:

Proof from the tutorial that I don't understand

	$t(n)$	$\leq$	$2^{t(n)}$
	$t(n) * b^{t(n)}$	$\leq$	$2^{t(n)} * b^{t(n)}$
$=$	$(2b)^{t(n)}$	$\leq$	$(2^{b})^{t(n)}$
$=$	$2^{b*t(n)}$		

For all $n \geq 1$, therefore $c = b$ and $M = 1$

My proof that I fully understand

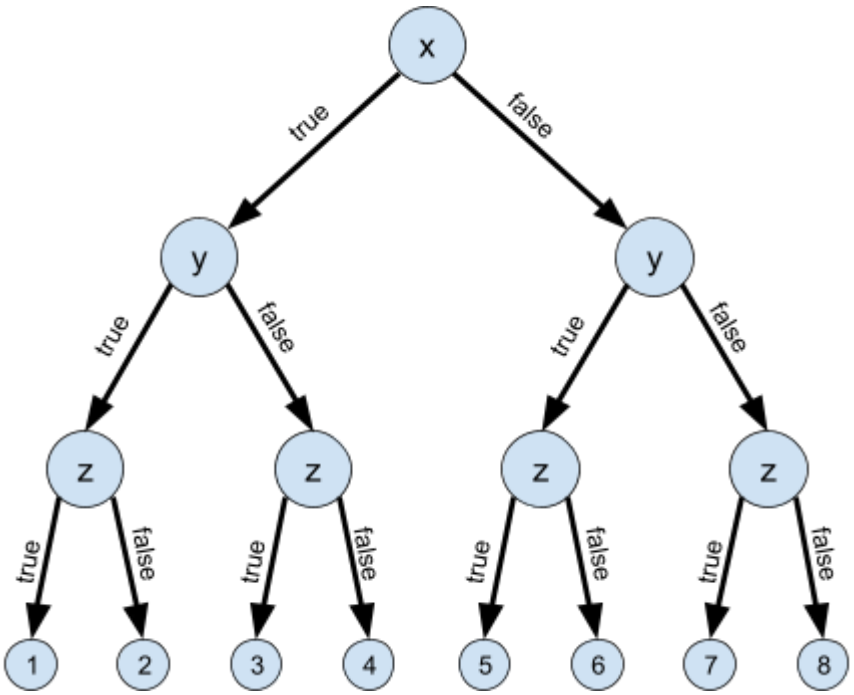
$t(n)$	$\leq$	$2^{t(n)}$
$t(n) * b^{t(n)}$	$\leq$	$2^{t(n)} * b^{t(n)}$
$t(n) * b^{t(n)}$	$\leq$	$2^{t(n)} * (2^{log_2(b)})^{t(n)}$
$t(n) * b^{t(n)}$	$\leq$	$2^{t(n)} * 2^{log_2(b) \times t(n)}$
$t(n) * b^{t(n)}$	$\leq$	$2^{log_2(b) \times t(n) + t(n)}$
$t(n) * b^{t(n)}$	$\leq$	$2^{(log_2(b) + 1) \times t(n)}$
$t(n) * b^{t(n)}$	$=$	$2^{O(t(n))}$

For all $n \geq 1$, therefore $c = log_2(b) + 1$ and $M = 1$

The SAT problem

- SAT: Boolean satisfiability
  - Given a boolean expression, does there exist an assignment of true and false (or 0s and 1s) to all variables such that the formula is satisfied?
  - Example: $(x \vee y \vee \neg z) \wedge (w \vee \neg z) \wedge (\neg w \vee x)$
  - Does there exist an assignment for the variables w, x, y, z such that the formula is satisfied?

- SAT is in NP: We can non-deterministically generate all possible assignments to the variables and then check if the formula is satisfied.
  - For each variable, we have two options: It can be either true or false
  - If we want to solve this problem in a deterministic way, we have to try out either true or false for each of the variables
  - Therefore, it will take 2^n time where n is the number of variables
  - However, a non-deterministic machine has the power to always guess the right assignment for each of the variable
  - For instance, if we have three variables x, y, z there are 8 different assignments for these variables
  - The non-deterministic machine will always pick the right assignment for each of the variables before moving on to the next variable

- We can visualise this in a computation tree. The non-deterministic machine will instantly pick the right path from the root to the leaf which makes the formula true (if such an assignment exists)
- Therefore, it can solve the problem in linear time (linear in the number of variables)
- 

- We don't know if SAT is in P because we currently do not have any polynomial time algorithm for it. However, no one has been able to prove that there isn't one.

The HAMPATH problem

- HAMPATH: Given a directed graph, does there exist a directed path connecting two given nodes that goes through each node exactly once?

- We can non-deterministically generate all sequences of nodes of length equal to the size of the graph, and then check the following:
    - No repetitions are found in the sequence
    - The start and end of the sequence coincide with the required nodes
    - The sequences defines a directed path through the graph

- We don't know if HAMPATH is in P

The TSP(D) problem

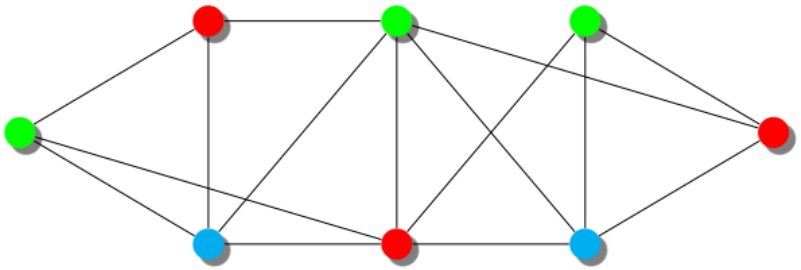
- TSP(D) refers to the Travelling Salesperson Problem.

- For this problem, you have a graph with weighted edges and an input D.
- Is there a route leading all around the graph and back to the start with a total distance at most D?

- This problem is in NP because the TM would have to 'guess' the order of cities to check in, then add up the distances to see if it's at most D.
- We don't know if TSP(D) is in P.

The 3COL problem

- 3COL: 3-colourability
- Given a graph G, is it possible to colour all the vertices with one of three available colours such that no two adjacent vertices are the same colour?
- Here is an example of a solution:



- This problem is in NP because we can non-deterministically guess the colour of each vertex; our computation splits into 3 branches of "is this node red?", "is this node blue?", or "is this node green?".
- We don't know if 3COL is in P.

NP = P?

- This is one of the biggest problems in computer science. Is NP equal for P? What if every problem in NP has a P algorithm, but we just haven't discovered them all yet?
- I don't know.

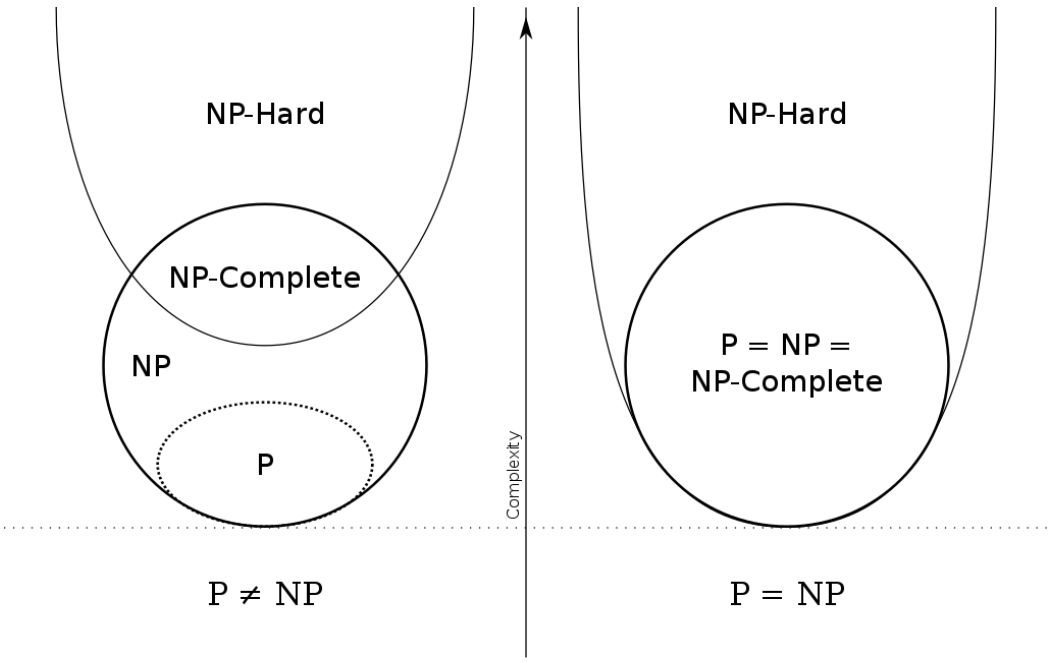
- But if P does equal NP, that means:
  - <u>P = NP = NP-complete</u>
  - To prove that P = NP-complete, we need to show that all problems in NP can reduce to all problems in P in polynomial time.
  - So we'll have some arbitrary problem X in P and some arbitrary problem Y in NP.
  - We will then reduce Y to X. When we do that, we'll have proved that all NP problems reduce to all P problems and that P = NP-complete, hence P = NP = NP-complete.

  - If Y is in NP, that means it's in P also (because P = NP).

- This proof only works if the language of X is non-trivial, meaning it's not the empty set or sigma star!
- If X is not trivial, that means there exists an input i_1 such that X will accept it, and an input i_2 such that X will reject it.
- Our reduction formula will remember this, and simulate an input y on Y. If Y accepts, then the formula returns i_1. If Y rejects, then the formula returns i_2.
- This is computable in polynomial time and maps yes/no instances of Y to yes/no instances of X, hence proving that if P = NP, then P = NP = NP-complete.

- <u>P ⊂ NP-hard</u>
- Following from the P = NP = NP-complete rule we proved earlier...
- Some NP-hard problems are NP-complete.
- If P = NP-complete, that means some NP-hard problems are also in P.
- Therefore, P is a strict subset of NP-hard.

- Here is what the problem sets will look like if P != NP and P = NP (just remember that trivial problems are in P and NP, just not NP-complete!):



Presburger arithmetic

- Here's a problem that's so hard that it's not even in NP. However, it is decidable (possible to solve).
- It goes like this:
- You have a set of variables, each with either a 'for all' for 'there exists'.
- There is also a formula that links up all the variables together.
- Is the formula true?

- If that explanation didn't make much sense, the problems look like this:

$$\forall x \exists y (x + y = x + 1)$$

$$\exists x \forall y \exists z (x + z = x + y + z)$$

$$\forall x \forall y \exists z (\neg(x = y) \Rightarrow (x + z = y))$$

- The question is: do these formulae hold?
- This is so hard that this is at least doubly exponential, which is $a^{(b^x)}$!

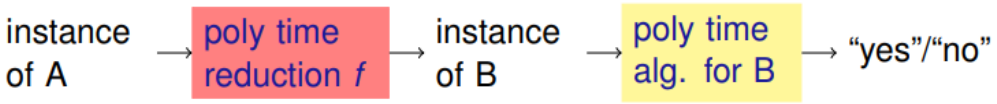
NP-Completeness

Polynomial time reduction

- Remember reduction with Turing machines?
- We can do the same here; we can reduce a problem to another problem.

- If we can reduce problem A to problem B, then any polynomial algorithm for B also applies for A, right?

- Almost! The reduction needs to be in polynomial time as well. This is called a polynomial time reduction.
- So to solve problem A, we need to reduce it to B in polynomial time, then solve B in polynomial time. Polynomial + polynomial = polynomial, so solving A would also be in polynomial time.
- If you're a visual learner, maybe this diagram will help (it goes from left to right):



- Polynomial time reduction is written like this:
- $A \leq_P B$

- More formally, it's defined like this:
- A function $f: \Sigma^* \to \Delta^*$ is polynomial time computable if there exists a polynomial time deterministic Turing machine that halts on each input $w \in \Sigma^*$ with just $f(w)$ on its tape.

- From this, we can conclude that if A reduces to B in polynomial time, and B is in P, then A is in P.
- Additionally, if A reduces to B in polynomial time, and A is not in P, then B is not in P.

3COL to SAT

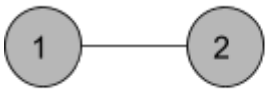
- We can reduce 3COL (3-colourability) to SAT (boolean satisfiability).

- We can do this by:
  - for each node ‘x’ in 3COL...
  - ... create 3 variables in SAT, rx (is ‘x’ red?), gx (is ‘x’ green?), and bx (is ‘x’ blue?).
- After we’ve created our variables, we need to define three expressions:
  1. For each vertex ‘x’, the expression:
     `(rx OR gx OR bx)`
     must be true (each vertex must be assigned a colour)
  2. For each vertex ‘x’, the expression:
     `( !(rx AND gx) AND !(rx AND bx) AND !(bx AND gx) )`
     must be true (each vertex may only have ONE colour)
  3. For each edge ‘x → y’, the expression:
     `!(rx AND ry) AND !(gx AND gy) AND !(bx AND by)`
     must be true (no two adjacent vertices should have the same colour)

- All these expressions are ‘AND’ed together, and that forms the final expression for SAT.

- Here’s a basic example:

3COL



SAT

```
(r1 OR g1 OR b1) AND (r2 OR g2 OR b2)

AND

( !(r1 AND g1) AND !(r1 AND b1) AND !(b1 AND g1) ) AND
( !(r2 AND g2) AND !(r2 AND b2) AND !(b2 AND g2) )

AND

!(r1 AND r2) AND !(g1 AND g2) AND !(b1 AND b2)
```

- So our 3COL problem is just two nodes with one edge joining them.
- The first block in the SAT expression makes sure that each node has a colour.
- The second block in the SAT expression makes sure each node has only ONE colour.
- The third block in the SAT expression makes sure that nodes 1 and 2 don’t share the same colour.

- Great, but is it in polynomial time?
- The number of characters in the output SAT problem is:
- $7m + 20m + 20l + (2m + l - 1)$
- Where $m = \#vertices$ and $l = \#edges$. I don’t know about you, but that looks like a polynomial to me.

- By applying this method, we can convert any 3COL problem to SAT.
- That means if we solve SAT in polynomial time, we can solve 3COL in polynomial time.
- Conversely, if we find that 3COL is not in P, then SAT cannot be in P.

NP-hard

- What's NP-hard?
- NP-hard is a set of problems where any NP problem can reduce to any NP-hard problem in polynomial time.
- This shows that NP-hard problems are, in a sense, hard, because every NP problem can reduce to it.

The Cook-Levin theorem

- The Cook-Levin theorem states that any NP problem can reduce to SAT in polynomial time.
- That means that SAT is an NP-hard problem, because every NP problem can reduce to it.
- This also means that if we can solve SAT in polynomial time, then every NP problem will be solvable in polynomial time, and NP will be equal to P.

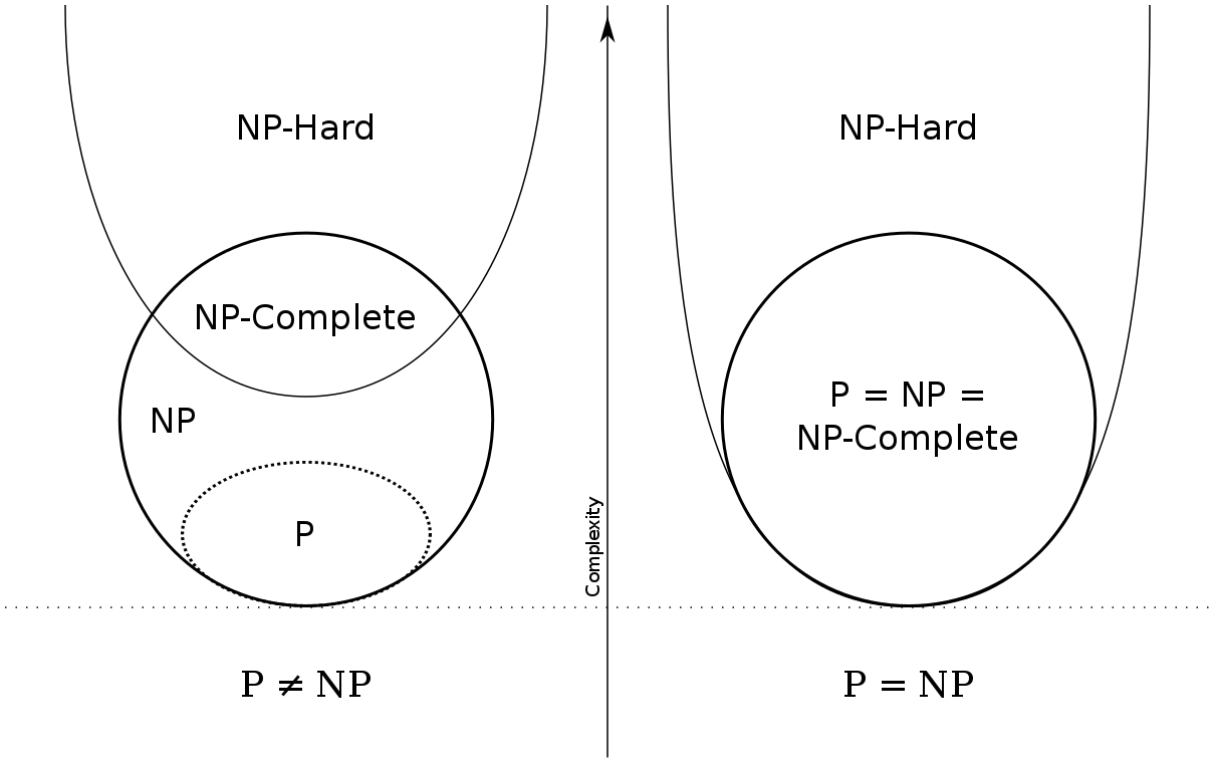
- To convert an NP problem X into SAT, you need to 'encode' the parameters of X into boolean variables and construct an expression for SAT.
- There's a proof for it, but it's not examinable. Basically, it's about converting a Turing machine into a boolean expression.

What is NP-completeness

- An NP-complete problem is a problem in NP and NP-hard.
- SAT is an example of an NP-complete problem, because it can be solved by a non-deterministic Turing machine and every other NP problem can be reduced to it in polynomial time.

- To find another NP-complete problem, you just need to have a problem X that is in NP, then find a reduction function from SAT to X.

- Just remember: if you solve an NP-complete problem in polynomial time, you've solved all NP problems in polynomial time, too, proving P = NP.



Space complexity

Space complexity

- First of all, what is space complexity?
- The space complexity of a decider is the function $f: N \to N$ where $f(n)$ is the maximum number of tape cells the machine scans on any branch of its computation, on any input of length n.
- Basically, what's the maximum number of tape cells we need to solve this problem?

- Here's a theorem for you:
- Assume $f(n) \geq n$. If a decider runs in $f(n)$ space, then it runs in $2^{O(f(n))}$ time.

- Why?
- Well, think of all the different possible combinations the Turing tape could be:

1	5	8	3	6	...

8	3	2	9	5	...

2	7	3	8	0	...

- There's lots of possibilities the tape could be!
- What are all the possible combinations of the tape?
- Well, first of all, we need to know how long the tape is. We already know that! It's the space complexity of the algorithm, which is f(n). That's how many tape cells we need to solve the problem.
- Let's just say that our decider has to run through every possible tape configuration to solve the problem.

- We need a bit of combinatorics from Foundations here; what are all the possible combinations of this tape?
- For each position on the tape, there are 'm' possibilities, where 'm' is the number of symbols available to put on the tape.
- So, we multiply for each tape cell, m * m * m * m ... f(n) times. This yields us:
- $m^{f(n)}$

- We still have a ways to go!
- We have our base 'm'. However, with a little mathsy magic, we can generalise this further and make our base '2':
- What's the opposite of exponential? That's right; it's logarithms!
- We can simply take the logarithm of 'm' and convert the base to a '2'!
- $(2^{log_2(m)})^{f(n)}$
- Still remember your GCSE maths? When you have a power to the power, you multiply the powers.
- $2^{log_2(m) \times f(n)}$
- Wait a minute... $log_2(m)$ is just a constant. It could be anything, depending on what 'm' is! Look familiar? That's the constant 'c' found in Big-O notation! Therefore, we can introduce Big-O notation to generalise that constant.
- $2^{O(f(n))}$
- As you can see, we have resulted in $2^{O(f(n))}$, thereby proving the link between time and space complexity.

- What does this mean?
- It means the time complexity is 2 to the power of O(space complexity).
- Remember our assumption: the decider checks all possible combinations of tape. Therefore, this is simply an "upper bound". In other words, a decider of space complexity linear or above will have at worst exponential time complexity.

PSPACE

- PSPACE is the set of all decision problems that can be solved by a deterministic Turing machine in polynomial space.

NPSPACE

- NPSPACE is the set of all decision problems that can be solved by a non-deterministic Turing machine in polynomial space.
- Anything in PSPACE is in NPSPACE, and anything in NPSPACE is in PSPACE *(see Savitch's Theorem, which is literally just below)*

Savitch's Theorem

- Here's another theorem, called Savitch's theorem:
- For $f(n) \geq n$, every $f(n)$ space non-deterministic Turing machine has an equivalent $f(n)^2$ space deterministic Turing machine.
- This means that **PSPACE = NPSPACE**.
- There's a proof for this, but it's not examinable.

EXPTIME

- The class EXPTIME is the set of all decision problems that can be solved by a deterministic Turing machine in time $O(2^{n^k})$ for some $k \in N$.

Relationships between complexity classes

- P ⊆ PSPACE: a machine that runs in polynomial time can only use polynomial space
- NP ⊆ NPSPACE: same reason
- Hence, using Savitch's theorem, NP ⊆ PSPACE
- PSPACE ⊆ EXPTIME:
    - If a decider runs in f(n) space, then it runs in $2^{O(f(n))}$ time.

- We don't know which inclusion is strict.
- All we know is P ⊂ EXPTIME.

- **Note**: One can also define EXPSPACE. We know that
    - EXPTIME ⊆ EXPSPACE
    - PSPACE ⊂ EXPSPACE

PSPACE-hard

- A problem X is PSPACE-hard if every problem A in PSPACE is polynomial-time reducible to X.
- Examples of PSPACE-hard problems are Chess and Go.

- Why not space-reducible?
- We defined it like that because we want reductions to be easy compared to problems in NP!

- If we used polynomial space reduction from A to B, then found an easy solution for B, it could potentially be factorial time complexity for A!
- However, because we use time-reducible, it has to be both polynomial time **and** polynomial space (because P ⊆ PSPACE).

PSPACE-completeness

- PSPACE-complete simply means that a problem is both PSPACE and PSPACE-hard.

- An example of a PSPACE-complete problem is TQBF.
- Let TQBF be the problem of deciding whether a fully quantified boolean formula is true.
- Examples of fully quantified boolean formulas:
  - $\forall x \forall y (x \vee y)$
  - $\forall x \exists y ((x \vee y) \wedge (\neg x \vee \neg y))$
- Are there formulas true for all cases? Yes or no? That is the TQBF problem.

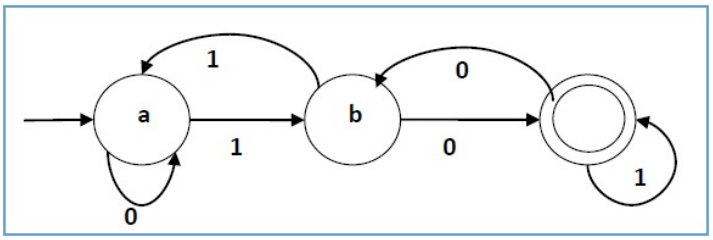
TL;DR

- Can't be bothered to read all these notes?
- Need a bite-sized form of these notes?
- The TL;DR section has your back!

Automata theory: Regular languages

- **Alphabet**: a set of symbols
- **Language**: any string that can be made with an alphabet

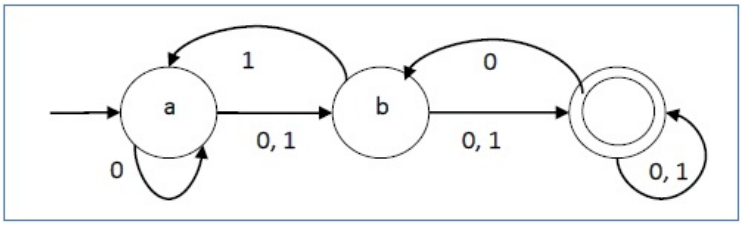
- DFAs start at the initial state, then goes through transitions (each character in the string) until it reaches an accept state.
- DFAs cannot guess; they must discretely be in only one state at a time, and each state must have all possible types of transitions from it



- DFAs either accept a string or reject a string

- DFAs are deterministic, NFAs are non-deterministic

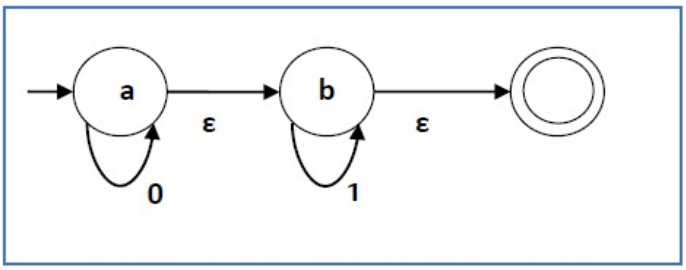
- NFAs can make guesses



- NFAs and DFAs have the same power

- The set of all strings that a DFA / NFA can accept is called it's language

- εNFAs have epsilon-moves, which are free moves that do not cost any symbols in the string



Finite automata with Null Moves

- A language accepted by some DFA or NFA is called 'regular'

- Regular expressions have the same power as DFAs and NFAs

$$((a \cdot b) + c)^*$$

- "a*" means 0 or more repetitions of a
- "a + b" means a or b
- "ab" means a followed by b

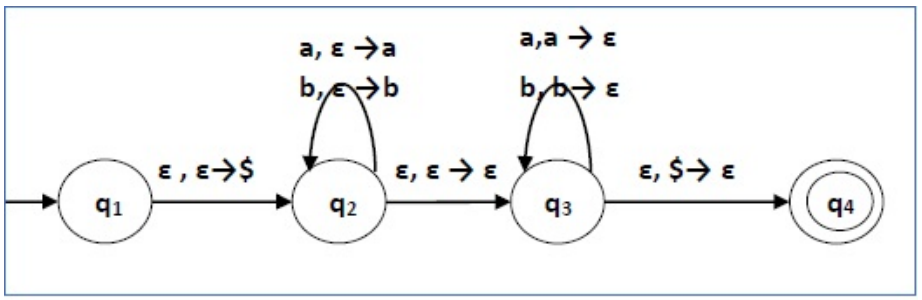
- You can always convert an NFA/DFA to regex or vice versa

- You can prove a language is regular by constructing a DFA / NFA / regex
- You can prove a language is not regular by using the pumping lemma

Automata theory: Context free languages

- A language is context free if it can be accepted by a PDA or CFG

- Pushdown automata (PDA) are like DFAs and NFAs, but you get a stack.



PDA for L= {ww^R | w = (a+b)*}

- Each transition, an element is popped or pushed onto the stack

- PDAs accept by final state or by empty stack

- Context free grammars are just as powerful as PDAs

$$
\begin{aligned}
R &\rightarrow XRX \mid S \\
S &\rightarrow \mathtt{a}T\mathtt{b} \mid \mathtt{b}T\mathtt{a} \\
T &\rightarrow XTX \mid X \mid \varepsilon \\
X &\rightarrow \mathtt{a} \mid \mathtt{b}
\end{aligned}
$$

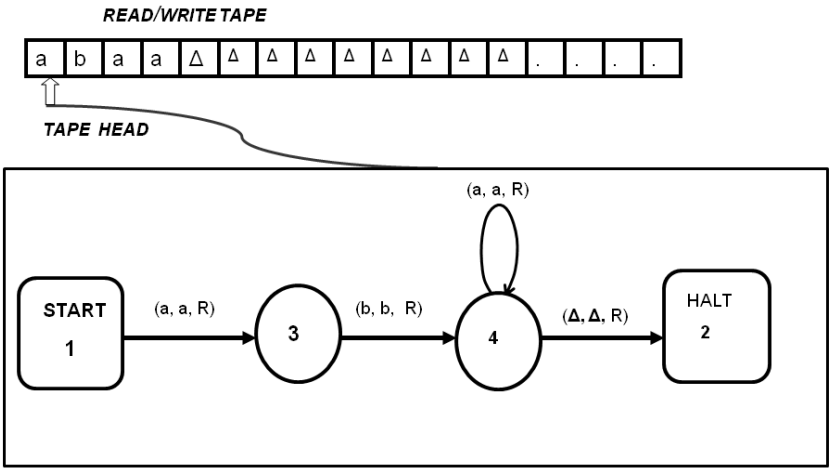
- There are two forms of CFGs:
- Chomsky normal form
- $A \rightarrow BC$
  $A \rightarrow \alpha$
- Greibach normal form
- $S \rightarrow bC$
  $C \rightarrow c$

- You can always convert a CFG to either Chomsky or Greibach
- You can always convert a CFG to PDA or vice versa

- You can prove a language isn't context-free using the pumping lemma

Computability theory

- A Turing machine is like a PDA, but instead of a stack you get tape. You can move left or right on the tape, write to the tape and read from the tape.



A Turing Machine for aba*

- They are the closest mathematical abstraction of an actual computer.

- A language is recursively enumerable (r.e) if it can be accepted by a Turing machine.
- A language is recursive if a Turing machine can accept and reject strings from that language without looping.

- Multi-tape TMs are just as powerful as single-tape TMs

- A Universal Turing machine can simulate a Turing machine with a given input

- The Halting problem: Is there a Turing machine that can accept if an input halts on a TM and reject if an input loops on a TM?
- It's impossible. There is a diagonalisation proof for this.

- The Halting problem is 'undecidable'.

- The membership problem is also 'undecidable', because if we had a TM to solve the membership problem, we could use that to solve the Halting problem.

- **Reduction**: a computable function that maps inputs of one problem to another.

- Rice's theorem states that any property about a Turing machine's language is undecidable:
    - You can't tell if a TM accepts the empty string
    - You can't tell if a TM accepts any string
    - You can't tell if a TM accepts all the strings
    - etc.

Complexity theory

- Big-O represents the best algorithm we currently have; we could do better but this is the best we have
- Big-Θ represents that an algorithm can scale no better than this

- Time complexity is the number of steps it takes for a Turing machine to solve a problem.

- The class P is the set of all decision problems that can be solved by a deterministic Turing machine in polynomial time

- The class NP is the set of all decision problems that can be solved by a non-deterministic Turing machine in polynomial time.

- The class NP-hard is the set of all problems such that all the NP problems can reduce into any NP-hard problem in polynomial time.

- The class NP-complete is the set of all problems such that they are in both NP and NP-hard.

- If you find a (deterministic) polynomial algorithm for an NP-complete problem, then P = NP.
- If P = NP, then:
    - P = NP = NP-complete
    - P ⊂ NP-hard

- Space complexity is the minimum number of tape cells required for a Turing machine to solve a problem.

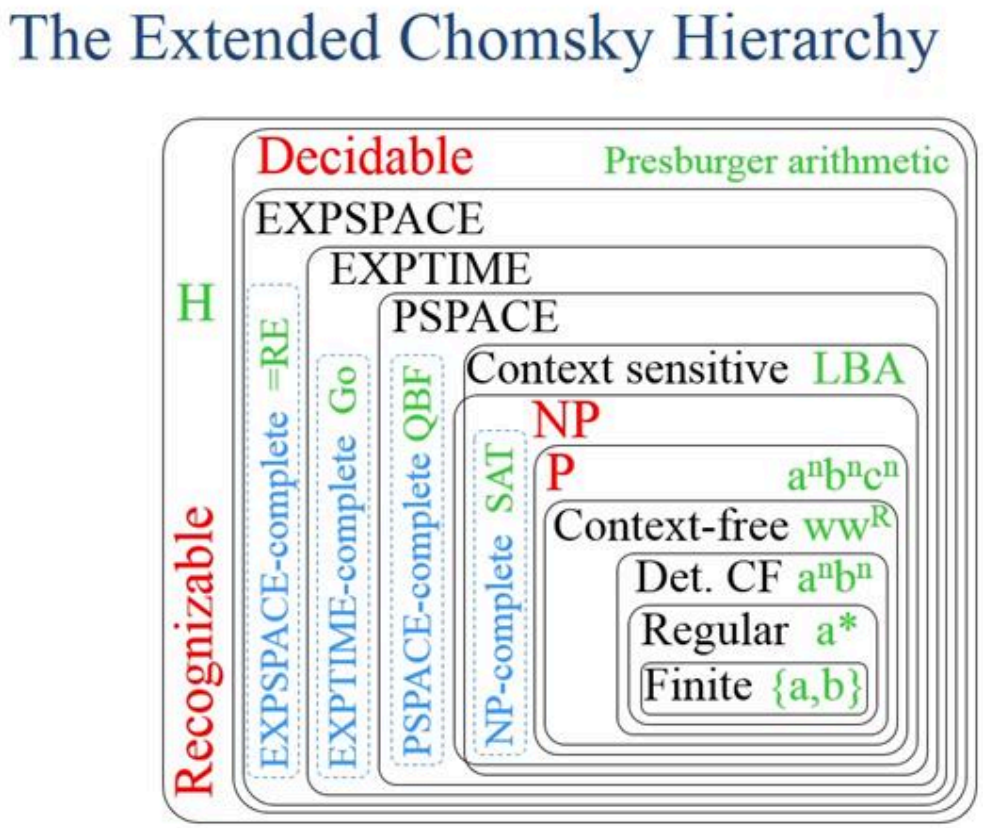
- PSPACE is the set of all decision problems that can be solved by a deterministic Turing machine in polynomial space.

- NPSPACE is the set of all decision problems that can be solved by a non-deterministic Turing machine in polynomial space.

- Savitch's Theorem states that for $f(n) \geq n$, every $f(n)$ space non-deterministic Turing machine has an equivalent $f(n)^2$ space deterministic Turing machine.
- Therefore, PSPACE = NPSPACE.

- EXPTIME is the set of all decision problems that can be solved by a deterministic Turing machine in time $O(2^{n^k})$ for some $k \in N$.

- PSPACE = NPSPACE
- NP ⊆ PSPACE
- PSPACE ⊆ EXPTIME
- P ⊂ EXPTIME

- PSPACE-hard is the set of all problems such that all the PSPACE problems can reduce into any PSPACE-hard problems in polynomial time.

- PSPACE-complete is the set of all problems that are in PSPACE and PSPACE-hard.